There are many situations where the list is built up by insertions at the high end, and then only array accesses (i.e., `findKth` operations) occur. In such a case, the array is a suitable implementation. However, if insertions and deletions occur throughout the list and, in particular, at the front of the list, then the array is not a good option. The next section deals with the alternative: the *linked list*.

3.2.2 Simple Linked Lists

In order to avoid the linear cost of insertion and deletion, we need to ensure that the list is not stored contiguously, since otherwise entire parts of the list will need to be moved. Figure 3.1 shows the general idea of a **linked list.**

The linked list consists of a series of nodes, which are not necessarily adjacent in memory. Each node contains the element and a link to a node containing its successor. We call this the `next` link. The last cell's `next` link points to `nullptr`.

To execute `printList()` or `find(x)`, we merely start at the first node in the list and then traverse the list by following the `next` links. This operation is clearly linear-time, as in the array implementation; although, the constant is likely to be larger than if an array implementation were used. The `findKth` operation is no longer quite as efficient as an array implementation; `findKth(i)` takes $O(i)$ time and works by traversing down the list in the obvious manner. In practice, this bound is pessimistic, because frequently the calls to `findKth` are in sorted order (by i). As an example, `findKth(2)`, `findKth(3)`, `findKth(4)`, and `findKth(6)` can all be executed in one scan down the list.

The `remove` method can be executed in one `next` pointer change. Figure 3.2 shows the result of deleting the third element in the original list.

The `insert` method requires obtaining a new node from the system by using a `new` call and then executing two `next` pointer maneuvers. The general idea is shown in Figure 3.3. The dashed line represents the old pointer.

As we can see, in principle, if we know where a change is to be made, inserting or removing an item from a linked list does not require moving lots of items, and instead involves only a constant number of changes to node links.

The special case of adding to the front or removing the first item is thus a constant-time operation, presuming of course that a link to the front of the linked list is maintained.

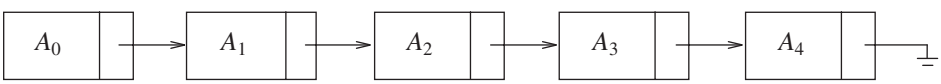


Figure 3.1 A linked list

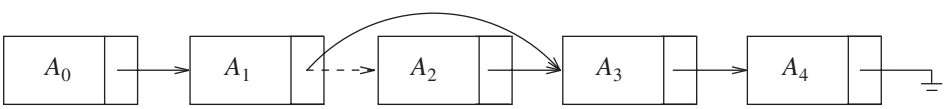


Figure 3.2 Deletion from a linked list

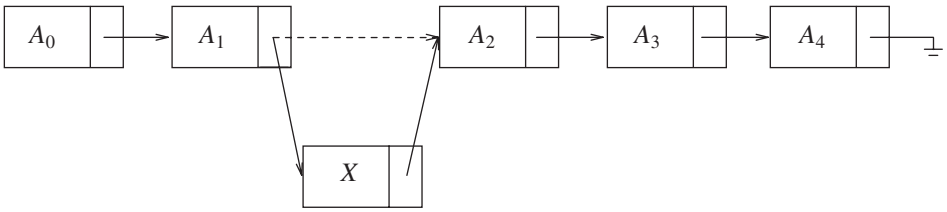


Figure 3.3 Insertion into a linked list

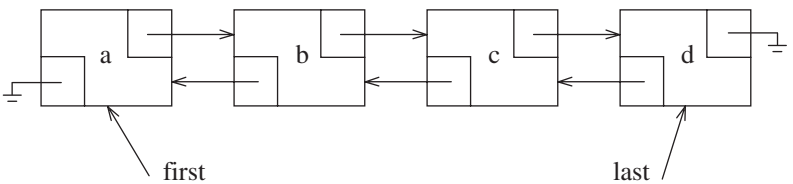


Figure 3.4 A doubly linked list

The special case of adding at the end (i.e., making the new item the last item) can be constant-time, as long as we maintain a link to the last node. Thus, a typical linked list keeps links to both ends of the list. Removing the last item is trickier, because we have to find the next-to-last item, change its *next* link to `nullptr`, and then update the link that maintains the last node. In the classic linked list, where each node stores a link to its next node, having a link to the last node provides no information about the next-to-last node.

The obvious idea of maintaining a third link to the next-to-last node doesn't work, because it too would need to be updated during a remove. Instead, we have every node maintain a link to its previous node in the list. This is shown in Figure 3.4 and is known as a **doubly linked list.**

3.3 `vector` and `list` in the STL

The C++ language includes, in its library, an implementation of common data structures. This part of the language is popularly known as the **Standard Template Library** (STL). The List ADT is one of the data structures implemented in the STL. We will see some others in Chapters 4 and 5. In general, these data structures are called **collections** or **containers**.

There are two popular implementations of the List ADT. The `vector` provides a growable array implementation of the List ADT. The advantage of using the `vector` is that it is indexable in constant time. The disadvantage is that insertion of new items and removal of existing items is expensive, unless the changes are made at the end of the `vector`. The `list` provides a doubly linked list implementation of the List ADT. The advantage of using the

list is that insertion of new items and removal of existing items is cheap, provided that the position of the changes is known. The disadvantage is that the list is not easily indexable. Both vector and list are inefficient for searches. Throughout this discussion, list refers to the doubly linked list in the STL, whereas list (typeset without the monospace font) refers to the more general List ADT.

Both vector and list are class templates that are instantiated with the type of items that they store. Both have several methods in common. The first three methods shown are actually available for all the STL containers:

- `int size( ) const`: returns the number of elements in the container.
- `void clear( )`: removes all elements from the container.
- `bool empty( ) const`: returns true if the container contains no elements, and false otherwise.

Both vector and list support adding and removing from the end of the list in constant time. Both vector and list support accessing the front item in the list in constant time. The operations are:

- `void push_back( const Object & x )`: adds x to the end of the list.
- `void pop_back( )`: removes the object at the end of the list.
- `const Object & back( ) const`: returns the object at the end of the list (a mutator that returns a reference is also provided).
- `const Object & front( ) const`: returns the object at the front of the list (a mutator that returns a reference is also provided).

Because a doubly linked list allows efficient changes at the front, but a vector does not, the following two methods are available only for list:

- `void push_front( const Object & x )`: adds x to the front of the list.
- `void pop_front( )`: removes the object at the front of the list.

The vector has its own set of methods that are not part of list. Two methods allow efficient indexing. The other two methods allow the programmer to view and change the internal capacity. These methods are:

- `Object & operator[] ( int idx )`: returns the object at index idx in the vector, with no bounds-checking (an accessor that returns a constant reference is also provided).
- `Object & at( int idx )`: returns the object at index idx in the vector, with bounds-checking (an accessor that returns a constant reference is also provided).
- `int capacity( ) const`: returns the internal capacity of the vector. (See Section 3.4 for more details.)
- `void reserve( int newCapacity )`: sets the new capacity. If a good estimate is available, it can be used to avoid expansion of the vector. (See Section 3.4 for more details.)

3.3.1 Iterators

Some operations on lists, most critically those to insert and remove from the middle of the list, require the notion of a position. In the STL, a position is represented by a nested type, `iterator`. In particular, for a `list<string>`, the position is represented by the type `list<string>::iterator`; for a `vector<int>`, the position is represented by a class `vector<int>::iterator`, and so on. In describing some methods, we'll simply use `iterator` as a shorthand, but when writing code, we will use the actual nested class name.

Initially, there are three main issues to address: first, how one gets an iterator; second, what operations the iterators themselves can perform; third, which List ADT methods require iterators as parameters.

Getting an Iterator

For the first issue, the STL lists (and all other STL containers) define a pair of methods:

- `iterator begin( )`: returns an appropriate iterator representing the first item in the container.
- `iterator end( )`: returns an appropriate iterator representing the endmarker in the container (i.e., the position after the last item in the container).

The `end` method seems a little unusual, because it returns an iterator that is "out-of-bounds." To see the idea, consider the following code typically used to print the items in a `vector v` prior to the introduction of range-based `for` loops in C++11:

```
for( int i = 0; i != v.size( ); ++i )
    cout << v[ i ] << endl;
```

If we were to rewrite this code using iterators, we would see a natural correspondence with the `begin` and `end` methods:

```
for( vector<int>::iterator itr = v.begin( ); itr != v.end( ); itr.??? )
    cout << itr.??? << endl;
```

In the loop termination test, both `i!=v.size( )` and `itr!=v.end( )` are intended to test if the loop counter has become "out-of-bounds." The code fragment also brings us to the second issue, which is that the iterator must have methods associated with it (these unknown methods are represented by ???).

Iterator Methods

Based on the code fragment above, it is obvious that iterators can be compared with `!=` and `==`, and likely have copy constructors and `operator=` defined. Thus, iterators have methods, and many of the methods use operator overloading. Besides copying, the most commonly used operations on iterators include the following:

- `itr++` and `++itr`: advances the iterator `itr` to the next location. Both the prefix and postfix forms are allowable.

- `*itr`: returns a reference to the object stored at iterator `itr`'s location. The reference returned may or may not be modifiable (we discuss these details shortly).
- `itr1==itr2`: returns true if iterators `itr1` and `itr2` refer to the same location and false otherwise.
- `itr1!=itr2`: returns true if iterators `itr1` and `itr2` refer to a different location and false otherwise.

With these operators, the code to print would be

```
for( vector<int>::iterator itr = v.begin( ); itr != v.end( ); ++itr )
    cout << *itr << endl;
```

The use of operator overloading allows one to access the current item, then advance to the next item using `*itr++`. Thus, an alternative to the fragment above is

```
vector<int>::iterator itr = v.begin( );
while( itr !=v.end( ) )
    cout << *itr++ << endl;
```

Container Operations That Require Iterators

For the last issue, the three most popular methods that require iterators are those that add or remove from the list (either a `vector` or `list`) at a specified position:

- `iterator insert( iterator pos, const Object & x )`: adds `x` into the list, prior to the position given by the iterator `pos`. This is a constant-time operation for `list`, but not for `vector`. The return value is an iterator representing the position of the inserted item.
- `iterator erase( iterator pos )`: removes the object at the position given by the iterator. This is a constant-time operation for `list`, but not for `vector`. The return value is the position of the element that followed `pos` prior to the call. This operation invalidates `pos`, which is now stale, since the container item it was viewing has been removed.
- `iterator erase( iterator start, iterator end )`: removes all items beginning at position `start`, up to, but not including `end`. Observe that the entire list can be erased by the call `c.erase( c.begin( ), c.end( ) )`.

3.3.2 Example: Using `erase` on a List

As an example, we provide a routine that removes every other item in a list, starting with the initial item. Thus if the list contains 6, 5, 1, 4, 2, then after the method is invoked it will contain 5, 4. We do this by stepping through the list and using the `erase` method on every second item. On a `list`, this will be a linear-time routine because each of the calls to `erase` takes constant time, but in a `vector` the entire routine will take quadratic time because each of the calls to `erase` is inefficient, using $O(N)$ time. As a result, we would normally write the code for a `list` only. However, for experimentation purposes, we write a general function template that will work with both a `list` or a `vector`, and then provide

```
1  template <typename Container>
2  void removeEveryOtherItem( Container & lst )
3  {
4      auto itr = lst.begin( );     // itr is a Container::iterator
5
6      while( itr != lst.end( ) )
7      {
8          itr = lst.erase( itr );
9          if( itr != lst.end( ) )
10             ++itr;
11     }
12 }
```

Figure 3.5 Using iterators to remove every other item in a List (either a `vector` or `list`). Efficient for a `list`, but not for a `vector`.

timing information. The function template is shown in Figure 3.5. The use of `auto` at line 4 is a C++11 feature that allows us to avoid the longer type `Container::iterator`. If we run the code, passing a `list<int>`, it takes 0.039 sec for a 800,000-item `list`, and 0.073 sec for an 1,600,000-item `list`, and is clearly a linear-time routine, because the running time increases by the same factor as the input size. When we pass a `vector<int>`, the routine takes almost five minutes for an 800,000-item `vector` and about twenty minutes for an 1,600,000-item `vector`; the four fold increase in running time when the input increases by only a factor of two is consistent with quadratic behavior.

3.3.3 const_iterators

The result of `*itr` is not just the value of the item that the iterator is viewing but also the item itself. This distinction makes the iterators very powerful but also introduces some complications. To see the benefit, suppose we want to change all the items in a collection to a specified value. The following routine works for both `vector` and `list` and runs in linear time. It's a wonderful example of writing generic, type-independent code.

```
template <typename Container, typename Object>
void change( Container & c, const Object & newValue )
{
    typename Container::iterator itr = c.begin( );
    while( itr != c.end( ) )
        *itr++ = newValue;
}
```

To see the potential problem, suppose the `Container c` was passed to a routine using call-by-constant reference. This means we would expect that no changes would be allowed to `c`, and the compiler would ensure this by not allowing calls to any of `c`'s mutators. Consider the following code that prints a `list` of integers but also tries to sneak in a change to the `list`:

```
void print( const list<int> & lst, ostream & out = cout )
{
    typename Container::iterator itr = lst.begin( );
    while( itr != lst.end( ) )
    {
        out << *itr << endl;
        *itr = 0;   // This is fishy!!!
        ++itr;
    }
}
```

If this code were legal, then the const-ness of the `list` would be completely meaningless, because it would be so easily bypassed. The code is not legal and will not compile. The solution provided by the STL is that every collection contains not only an `iterator` nested type but also a `const_iterator` nested type. The main difference between an `iterator` and a `const_iterator` is that `operator*` for `const_iterator` returns a constant reference, and thus `*itr` for a `const_iterator` cannot appear on the left-hand side of an assignment statement.

Further, the compiler will force you to use a `const_iterator` to traverse a constant collection. It does so by providing two versions of `begin` and two versions of `end`, as follows:

- `iterator begin( )`
- `const_iterator begin( ) const`
- `iterator end( )`
- `const_iterator end( ) const`

The two versions of `begin` can be in the same class only because the const-ness of a method (i.e., whether it is an accessor or mutator) is considered to be part of the signature. We saw this trick in Section 1.7.2 and we will see it again in Section 3.4, both in the context of overloading `operator[]`.

If `begin` is invoked on a nonconstant container, the "mutator" version that returns an `iterator` is invoked. However, if `begin` is invoked on a constant container, what is returned is a `const_iterator`, and the return value may not be assigned to an `iterator`. If you try to do so, a compiler error is generated. Once `itr` is a `const_iterator`, `*itr=0` is easily detected as being illegal.

If you use `auto` to declare your iterators, the compiler will deduce for you whether an `iterator` or `const_iterator` is substituted; to a large extent, this relieves the programmer from having to keep track of the correct iterator type and is precisely one of the intended uses of `auto`. Additionally, library classes such as `vector` and `list` that provide iterators as described above are compatible with the range-based `for` loop, as are user-defined classes.

An additional feature in C++11 allows one to write code that works even if the `Container` type does not have `begin` and `end` member functions. Non-member free functions `begin` and `end` are defined that allow one to use `begin(c)` in any place where `c.begin()` is allowed. Writing generic code using `begin(c)` instead of `c.begin()` has the advantage that it allows the generic code to work on containers that have `begin/end` as members, as well as those that do not have `begin/end` but which can later be augmented with appropriate

```
1  template <typename Container>
2  void print( const Container & c, ostream & out = cout )
3  {
4      if( c.empty( ) )
5          out << "(empty)";
6      else
7      {
8          auto itr = begin( c );   // itr is a Container::const_iterator
9
10         out << "[ " << *itr++;   // Print first item
11
12         while( itr != end( c ) )
13             out << ", " << *itr++;
14         out << " ]" << endl;
15     }
16 }
```

Figure 3.6 Printing any container

non-member functions. The addition of `begin` and `end` as free functions in C++11 is made possible by the addition of language features `auto` and `decltype`, as shown in the code below.

```
template<typename Container>
auto begin( Container & c ) -> decltype( c.begin( ) )
{
    return c.begin( );
}

template<typename Container>
auto begin( const Container & c ) -> decltype( c.begin( ) )
{
    return c.begin( );
}
```

In this code, the return type of `begin` is deduced to be the type of `c.begin()` .

The code in Figure 3.6 makes use of `auto` to declare the iterator (as in Fig. 3.5) and uses non-member functions `begin` and `end`.

3.4 Implementation of `vector`

In this section, we provide the implementation of a usable `vector` class template. The `vector` will be a first-class type, meaning that unlike the primitive array in C++, the `vector` can be copied, and the memory it uses can be automatically reclaimed (via its destructor). In Section 1.5.7, we described some important features of C++ primitive arrays:

- The array is simply a pointer variable to a block of memory; the actual array size must be maintained separately by the programmer.
- The block of memory can be allocated via `new[]` but then must be freed via `delete[]`.
- The block of memory cannot be resized (but a new, presumably larger block can be obtained and initialized with the old block, and then the old block can be freed).

To avoid ambiguities with the library class, we will name our class template `Vector`. Before examining the `Vector` code, we outline the main details:

1. The `Vector` will maintain the primitive array (via a pointer variable to the block of allocated memory), the array capacity, and the current number of items stored in the `Vector`.
2. The `Vector` will implement the Big-Five to provide deep-copy semantics for the copy constructor and `operator=`, and will provide a destructor to reclaim the primitive array. It will also implement C++11 move semantics.
3. The `Vector` will provide a `resize` routine that will change (generally to a larger number) the size of the `Vector` and a `reserve` routine that will change (generally to a larger number) the capacity of the `Vector`. The capacity is changed by obtaining a new block of memory for the primitive array, copying the old block into the new block, and reclaiming the old block.
4. The `Vector` will provide an implementation of `operator[]` (as mentioned in Section 1.7.2, `operator[]` is typically implemented with both an accessor and mutator version).
5. The `Vector` will provide basic routines, such as `size`, `empty`, `clear` (which are typically one-liners), `back`, `pop_back`, and `push_back`. The `push_back` routine will call `reserve` if the size and capacity are same.
6. The `Vector` will provide support for the nested types `iterator` and `const_iterator`, and associated `begin` and `end` methods.

Figure 3.7 and Figure 3.8 show the `Vector` class. Like its STL counterpart, there is limited error checking. Later we will briefly discuss how error checking can be provided.

As shown on lines 118 to 120, the `Vector` stores the size, capacity, and primitive array as its data members. The constructor at lines 7 to 9 allows the user to specify an initial size, which defaults to zero. It then initializes the data members, with the capacity slightly larger than the size, so a few `push_back`s can be performed without changing the capacity.

The copy constructor, shown at lines 11 to 17, makes a new `Vector` and can then be used by a casual implementation of `operator=` that uses the standard idiom of swapping in a copy. This idiom works only if swapping is done by moving, which itself requires the implementation of the move constructor and move `operator=` shown at lines 29 to 44. Again, these use very standard idioms. Implementation of the copy assignment `operator=` using a copy constructor and swap, while simple, is certainly not the most efficient method, especially in the case where both `Vector`s have the same size. In that special case, which can be tested for, it can be more efficient to simply copy each element one by one using `Object`'s `operator=`.

```
1  #include <algorithm>
2
3  template <typename Object>
4  class Vector
5  {
6    public:
7      explicit Vector( int initSize = 0 ) : theSize{ initSize },
8           theCapacity{ initSize + SPARE_CAPACITY }
9        { objects = new Object[ theCapacity ]; }
10
11     Vector( const Vector & rhs ) : theSize{ rhs.theSize },
12         theCapacity{ rhs.theCapacity }, objects{ nullptr }
13     {
14         objects = new Object[ theCapacity  ];
15         for( int k = 0; k < theSize; ++k )
16             objects[ k ] = rhs.objects[ k ];
17     }
18
19     Vector & operator= ( const Vector & rhs )
20     {
21         Vector copy = rhs;
22         std::swap( *this, copy );
23         return *this;
24     }
25
26     ~Vector( )
27       { delete [ ] objects; }
28
29     Vector( Vector && rhs ) : theSize{ rhs.theSize },
30         theCapacity{ rhs.theCapacity }, objects{ rhs.objects }
31     {
32         rhs.objects = nullptr;
33         rhs.theSize = 0;
34         rhs.theCapacity = 0;
35     }
36
37     Vector & operator= ( Vector && rhs )
38     {
39         std::swap( theSize, rhs.theSize );
40         std::swap( theCapacity, rhs.theCapacity );
41         std::swap( objects, rhs.objects );
42
43         return *this;
44     }
45
```

Figure 3.7 `vector` class (Part 1 of 2)

```
46      void resize( int newSize )
47      {
48          if( newSize > theCapacity )
49              reserve( newSize * 2 );
50          theSize = newSize;
51      }
52
53      void reserve( int newCapacity )
54      {
55          if( newCapacity < theSize )
56              return;
57
58          Object *newArray = new Object[ newCapacity ];
59          for( int k = 0; k < theSize; ++k )
60              newArray[ k ] = std::move( objects[ k ] );
61
62          theCapacity = newCapacity;
63          std::swap( objects, newArray );
64          delete [ ] newArray;
65      }
```

Figure 3.7 *(continued)*

The `resize` routine is shown at lines 46 to 51. The code simply sets the `theSize` data member, after possibly expanding the capacity. Expanding capacity is very expensive. So if the capacity is expanded, it is made twice as large as the size to avoid having to change the capacity again unless the size increases dramatically (the `+1` is used in case the size is 0). Expanding capacity is done by the `reserve` routine, shown at lines 53 to 65. It consists of allocation of a new array at line 58, moving the old contents at lines 59 and 60, and the reclaiming of the old array at line 64. As shown at lines 55 and 56, the `reserve` routine can also be used to shrink the underlying array, but only if the specified new capacity is at least as large as the size. If it isn't, the `reserve` request is ignored.

The two versions of `operator[]` are trivial (and in fact very similar to the implementations of `operator[]` in the `matrix` class in Section 1.7.2) and are shown in lines 67 to 70. Error checking is easily added by making sure that `index` is in the range `0` to `size()-1`, inclusive, and throwing an exception if it is not.

A host of short routines, namely, `empty`, `size`, `capacity`, `push_back`, `pop_back`, and `back`, are implemented in lines 72 to 101. At lines 83 and 90, we see the use of the postfix `++` operator, which uses `theSize` to index the array and then increases `theSize`. We saw the same idiom when discussing iterators: `*itr++` uses `itr` to decide which item to view and then advances `itr`. The positioning of the `++` matters: In the prefix `++` operator, `*++itr` advances `itr` and then uses the new `itr` to decide which item to view, and likewise, `objects[++theSize]` would increment `theSize` and use the new value to index the array (which is not what we would want). `pop_back` and `back` could both benefit from error checks in which an exception is thrown if the size is 0.

```
67      Object & operator[]( int index )
68        { return objects[ index ]; }
69      const Object & operator[]( int index ) const
70        { return objects[ index ]; }
71
72      bool empty( ) const
73        { return size( ) == 0; }
74      int size( ) const
75        { return theSize; }
76      int capacity( ) const
77        { return theCapacity; }
78
79      void push_back( const Object & x )
80      {
81          if( theSize == theCapacity )
82              reserve( 2 * theCapacity + 1 );
83          objects[ theSize++ ] = x;
84      }
85
86      void push_back( Object && x )
87      {
88          if( theSize == theCapacity )
89              reserve( 2 * theCapacity + 1 );
90          objects[ theSize++ ] = std::move( x );
91      }
92
93      void pop_back( )
94      {
95          --theSize;
96      }
97
98      const Object & back ( ) const
99      {
100          return objects[ theSize - 1 ];
101     }
102
103     typedef Object * iterator;
104     typedef const Object * const_iterator;
105
106     iterator begin( )
107       { return &objects[ 0 ]; }
108     const_iterator begin( ) const
109       { return &objects[ 0 ]; }
```

Figure 3.8 `vector` class (Part 2 of 2)

```
110       iterator end( )
111         { return &objects[ size( ) ]; }
112       const_iterator end( ) const
113         { return &objects[ size( ) ]; }
114
115       static const int SPARE_CAPACITY = 16;
116
117     private:
118       int theSize;
119       int theCapacity;
120       Object * objects;
121   };
```

Figure 3.8 *(continued)*

Finally, at lines 103 to 113 we see the declaration of the `iterator` and `const_iterator` nested types and the two `begin` and two `end` methods. This code makes use of the fact that in C++, a pointer variable has all the same operators that we expect for an `iterator`. Pointer variables can be copied and compared; the `*` operator yields the object being pointed at, and, most peculiarly, when `++` is applied to a pointer variable, the pointer variable then points at the object that would be stored next sequentially: If the pointer is pointing inside an array, incrementing the pointer positions it at the next array element. These semantics for pointers date back to the early 70s with the C programming language, upon which C++ is based. The STL iterator mechanism was designed in part to mimic pointer operations.

Consequently, at lines 103 and 104, we see `typedef` statements that state the `iterator` and `const_iterator` are simply other names for a pointer variable, and `begin` and `end` need to simply return the memory addresses representing the first array position and the first invalid array position, respectively.

The correspondence between iterators and pointers for the `vector` type means that using a `vector` instead of the C++ array is likely to carry little overhead. The disadvantage is that, as written, the code has no error checks. If the iterator `itr` goes crashing past the end marker, neither `++itr` nor `*itr` will necessarily signal an error. To fix this problem would require that the `iterator` and `const_iterator` be actual nested class types rather than simply pointer variables. Using nested class types is much more common and is what we will see in the `List` class in Section 3.5.

3.5 Implementation of `list`

In this section, we provide the implementation of a usable `list` class template. As in the case of the `vector` class, our list class will be named `List` to avoid ambiguities with the library class.

Recall that the `List` class will be implemented as a doubly linked list and that we will need to maintain pointers to both ends of the list. Doing so allows us to maintain constant time cost per operation, so long as the operation occurs at a known position. The known position can be at either end or at a position specified by an iterator.

In considering the design, we will need to provide four classes:

1. The `List` class itself, which contains links to both ends, the size of the list, and a host of methods.
2. The `Node` class, which is likely to be a private nested class. A node contains the data and pointers to the previous and next nodes, along with appropriate constructors.
3. The `const_iterator` class, which abstracts the notion of a position, and is a public nested class. The `const_iterator` stores a pointer to "current" node, and provides implementation of the basic iterator operations, all in the form of overloaded operators such as `=`, `==`, `!=`, and `++`.
4. The `iterator` class, which abstracts the notion of a position, and is a public nested class. The `iterator` has the same functionality as `const_iterator`, except that `operator*` returns a reference to the item being viewed, rather than a constant reference to the item. An important technical issue is that an `iterator` can be used in any routine that requires a `const_iterator`, but not vice versa. In other words, `iterator` *IS-A* `const_iterator`.

Because the iterator classes store a pointer to the "current node," and the end marker is a valid position, it makes sense to create an extra node at the end of the list to represent the endmarker. Further, we can create an extra node at the front of the list, logically representing the beginning marker. These extra nodes are sometimes known as **sentinel nodes**; specifically, the node at the front is sometimes known as a **header node**, and the node at the end is sometimes known as a **tail node**.

The advantage of using these extra nodes is that they greatly simplify the coding by removing a host of special cases. For instance, if we do not use a header node, then removing the first node becomes a special case, because we must reset the list's link to the first node during the remove and because the remove algorithm in general needs to access the node prior to the node being removed (and without a header node, the first node does not have a node prior to it). Figure 3.9 shows a doubly linked list with header and tail nodes. Figure 3.10 shows an empty list.

Figure 3.11 and Figure 3.12 show the outline and partial implementation of the `List` class.

We can see at line 5 the beginning of the declaration of the private nested `Node` class. Rather than using the `class` keyword, we use `struct`. In C++, the `struct` is a relic from the C programming language. A `struct` in C++ is essentially a `class` in which the members default to public. Recall that in a `class`, the members default to private. Clearly the `struct`

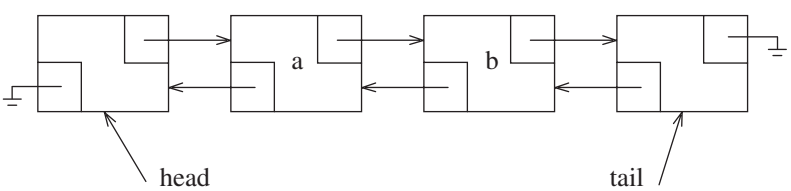


Figure 3.9 A doubly linked list with header and tail nodes

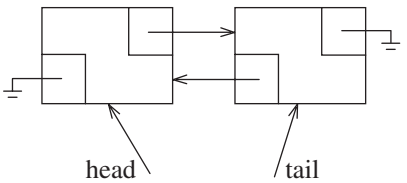


Figure 3.10 An empty doubly linked list with header and tail nodes

keyword is not needed, but you will often see it and it is commonly used by programmers to signify a type that contains mostly data that are accessed directly, rather than through methods. In our case, making the members public in the `Node` class will not be a problem, since the `Node` class is itself private and inaccessible outside of the `List` class.

At line 9 we see the beginning of the declaration of the public nested `const_iterator` class, and at line 12 we see the beginning of the declaration of the public nested `iterator` class. The unusual syntax is **inheritance**, which is a powerful construct not otherwise used in the book. The inheritance syntax states that `iterator` has exactly the same functionality as `const_iterator`, with possibly some additions, and that `iterator` is type-compatible with `const_iterator` and can be used wherever `const_iterator` is needed. We'll discuss those details when we see the actual implementations later.

Lines 80 to 82 contain the data members for `List`, namely, the pointers to the header and tail nodes. We also keep track of the size in a data member so that the `size` method can be implemented in constant time.

The rest of the `List` class consists of the constructor, the Big-Five, and a host of methods. Many of the methods are one-liners. `begin` and `end` return appropriate iterators; the call at line 30 is typical of the implementation, in which we return a constructed iterator (thus the `iterator` and `const_iterator` classes each have a constructor that takes a pointer to a `Node` as its parameter).

The `clear` method at lines 43 to 47 works by repeatedly removing items until the `List` is empty. Using this strategy allows `clear` to avoid getting its hands dirty reclaiming nodes because the node reclamation is now funneled to `pop_front`. The methods at lines 48 to 67 all work by cleverly obtaining and using an appropriate iterator. Recall that the `insert` method inserts prior to a position, so `push_back` inserts prior to the endmarker, as required. In `pop_back`, note that `erase(--end())` creates a temporary iterator corresponding to the endmarker, retreats the temporary iterator, and uses that iterator to `erase`. Similar behavior occurs in `back`. Note also that in the case of the `pop_front` and `pop_back` operations, we again avoid dealing with node reclamation.

Figure 3.13 shows the `Node` class, consisting of the stored item, pointers to the previous and next `Node`, and a constructor. All the data members are public.

Figure 3.14 shows the `const_iterator` class, and Figure 3.15 shows the `iterator` class. As we mentioned earlier, the syntax at line 39 in Figure 3.15 indicates an advanced feature known as *inheritance* and means that `iterator` *IS-A* `const_iterator`. When the `iterator` class is written this way, it inherits all the data and methods from `const_iterator`. It may then add new data, add new methods, and override (i.e., redefine) existing methods. In the most general scenario, there is significant syntactical baggage (often resulting in the keyword `virtual` appearing in the code).

```
1  template <typename Object>
2  class List
3  {
4    private:
5      struct Node
6        { /* See Figure 3.13 */ };
7
8    public:
9      class const_iterator
10       { /* See Figure 3.14 */ };
11
12     class iterator : public const_iterator
13       { /* See Figure 3.15 */ };
14
15   public:
16     List( )
17       { /* See Figure 3.16 */ }
18     List( const List & rhs )
19       { /* See Figure 3.16 */ }
20     ~List( )
21       { /* See Figure 3.16 */ }
22     List & operator= ( const List & rhs )
23       { /* See Figure 3.16 */ }
24     List( List && rhs )
25       { /* See Figure 3.16 */ }
26     List & operator= ( List && rhs )
27       { /* See Figure 3.16 */ }
28
29     iterator begin( )
30       { return { head->next }; }
31     const_iterator begin( ) const
32       { return { head->next }; }
33     iterator end( )
34       { return { tail }; }
35     const_iterator end( ) const
36       { return { tail }; }
37
38     int size( ) const
39       { return theSize; }
40     bool empty( ) const
41       { return size( ) == 0; }
42
43     void clear( )
44     {
45         while( !empty( ) )
46             pop_front( );
47     }
```

Figure 3.11 `List` class (Part 1 of 2)

```
48      Object & front( )
49        { return *begin( ); }
50      const Object & front( ) const
51        { return *begin( ); }
52      Object & back( )
53        { return *--end( ); }
54      const Object & back( ) const
55        { return *--end( ); }
56      void push_front( const Object & x )
57        { insert( begin( ), x ); }
58      void push_front( Object && x )
59        { insert( begin( ), std::move( x ) ); }
60      void push_back( const Object & x )
61        { insert( end( ), x ); }
62      void push_back( Object && x )
63        { insert( end( ), std::move( x ) ); }
64      void pop_front( )
65        { erase( begin( ) ); }
66      void pop_back( )
67        { erase( --end( ) ); }
68
69      iterator insert( iterator itr, const Object & x )
70        { /* See Figure 3.18 */ }
71      iterator insert( iterator itr, Object && x )
72        { /* See Figure 3.18 */ }
73
74      iterator erase( iterator itr )
75        { /* See Figure 3.20 */ }
76      iterator erase( iterator from, iterator to )
77        { /* See Figure 3.20 */ }
78
79    private:
80      int   theSize;
81      Node *head;
82      Node *tail;
83
84      void init( )
85        { /* See Figure 3.16 */ }
86  };
```

Figure 3.12 `List` class (Part 2 of 2)

```
1       struct Node
2       {
3           Object  data;
4           Node   *prev;
5           Node   *next;
6
7           Node( const Object & d = Object{ }, Node * p = nullptr,
8                                               Node * n = nullptr )
9             : data{ d }, prev{ p }, next{ n } { }
10
11          Node( Object && d, Node * p = nullptr, Node * n = nullptr )
12            : data{ std::move( d ) }, prev{ p }, next{ n } { }
13      };
```

Figure 3.13 Nested `Node` class for `List` class

However, in our case, we can avoid much of the syntactical baggage because we are not adding new data, nor are we intending to change the behavior of an existing method. We are, however, adding some new methods in the `iterator` class (with very similar signatures to the existing methods in the `const_iterator` class). As a result, we can avoid using `virtual`. Even so, there are quite a few syntax tricks in `const_iterator`.

At lines 28 and 29, `const_iterator` stores as its single data member a pointer to the "current" node. Normally, this would be private, but if it were private, then `iterator` would not have access to it. Marking members of `const_iterator` as `protected` allows the classes that inherit from `const_iterator` to have access to these members, but does not allow other classes to have access.

At lines 34 and 35 we see the constructor for `const_iterator` that was used in the `List` class implementation of `begin` and `end`. We don't want all classes to see this constructor (iterators are not supposed to be visibly constructed from pointer variables), so it can't be public, but we also want the `iterator` class to be able to see it, so logically this constructor is made protected. However, this doesn't give `List` access to the constructor. The solution is the **friend declaration** at line 37, which grants the `List` class access to `const_iterator`'s nonpublic members.

The public methods in `const_iterator` all use operator overloading. `operator==`, `operator!=`, and `operator*` are straightforward. At lines 10 to 21 we see the implementation of `operator++`. Recall that the prefix and postfix versions of `operator++` are completely different in semantics (and precedence), so we need to write separate routines for each form. They have the same name, so they must have different signatures to be distinguished. C++ requires that we give them different signatures by specifying an empty parameter list for the prefix form and a single (anonymous) `int` parameter for the postfix form. Then `++itr` calls the zero-parameter `operator++`; and `itr++` calls the one-parameter `operator++`. The `int` parameter is never used; it is present only to give a different signature. The implementation suggests that, in many cases where there is a choice between using the prefix or postfix `operator++`, the prefix form will be faster than the postfix form.

In the `iterator` class, the protected constructor at line 64 uses an initialization list to initialize the inherited current node. We do not have to reimplement `operator==`

```
1     class const_iterator
2     {
3       public:
4         const_iterator( ) : current{ nullptr }
5           { }
6
7         const Object & operator* ( ) const
8           { return retrieve( ); }
9
10        const_iterator & operator++ ( )
11        {
12            current = current->next;
13            return *this;
14        }
15
16        const_iterator operator++ ( int )
17        {
18            const_iterator old = *this;
19            ++( *this );
20            return old;
21        }
22
23        bool operator== ( const const_iterator & rhs ) const
24          { return current == rhs.current; }
25        bool operator!= ( const const_iterator & rhs ) const
26          { return !( *this == rhs ); }
27
28      protected:
29        Node *current;
30
31        Object & retrieve( ) const
32          { return current->data; }
33
34        const_iterator( Node *p ) : current{ p }
35          { }
36
37        friend class List<Object>;
38    };
```

Figure 3.14 Nested `const_iterator` class for `List` class

and `operator!=` because those are inherited unchanged. We do provide a new pair of `operator++` implementations (because of the changed return type) that hide the originals in the `const_iterator`, and we provide an accessor/mutator pair for `operator*`. The accessor `operator*`, shown at lines 47 and 48, simply uses the same implementation as in `const_iterator`. The accessor is explicitly implemented in `iterator` because otherwise the original implementation is hidden by the newly added mutator version.

```
39      class iterator : public const_iterator
40      {
41        public:
42          iterator( )
43            { }
44
45          Object & operator* ( )
46            { return const_iterator::retrieve( ); }
47          const Object & operator* ( ) const
48            { return const_iterator::operator*( ); }
49
50          iterator & operator++ ( )
51          {
52              this->current = this->current->next;
53              return *this;
54          }
55
56          iterator operator++ ( int )
57          {
58              iterator old = *this;
59              ++( *this );
60              return old;
61          }
62
63        protected:
64          iterator( Node *p ) : const_iterator{ p }
65            { }
66
67          friend class List<Object>;
68      };
```

Figure 3.15 Nested `iterator` class for `List` class

Figure 3.16 shows the constructor and Big-Five. Because the zero-parameter constructor and copy constructor must both allocate the header and tail nodes, we provide a private `init` routine. `init` creates an empty `List`. The destructor reclaims the header and tail nodes; all the other nodes are reclaimed when the destructor invokes `clear`. Similarly, the copy constructor is implemented by invoking public methods rather than attempting low-level pointer manipulations.

Figure 3.17 illustrates how a new node containing `x` is spliced in between a node pointed at by `p` and `p.prev`. The assignment to the node pointers can be described as follows:

```
Node *newNode = new Node{ x, p->prev, p };  // Steps 1 and 2
p->prev->next = newNode;                    // Step 3
p->prev = newNode;                          // Step 4
```

```
1       List( )
2         { init( ); }
3
4       ~List( )
5       {
6           clear( );
7           delete head;
8           delete tail;
9       }
10
11      List( const List & rhs )
12      {
13          init( );
14          for( auto & x : rhs )
15              push_back( x );
16      }
17
18      List & operator= ( const List & rhs )
19      {
20          List copy = rhs;
21          std::swap( *this, copy );
22          return *this;
23      }
24
25
26      List( List && rhs )
27        : theSize{ rhs.theSize }, head{ rhs.head }, tail{ rhs.tail }
28      {
29          rhs.theSize = 0;
30          rhs.head = nullptr;
31          rhs.tail = nullptr;
32      }
33
34      List & operator= ( List && rhs )
35      {
36          std::swap( theSize, rhs.theSize );
37          std::swap( head, rhs.head );
38          std::swap( tail, rhs.tail );
39
40          return *this;
41      }
42
43      void init( )
44      {
45          theSize = 0;
46          head = new Node;
47          tail = new Node;
48          head->next = tail;
49          tail->prev = head;
50      }
```

Figure 3.16 Constructor, Big-Five, and private `init` routine for `List` class

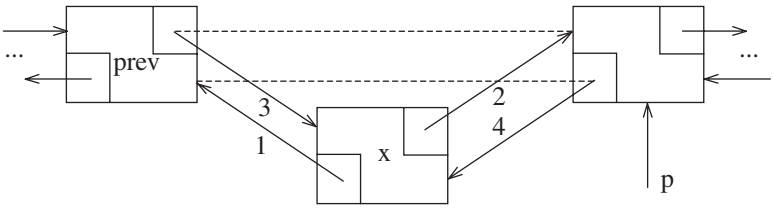


Figure 3.17 Insertion in a doubly linked list by getting a new node and then changing pointers in the order indicated

Steps 3 and 4 can be combined, yielding only two lines:

```
Node *newNode = new Node{ x, p->prev, p };  // Steps 1 and 2
p->prev = p->prev->next = newNode;          // Steps 3 and 4
```

But then these two lines can also be combined, yielding:

```
p->prev = p->prev->next = new Node{ x, p->prev, p };
```

This makes the `insert` routine in Figure 3.18 short.

Figure 3.19 shows the logic of removing a node. If `p` points to the node being removed, only two pointers change before the node can be reclaimed:

```
p->prev->next = p->next;
p->next->prev = p->prev;
delete p;
```

Figure 3.20 shows a pair of `erase` routines. The first version of `erase` contains the three lines of code shown above and the code to return an `iterator` representing the item after

```
1       // Insert x before itr.
2       iterator insert( iterator itr, const Object & x )
3       {
4           Node *p = itr.current;
5           theSize++;
6           return { p->prev = p->prev->next = new Node{ x, p->prev, p } };
7       }
8
9       // Insert x before itr.
10      iterator insert( iterator itr, Object && x )
11      {
12          Node *p = itr.current;
13          theSize++;
14          return { p->prev = p->prev->next
15                                  = new Node{ std::move( x ), p->prev, p } };
16      }
```

Figure 3.18 `insert` routine for `List` class

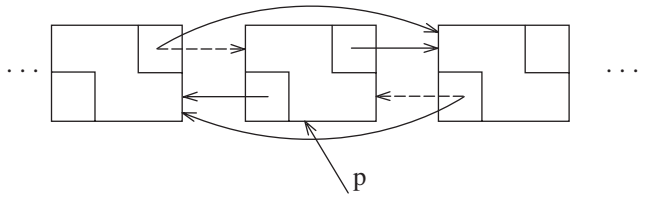


Figure 3.19 Removing node specified by p from a doubly linked list

```
1       // Erase item at itr.
2       iterator erase( iterator itr )
3       {
4           Node *p = itr.current;
5           iterator retVal{ p->next };
6           p->prev->next = p->next;
7           p->next->prev = p->prev;
8           delete p;
9           theSize--;
10
11          return retVal;
12      }
13
14      iterator erase( iterator from, iterator to )
15      {
16          for( iterator itr = from; itr != to; )
17              itr = erase( itr );
18
19          return to;
20      }
```

Figure 3.20 erase routines for List class

the erased element. Like insert, erase must update theSize. The second version of erase simply uses an iterator to call the first version of erase. Note that we cannot simply use itr++ in the for loop at line 16 and ignore the return value of erase at line 17. The value of itr is stale immediately after the call to erase, which is why erase returns an iterator.

In examining the code, we can see a host of errors that can occur and for which no checks are provided. For instance, iterators passed to erase and insert can be uninitialized or for the wrong list! Iterators can have ++ or * applied to them when they are already at the endmarker or are uninitialized.

An uninitialized iterator will have current pointing at nullptr, so that condition is easily tested. The endmarker's next pointer points at nullptr, so testing for ++ or * on an endmarker condition is also easy. However, in order to determine if an iterator passed to erase or insert is an iterator for the correct list, the iterator must store an additional data member representing a pointer to the List from which it was constructed.

```
1     protected:
2       const List<Object> *theList;
3       Node *current;
4
5       const_iterator( const List<Object> & lst, Node *p )
6         : theList{ &lst }, current{ p }
7       {
8       }
9
10      void assertIsValid( ) const
11      {
12          if( theList == nullptr || current == nullptr || current == theList->head )
13              throw IteratorOutOfBoundsException{ };
14      }
```

Figure 3.21 Revised protected section of `const_iterator` that incorporates ability to perform additional error checks

We will sketch the basic idea and leave the details as an exercise. In the `const_iterator` class, we add a pointer to the `List` and modify the protected constructor to take the `List` as a parameter. We can also add methods that throw an exception if certain assertions aren't met. The revised protected section looks something like the code in Figure 3.21. Then all calls to `iterator` and `const_iterator` constructors that formerly took one parameter now take two, as in the `begin` method for `List`:

```
const_iterator begin( ) const
{
    const_iterator itr{ *this, head };
    return ++itr;
}
```

Then `insert` can be revised to look something like the code in Figure 3.22. We leave the details of these modifications as an exercise.

```
1       // Insert x before itr.
2       iterator insert( iterator itr, const Object & x )
3       {
4           itr.assertIsValid( );
5           if( itr.theList != this )
6               throw IteratorMismatchException{ };
7
8           Node *p = itr.current;
9           theSize++;
10          return { *this, p->prev = p->prev->next = new Node{ x, p->prev, p } };
11      }
```

Figure 3.22 `List insert` with additional error checks

3.6 The Stack ADT

A **stack** is a list with the restriction that insertions and deletions can be performed in only one position, namely, the end of the list, called the **top.**

3.6.1 Stack Model

The fundamental operations on a stack are `push`, which is equivalent to an insert, and `pop`, which deletes the most recently inserted element. The most recently inserted element can be examined prior to performing a `pop` by use of the `top` routine. A `pop` or `top` on an empty stack is generally considered an error in the stack ADT. On the other hand, running out of space when performing a `push` is an implementation limit but not an ADT error.

Stacks are sometimes known as LIFO (last in, first out) lists. The model depicted in Figure 3.23 signifies only that `push`es are input operations and `pop`s and `top`s are output. The usual operations to make empty stacks and test for emptiness are part of the repertoire, but essentially all that you can do to a stack is `push` and `pop`.

Figure 3.24 shows an abstract stack after several operations. The general model is that there is some element that is at the top of the stack, and it is the only element that is visible.

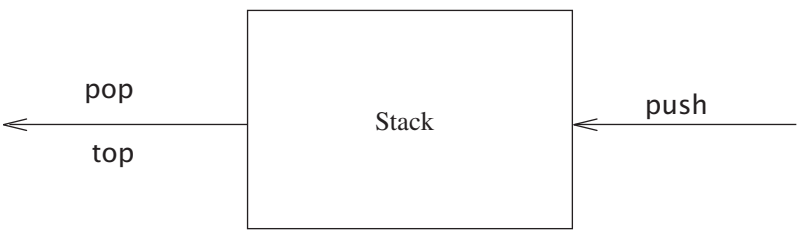


Figure 3.23 Stack model: Input to a stack is by `push`; output is by `pop` and `top`

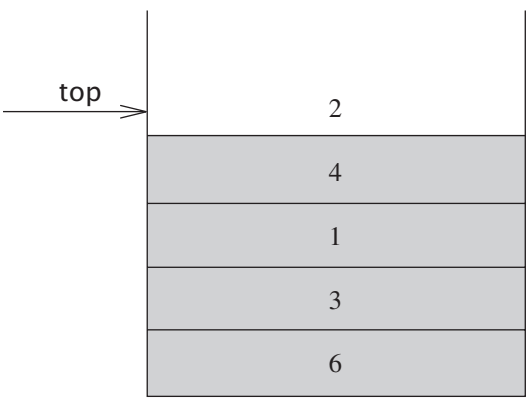


Figure 3.24 Stack model: Only the top element is accessible

3.6.2 Implementation of Stacks

Since a stack is a list, any list implementation will do. Clearly `list` and `vector` support stack operations; 99% of the time they are the most reasonable choice. Occasionally it can be faster to design a special-purpose implementation. Because stack operations are constant-time operations, this is unlikely to yield any discernable improvement except under very unique circumstances.

For these special times, we will give two popular stack implementations. One uses a linked structure, and the other uses an array, and both simplify the logic in `vector` and `list`, so we do not provide code.

Linked List Implementation of Stacks

The first implementation of a stack uses a singly linked list. We perform a `push` by inserting at the front of the list. We perform a `pop` by deleting the element at the front of the list. A `top` operation merely examines the element at the front of the list, returning its value. Sometimes the `pop` and `top` operations are combined into one.

Array Implementation of Stacks

An alternative implementation avoids links and is probably the more popular solution. It uses the `back`, `push_back`, and `pop_back` implementation from `vector`, so the implementation is trivial. Associated with each stack is `theArray` and `topOfStack`, which is -1 for an empty stack (this is how an empty stack is initialized). To push some element `x` onto the stack, we increment `topOfStack` and then set `theArray[topOfStack] = x`. To pop, we set the return value to `theArray[topOfStack]` and then decrement `topOfStack`.

Notice that these operations are performed in not only constant time but very fast constant time. On some machines, `push`es and `pop`s (of integers) can be written in one machine instruction, operating on a register with auto-increment and auto-decrement addressing. The fact that most modern machines have stack operations as part of the instruction set enforces the idea that the stack is probably the most fundamental data structure in computer science, after the array.

3.6.3 Applications

It should come as no surprise that if we restrict the operations allowed on a list, those operations can be performed very quickly. The big surprise, however, is that the small number of operations left are so powerful and important. We give three of the many applications of stacks. The third application gives a deep insight into how programs are organized.

Balancing Symbols

Compilers check your programs for syntax errors, but frequently a lack of one symbol (such as a missing brace or comment starter) can cause the compiler to spill out a hundred lines of diagnostics without identifying the real error.

A useful tool in this situation is a program that checks whether everything is balanced. Thus, every right brace, bracket, and parenthesis must correspond to its left counterpart.

The sequence `[()]` is legal, but `[(])` is wrong. Obviously, it is not worthwhile writing a huge program for this, but it turns out that it is easy to check these things. For simplicity, we will just check for balancing of parentheses, brackets, and braces and ignore any other character that appears.

The simple algorithm uses a stack and is as follows:

> Make an empty stack. Read characters until end of file. If the character is an opening symbol, push it onto the stack. If it is a closing symbol and the stack is empty, report an error. Otherwise, pop the stack. If the symbol popped is not the corresponding opening symbol, then report an error. At end of file, if the stack is not empty, report an error.

You should be able to convince yourself that this algorithm works. It is clearly linear and actually makes only one pass through the input. It is thus online and quite fast. Extra work can be done to attempt to decide what to do when an error is reported—such as identifying the likely cause.

Postfix Expressions

Suppose we have a pocket calculator and would like to compute the cost of a shopping trip. To do so, we add a list of numbers and multiply the result by 1.06; this computes the purchase price of some items with local sales tax added. If the items are 4.99, 5.99, and 6.99, then a natural way to enter this would be the sequence

$$4.99 + 5.99 + 6.99 * 1.06 =$$

Depending on the calculator, this produces either the intended answer, 19.05, or the scientific answer, 18.39. Most simple four-function calculators will give the first answer, but many advanced calculators know that multiplication has higher precedence than addition.

On the other hand, some items are taxable and some are not, so if only the first and last items were actually taxable, then the sequence

$$4.99 * 1.06 + 5.99 + 6.99 * 1.06 =$$

would give the correct answer (18.69) on a scientific calculator and the wrong answer (19.37) on a simple calculator. A scientific calculator generally comes with parentheses, so we can always get the right answer by parenthesizing, but with a simple calculator we need to remember intermediate results.

A typical evaluation sequence for this example might be to multiply 4.99 and 1.06, saving this answer as A_1. We then add 5.99 and A_1, saving the result in A_1. We multiply 6.99 and 1.06, saving the answer in A_2, and finish by adding A_1 and A_2, leaving the final answer in A_1. We can write this sequence of operations as follows:

$$4.99\ 1.06 * 5.99 + 6.99\ 1.06\ * +$$

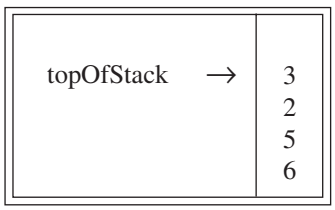
This notation is known as **postfix**, or **reverse Polish notation**, and is evaluated exactly as we have described above. The easiest way to do this is to use a stack. When a number is seen, it is pushed onto the stack; when an operator is seen, the operator is applied to the

two numbers (symbols) that are popped from the stack, and the result is pushed onto the stack. For instance, the postfix expression

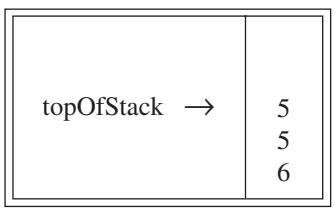
$$6\ 5\ 2\ 3 + 8 * + 3 + *$$

is evaluated as follows:

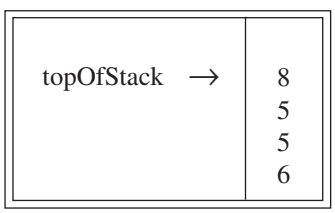
The first four symbols are placed on the stack. The resulting stack is



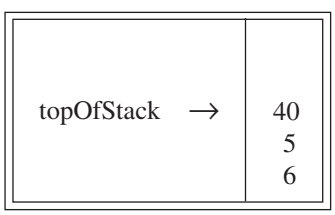
Next, a '+' is read, so 3 and 2 are popped from the stack, and their sum, 5, is pushed.



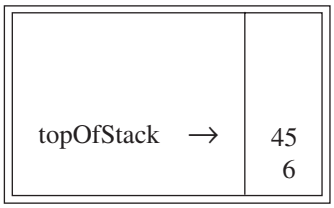
Next, 8 is pushed.



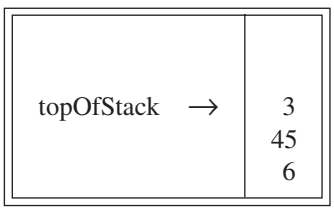
Now a '*' is seen, so 8 and 5 are popped, and $5 * 8 = 40$ is pushed.



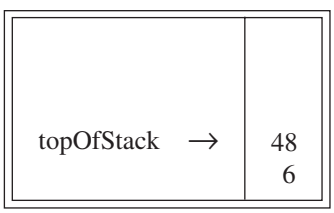
Next, a '+' is seen, so 40 and 5 are popped, and $5 + 40 = 45$ is pushed.



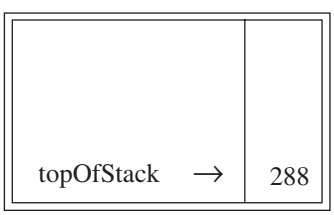
Now, 3 is pushed.



Next, '+' pops 3 and 45 and pushes $45 + 3 = 48$.



Finally, a '*' is seen and 48 and 6 are popped; the result, $6 * 48 = 288$, is pushed.



The time to evaluate a postfix expression is $O(N)$, because processing each element in the input consists of stack operations and therefore takes constant time. The algorithm to do so is very simple. Notice that when an expression is given in postfix notation, there is no need to know any precedence rules; this is an obvious advantage.

Infix to Postfix Conversion

Not only can a stack be used to evaluate a postfix expression, but we can also use a stack to convert an expression in standard form (otherwise known as **infix**) into postfix. We will concentrate on a small version of the general problem by allowing only the operators `+`, `*`, `(`, `)`, and insisting on the usual precedence rules. We will further assume that the expression is legal. Suppose we want to convert the infix expression

```
a + b * c + ( d * e + f ) * g
```

into postfix. A correct answer is `a b c * + d e * f + g * +`.

When an operand is read, it is immediately placed onto the output. Operators are not immediately output, so they must be saved somewhere. The correct thing to do is to place operators that have been seen, but not placed on the output, onto the stack. We will also stack left parentheses when they are encountered. We start with an initially empty stack.

If we see a right parenthesis, then we pop the stack, writing symbols until we encounter a (corresponding) left parenthesis, which is popped but not output.

If we see any other symbol (`+`, `*`, `(`), then we pop entries from the stack until we find an entry of lower priority. One exception is that we never remove a `(` from the stack except when processing a `)`. For the purposes of this operation, `+` has lowest priority and `(` highest. When the popping is done, we push the operator onto the stack.

Finally, if we read the end of input, we pop the stack until it is empty, writing symbols onto the output.

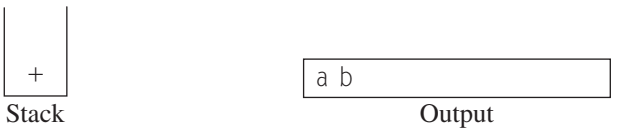
The idea of this algorithm is that when an operator is seen, it is placed on the stack. The stack represents pending operators. However, some of the operators on the stack that have high precedence are now known to be completed and should be popped, as they will no longer be pending. Thus prior to placing the operator on the stack, operators that are on the stack, and which are to be completed prior to the current operator, are popped. This is illustrated in the following table:

Expression	Stack When Third Operator Is Processed	Action
`a*b-c+d`	`-`	`-` is completed; `+` is pushed
`a/b+c*d`	`+`	Nothing is completed; `*` is pushed
`a-b*c/d`	`- *`	`*` is completed; `/` is pushed
`a-b*c+d`	`- *`	`*` and `-` are completed; `+` is pushed

Parentheses simply add an additional complication. We can view a left parenthesis as a high-precedence operator when it is an input symbol (so that pending operators remain pending) and a low-precedence operator when it is on the stack (so that it is not accidentally removed by an operator). Right parentheses are treated as the special case.

To see how this algorithm performs, we will convert the long infix expression above into its postfix form. First, the symbol `a` is read, so it is passed through to the output.

Then + is read and pushed onto the stack. Next b is read and passed through to the output. The state of affairs at this juncture is as follows:



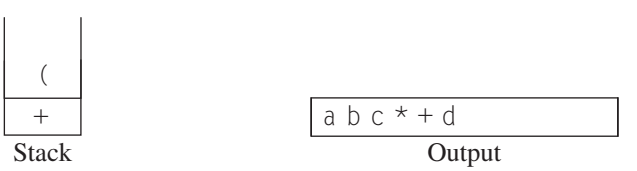
Next, a * is read. The top entry on the operator stack has lower precedence than *, so nothing is output and * is put on the stack. Next, c is read and output. Thus far, we have



The next symbol is a +. Checking the stack, we find that we will pop a * and place it on the output; pop the other +, which is not of *lower* but equal priority, on the stack; and then push the +.



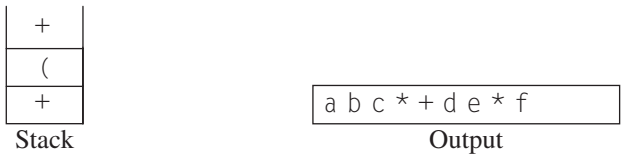
The next symbol read is a (. Being of highest precedence, this is placed on the stack. Then d is read and output.



We continue by reading a *. Since open parentheses do not get removed except when a closed parenthesis is being processed, there is no output. Next, e is read and output.



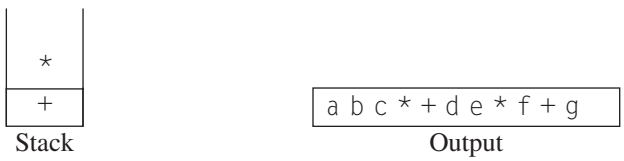
The next symbol read is a +. We pop and output * and then push +. Then we read and output f.



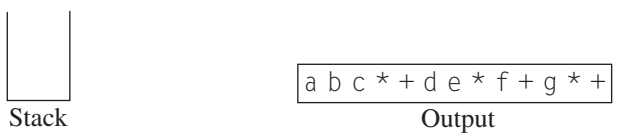
Now we read a), so the stack is emptied back to the (. We output a +.



We read a * next; it is pushed onto the stack. Then g is read and output.



The input is now empty, so we pop and output symbols from the stack until it is empty.



As before, this conversion requires only $O(N)$ time and works in one pass through the input. We can add subtraction and division to this repertoire by assigning subtraction and addition equal priority and multiplication and division equal priority. A subtle point is that the expression a - b - c will be converted to a b - c - and not a b c - -. Our algorithm does the right thing, because these operators associate from left to right. This is not necessarily the case in general, since exponentiation associates right to left: $2^{2^3} = 2^8 = 256$, not $4^3 = 64$. We leave as an exercise the problem of adding exponentiation to the repertoire of operators.

Function Calls

The algorithm to check balanced symbols suggests a way to implement function calls in compiled procedural and object-oriented languages. The problem here is that when a call is made to a new function, all the variables local to the calling routine need to be saved by the system, since otherwise the new function will overwrite the memory used by the calling routine's variables. Furthermore, the current location in the routine must be saved

so that the new function knows where to go after it is done. The variables have generally been assigned by the compiler to machine registers, and there are certain to be conflicts (usually all functions get some variables assigned to register #1), especially if recursion is involved. The reason that this problem is similar to balancing symbols is that a function call and function return are essentially the same as an open parenthesis and closed parenthesis, so the same ideas should work.

When there is a function call, all the important information that needs to be saved, such as register values (corresponding to variable names) and the return address (which can be obtained from the program counter, which is typically in a register), is saved "on a piece of paper" in an abstract way and put at the top of a pile. Then the control is transferred to the new function, which is free to replace the registers with its values. If it makes other function calls, it follows the same procedure. When the function wants to return, it looks at the "paper" at the top of the pile and restores all the registers. It then makes the return jump.

Clearly, all of this work can be done using a stack, and that is exactly what happens in virtually every programming language that implements recursion. The information saved is called either an **activation record** or **stack frame**. Typically, a slight adjustment is made: The current environment is represented at the top of the stack. Thus, a return gives the previous environment (without copying). The stack in a real computer frequently grows from the high end of your memory partition downward, and on many systems there is no checking for overflow. There is always the possibility that you will run out of stack space by having too many simultaneously active functions. Needless to say, running out of stack space is always a fatal error.

In languages and systems that do not check for stack overflow, programs crash without an explicit explanation. In normal events, you should not run out of stack space; doing so is usually an indication of runaway recursion (forgetting a base case). On the other hand, some perfectly legal and seemingly innocuous programs can cause you to run out of stack space. The routine in Figure 3.25, which prints out a container, is perfectly legal and actually correct. It properly handles the base case of an empty container, and the recursion is fine. This program can be *proven* correct. Unfortunately, if the container

```
1  /**
2   * Print container from start up to but not including end.
3   */
4  template <typename Iterator>
5  void print( Iterator start, Iterator end, ostream & out = cout )
6  {
7      if( start == end )
8          return;
9
10     out << *start++ << endl;   // Print and advance start
11     print( start, end, out );
12 }
```

Figure 3.25 A bad use of recursion: printing a container

```
1  /**
2   * Print container from start up to but not including end.
3   */
4  template <typename Iterator>
5  void print( Iterator start, Iterator end, ostream & out = cout )
6  {
7      while( true )
8      {
9          if( start == end )
10             return;
11
12         out << *start++ << endl;   // Print and advance start
13     }
14 }
```

Figure 3.26 Printing a container without recursion; a compiler might do this (you should not)

contains 200,000 elements to print, there will be a stack of 200,000 activation records representing the nested calls of line 11. Activation records are typically large because of all the information they contain, so this program is likely to run out of stack space. (If 200,000 elements are not enough to make the program crash, replace the number with a larger one.)

This program is an example of an extremely bad use of recursion known as **tail recursion.** Tail recursion refers to a recursive call at the last line. Tail recursion can be mechanically eliminated by enclosing the body in a `while` loop and replacing the recursive call with one assignment per function argument. This simulates the recursive call because nothing needs to be saved; after the recursive call finishes, there is really no need to know the saved values. Because of this, we can just go to the top of the function with the values that would have been used in a recursive call. The function in Figure 3.26 shows the mechanically improved version generated by this algorithm. Removal of tail recursion is so simple that some compilers do it automatically. Even so, it is best not to find out that yours does not.

Recursion can always be completely removed (compilers do so in converting to assembly language), but doing so can be quite tedious. The general strategy requires using a stack and is worthwhile only if you can manage to put the bare minimum on the stack. We will not dwell on this further, except to point out that although nonrecursive programs are certainly generally faster than equivalent recursive programs, the speed advantage rarely justifies the lack of clarity that results from removing the recursion.

3.7 The Queue ADT

Like stacks, **queues** are lists. With a queue, however, insertion is done at one end whereas deletion is performed at the other end.

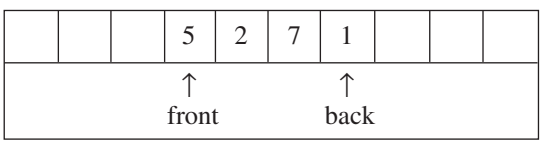
3.7.1 Queue Model

The basic operations on a queue are `enqueue`, which inserts an element at the end of the list (called the rear), and `dequeue`, which deletes (and returns) the element at the start of the list (known as the front). Figure 3.27 shows the abstract model of a queue.

3.7.2 Array Implementation of Queues

As with stacks, any list implementation is legal for queues. Like stacks, both the linked list and array implementations give fast $O(1)$ running times for every operation. The linked list implementation is straightforward and left as an exercise. We will now discuss an array implementation of queues.

For each queue data structure, we keep an array, `theArray`, and the positions `front` and `back`, which represent the ends of the queue. We also keep track of the number of elements that are actually in the queue, `currentSize`. The following table shows a queue in some intermediate state.



The operations should be clear. To `enqueue` an element `x`, we increment `currentSize` and `back`, then set `theArray[back] = x`. To `dequeue` an element, we set the return value to `theArray[front]`, decrement `currentSize`, and then increment `front`. Other strategies are possible (this is discussed later). We will comment on checking for errors presently.

There is one potential problem with this implementation. After 10 `enqueue`s, the queue appears to be full, since `back` is now at the last array index, and the next `enqueue` would be in a nonexistent position. However, there might only be a few elements in the queue, because several elements may have already been dequeued. Queues, like stacks, frequently stay small even in the presence of a lot of operations.

The simple solution is that whenever `front` or `back` gets to the end of the array, it is wrapped around to the beginning. The following tables show the queue during some operations. This is known as a **circular array** implementation.

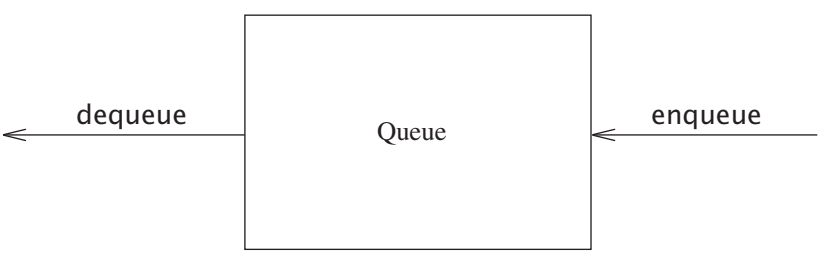
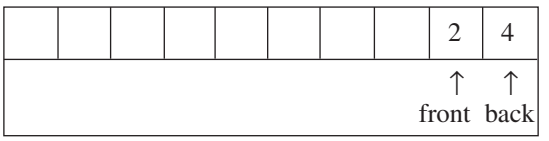
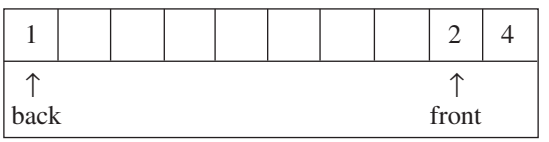


Figure 3.27 Model of a queue

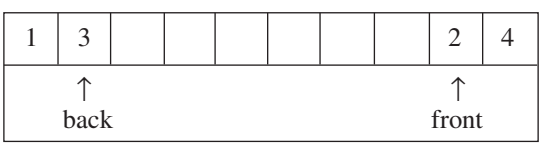
Initial state



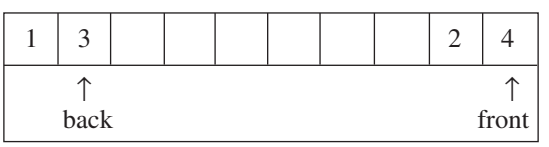
After `enqueue(1)`



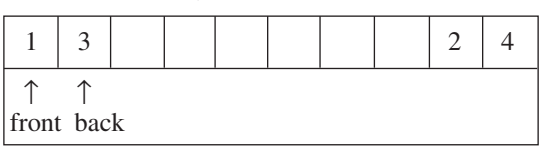
After `enqueue(3)`



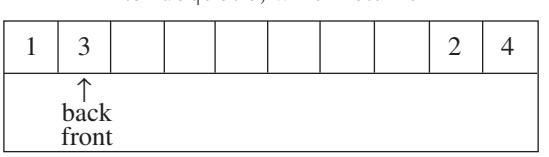
After `dequeue`, which returns 2

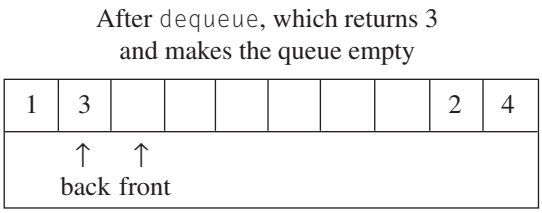


After `dequeue`, which returns 4



After `dequeue`, which returns 1





The extra code required to implement the wraparound is minimal (although it probably doubles the running time). If incrementing either `back` or `front` causes it to go past the array, the value is reset to the first position in the array.

Some programmers use different ways of representing the front and back of a queue. For instance, some do not use an entry to keep track of the size, because they rely on the base case that when the queue is empty, `back = front-1`. The size is computed implicitly by comparing `back` and `front`. This is a very tricky way to go, because there are some special cases, so be very careful if you need to modify code written this way. If the `currentSize` is not maintained as an explicit data member, then the queue is full when there are `theArray.capacity()-1` elements, since only `theArray.capacity()` different sizes can be differentiated and one of these is 0. Pick any style you like and make sure that all your routines are consistent. Since there are a few options for implementation, it is probably worth a comment or two in the code if you don't use the `currentSize` data member.

In applications where you are sure that the number of `enqueue`s is not larger than the capacity of the queue, the wraparound is not necessary. As with stacks, `dequeue`s are rarely performed unless the calling routines are certain that the queue is not empty. Thus error checks are frequently skipped for this operation, except in critical code. This is generally not justifiable, because the time savings that you are likely to achieve are minimal.

3.7.3 Applications of Queues

There are many algorithms that use queues to give efficient running times. Several of these are found in graph theory, and we will discuss them in Chapter 9. For now, we will give some simple examples of queue usage.

When jobs are submitted to a printer, they are arranged in order of arrival. Thus, essentially, jobs sent to a printer are placed on a queue.[1]

Virtually every real-life line is (supposed to be) a queue. For instance, lines at ticket counters are queues, because service is first-come first-served.

Another example concerns computer networks. There are many network setups of personal computers in which the disk is attached to one machine, known as the **file server.** Users on other machines are given access to files on a first-come first-served basis, so the data structure is a queue.

[1] We say *essentially* because jobs can be killed. This amounts to a deletion from the middle of the queue, which is a violation of the strict definition.

Further examples include the following:

- Calls to large companies are generally placed on a queue when all operators are busy.
- In large universities, where resources are limited, students must sign a waiting list if all computers are occupied. The student who has been at a computer the longest is forced off first, and the student who has been waiting the longest is the next user to be allowed on.

A whole branch of mathematics known as **queuing theory** deals with computing, probabilistically, how long users expect to wait on a line, how long the line gets, and other such questions. The answer depends on how frequently users arrive to the line and how long it takes to process a user once the user is served. Both of these parameters are given as probability distribution functions. In simple cases, an answer can be computed analytically. An example of an easy case would be a phone line with one operator. If the operator is busy, callers are placed on a waiting line (up to some maximum limit). This problem is important for businesses, because studies have shown that people are quick to hang up the phone.

If there are k operators, then this problem is much more difficult to solve. Problems that are difficult to solve analytically are often solved by a simulation. In our case, we would need to use a queue to perform the simulation. If k is large, we also need other data structures to do this efficiently. We shall see how to do this simulation in Chapter 6. We could then run the simulation for several values of k and choose the minimum k that gives a reasonable waiting time.

Additional uses for queues abound, and as with stacks, it is staggering that such a simple data structure can be so important.

Summary

This chapter describes the concept of ADTs and illustrates the concept with three of the most common abstract data types. The primary objective is to separate the implementation of the ADTs from their function. The program must know what the operations do, but it is actually better off not knowing how it is done.

Lists, stacks, and queues are perhaps the three fundamental data structures in all of computer science, and their use is documented through a host of examples. In particular, we saw how stacks are used to keep track of function calls and how recursion is actually implemented. This is important to understand, not just because it makes procedural languages possible, but because knowing how recursion is implemented removes a good deal of the mystery that surrounds its use. Although recursion is very powerful, it is not an entirely free operation; misuse and abuse of recursion can result in programs crashing.

Exercises

3.1 You are given a list, `L`, and another list, `P`, containing integers sorted in ascending order. The operation `printLots(L,P)` will print the elements in `L` that are in positions specified by `P`. For instance, if $P = 1, 3, 4, 6$, the elements in positions 1, 3, 4, and 6 in `L` are printed. Write the procedure `printLots(L,P)`. You may use only the public STL container operations. What is the running time of your procedure?

3.2 Swap two adjacent elements by adjusting only the links (and not the data) using
a. singly linked lists
b. doubly linked lists

3.3 Implement the STL `find` routine that returns the `iterator` containing the first occurrence of `x` in the range that begins at `start` and extends up to but not including `end`. If `x` is not found, `end` is returned. This is a nonclass (global function) with signature

```
template <typename Iterator, typename Object>
iterator find( Iterator start, Iterator end, const Object & x );
```

3.4 Given two sorted lists, L_1 and L_2, write a procedure to compute $L_1 \cap L_2$ using only the basic list operations.

3.5 Given two sorted lists, L_1 and L_2, write a procedure to compute $L_1 \cup L_2$ using only the basic list operations.

3.6 The *Josephus problem* is the following game: N people, numbered 1 to N, are sitting in a circle. Starting at person 1, a hot potato is passed. After M passes, the person holding the hot potato is eliminated, the circle closes ranks, and the game continues with the person who was sitting after the eliminated person picking up the hot potato. The last remaining person wins. Thus, if $M = 0$ and $N = 5$, players are eliminated in order, and player 5 wins. If $M = 1$ and $N = 5$, the order of elimination is 2, 4, 1, 5.
a. Write a program to solve the Josephus problem for general values of M and N. Try to make your program as efficient as possible. Make sure you dispose of cells.
b. What is the running time of your program?
c. If $M = 1$, what is the running time of your program? How is the actual speed affected by the `delete` routine for large values of N ($N > 100{,}000$)?

3.7 Modify the `Vector` class to add bounds checks for indexing.

3.8 Add `insert` and `erase` to the `Vector` class.

3.9 According to the C++ standard, for the `vector`, a call to `push_back`, `pop_back`, `insert`, or `erase` invalidates (potentially makes stale) all iterators viewing the `vector`. Why?

3.10 Modify the `Vector` class to provide stringent iterator checking by making iterators class types rather than pointer variables. The hardest part is dealing with stale iterators, as described in Exercise 3.9.

3.11 Assume that a singly linked list is implemented with a header node, but no tail node, and that it maintains only a pointer to the header node. Write a class that includes methods to
a. return the size of the linked list
b. print the linked list
c. test if a value `x` is contained in the linked list
d. add a value `x` if it is not already contained in the linked list
e. remove a value `x` if it is contained in the linked list

3.12 Repeat Exercise 3.11, maintaining the singly linked list in sorted order.

3.13 Add support for `operator-` to the `List` iterator classes.

3.14 Looking ahead in an STL iterator requires an application of `operator++`, which in turn advances the iterator. In some cases looking at the next item in the list, without advancing to it, may be preferable. Write the member function with the declaration

```
const_iterator operator+( int k ) const;
```

to facilitate this in a general case. The binary `operator+` returns an iterator that corresponds to `k` positions ahead of current.

3.15 Add the `splice` operation to the `List` class. The method declaration

```
void splice( iterator position, List<T> & lst );
```

removes all the items from `lst`, placing them prior to `position` in `List *this`. `lst` and `*this` must be different lists. Your routine must run in constant time.

3.16 Add reverse iterators to the STL `List` class implementation. Define `reverse_iterator` and `const_reverse_iterator`. Add the methods `rbegin` and `rend` to return appropriate reverse iterators representing the position prior to the endmarker and the position that is the header node. Reverse iterators internally reverse the meaning of the `++` and `--` operators. You should be able to print a list `L` in reverse by using the code

```
List<Object>::reverse_iterator itr = L.rbegin( );
while( itr != L.rend( ) )
    cout << *itr++ << endl;
```

3.17 Modify the `List` class to provide stringent iterator checking by using the ideas suggested at the end of Section 3.5.

3.18 When an `erase` method is applied to a `list`, it invalidates any `iterator` that is referencing the removed node. Such an iterator is called **stale**. Describe an efficient algorithm that guarantees that any operation on a stale iterator acts as though the iterator's `current` is `nullptr`. Note that there may be many stale iterators. You must explain which classes need to be rewritten in order to implement your algorithm.

3.19 Rewrite the `List` class without using header and tail nodes and describe the differences between the class and the class provided in Section 3.5.

3.20 An alternative to the deletion strategy we have given is to use **lazy deletion.** To delete an element, we merely mark it deleted (using an extra bit field). The number of deleted and nondeleted elements in the list is kept as part of the data structure. If there are as many deleted elements as nondeleted elements, we traverse the entire list, performing the standard deletion algorithm on all marked nodes.

a. List the advantages and disadvantages of lazy deletion.
b. Write routines to implement the standard linked list operations using lazy deletion.

3.21 Write a program to check for balancing symbols in the following languages:

a. Pascal (`begin/end`, `()`, `[]`, {}).
b. C++ (`/* */`, `()`, `[]`, {}).
* c. Explain how to print out an error message that is likely to reflect the probable cause.

3.22 Write a program to evaluate a postfix expression.

3.23 a. Write a program to convert an infix expression that includes `(`, `)`, `+`, `-`, `*`, and `/` to postfix.
b. Add the exponentiation operator to your repertoire.
c. Write a program to convert a postfix expression to infix.

3.24 Write routines to implement two stacks using only one array. Your stack routines should not declare an overflow unless every slot in the array is used.

3.25 * a. Propose a data structure that supports the stack `push` and `pop` operations and a third operation `findMin`, which returns the smallest element in the data structure, all in $O(1)$ worst-case time.
* b. Prove that if we add the fourth operation `deleteMin` which finds and removes the smallest element, then at least one of the operations must take $\Omega(\log N)$ time. (This requires reading Chapter 7.)

* **3.26** Show how to implement three stacks in one array.

3.27 If the recursive routine in Section 2.4 used to compute Fibonacci numbers is run for $N = 50$, is stack space likely to run out? Why or why not?

3.28 A *deque* is a data structure consisting of a list of items on which the following operations are possible:

`push(x)`: Insert item `x` on the front end of the deque.

`pop()`: Remove the front item from the deque and return it.

`inject(x)`: Insert item `x` on the rear end of the deque.

`eject()`: Remove the rear item from the deque and return it.

Write routines to support the deque that take $O(1)$ time per operation.

3.29 Write an algorithm for printing a singly linked list in reverse, using only constant extra space. This instruction implies that you cannot use recursion but you may assume that your algorithm is a list member function. Can such an algorithm be written if the routine is a constant member function?

3.30 a. Write an array implementation of self-adjusting lists. In a **self-adjusting list**, all insertions are performed at the front. A self-adjusting list adds a `find` operation, and when an element is accessed by a `find`, it is moved to the front of the list without changing the relative order of the other items.
b. Write a linked list implementation of self-adjusting lists.
* c. Suppose each element has a fixed probability, p_i, of being accessed. Show that the elements with highest access probability are expected to be close to the front.

3.31 Efficiently implement a stack class using a singly linked list, with no header or tail nodes.

3.32 Efficiently implement a queue class using a singly linked list, with no header or tail nodes.

3.33 Efficiently implement a queue class using a circular array. You may use a `vector` (rather than a primitive array) as the underlying array structure.

3.34 A linked list contains a cycle if, starting from some node p, following a sufficient number of `next` links brings us back to node p. p does not have to be the first node

in the list. Assume that you are given a linked list that contains N nodes; however, the value of N is unknown.

a. Design an $O(N)$ algorithm to determine if the list contains a cycle. You may use $O(N)$ extra space.

* b. Repeat part (a), but use only $O(1)$ extra space. (*Hint:* Use two iterators that are initially at the start of the list but advance at different speeds.)

3.35 One way to implement a queue is to use a circular linked list. In a **circular linked list**, the last node's `next` pointer points at the first node. Assume the list does not contain a header and that we can maintain, at most, one iterator corresponding to a node in the list. For which of the following representations can all basic queue operations be performed in constant worst-case time? Justify your answers.

a. Maintain an iterator that corresponds to the first item in the list.

b. Maintain an iterator that corresponds to the last item in the list.

3.36 Suppose we have a pointer to a node in a singly linked list that is guaranteed *not to be the last node* in the list. We do not have pointers to any other nodes (except by following links). Describe an $O(1)$ algorithm that logically removes the value stored in such a node from the linked list, maintaining the integrity of the linked list. (*Hint:* Involve the next node.)

3.37 Suppose that a singly linked list is implemented with both a header and a tail node. Describe constant-time algorithms to

a. insert item x before position p (given by an iterator)

b. remove the item stored at position p (given by an iterator)

CHAPTER 4

Trees

For large amounts of input, the linear access time of linked lists is prohibitive. In this chapter, we look at a simple data structure for which the average running time of most operations is $O(\log N)$. We also sketch a conceptually simple modification to this data structure that guarantees the above time bound in the worst case and discuss a second modification that essentially gives an $O(\log N)$ running time per operation for a long sequence of instructions.

The data structure that we are referring to is known as a **binary search tree**. The binary search tree is the basis for the implementation of two library collections classes, `set` and `map`, which are used in many applications. *Trees* in general are very useful abstractions in computer science, so we will discuss their use in other, more general applications. In this chapter, we will . . .

- See how trees are used to implement the file system of several popular operating systems.
- See how trees can be used to evaluate arithmetic expressions.
- Show how to use trees to support searching operations in $O(\log N)$ average time and how to refine these ideas to obtain $O(\log N)$ worst-case bounds. We will also see how to implement these operations when the data are stored on a disk.
- Discuss and use the `set` and `map` classes.

4.1 Preliminaries

A **tree** can be defined in several ways. One natural way to define a tree is recursively. A tree is a collection of nodes. The collection can be empty; otherwise, a tree consists of a distinguished node, r, called the **root,** and zero or more nonempty (sub)trees $T_1, T_2, \ldots, T_k$, each of whose roots are connected by a directed **edge** from r.

The root of each subtree is said to be a **child** of r, and r is the **parent** of each subtree root. Figure 4.1 shows a typical tree using the recursive definition.

From the recursive definition, we find that a tree is a collection of N nodes, one of which is the root, and $N - 1$ edges. That there are $N - 1$ edges follows from the fact that each edge connects some node to its parent, and every node except the root has one parent (see Fig. 4.2).

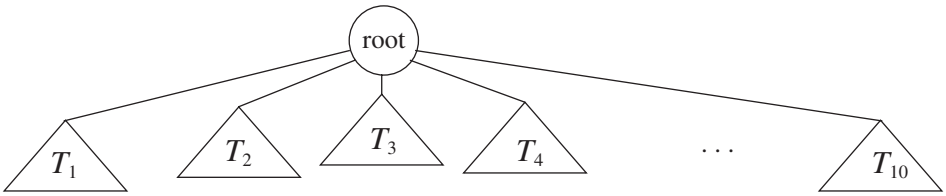


Figure 4.1 Generic tree

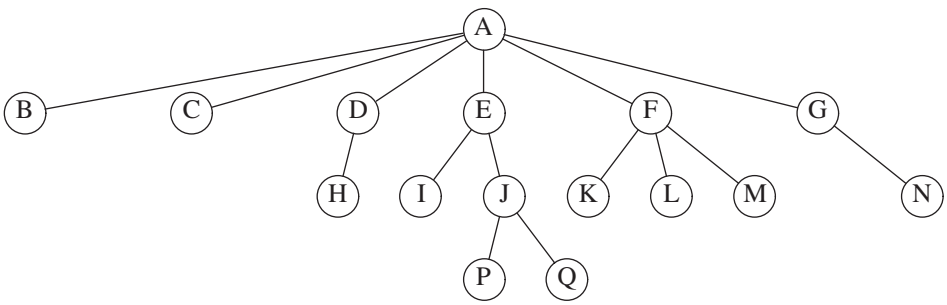


Figure 4.2 A tree

In the tree of Figure 4.2, the root is *A*. Node *F* has *A* as a parent and *K, L,* and *M* as children. Each node may have an arbitrary number of children, possibly zero. Nodes with no children are known as **leaves;** the leaves in the tree above are *B, C, H, I, P, Q, K, L, M*, and *N*. Nodes with the same parent are **siblings;** thus, *K, L,* and *M* are all siblings. **Grandparent** and **grandchild** relations can be defined in a similar manner.

A **path** from node n_1 to n_k is defined as a sequence of nodes $n_1, n_2, \ldots, n_k$ such that n_i is the parent of n_{i+1} for $1 \leq i < k$. The **length** of this path is the number of edges on the path, namely, $k - 1$. There is a path of length zero from every node to itself. Notice that in a tree there is exactly one path from the root to each node.

For any node n_i, the **depth** of n_i is the length of the unique path from the root to n_i. Thus, the root is at depth 0. The **height** of n_i is the length of the longest path from n_i to a leaf. Thus all leaves are at height 0. The height of a tree is equal to the height of the root. For the tree in Figure 4.2, *E* is at depth 1 and height 2; *F* is at depth 1 and height 1; the height of the tree is 3. The depth of a tree is equal to the depth of the deepest leaf; this is always equal to the height of the tree.

If there is a path from n_1 to n_2, then n_1 is an **ancestor** of n_2 and n_2 is a **descendant** of n_1. If $n_1 \neq n_2$, then n_1 is a **proper ancestor** of n_2 and n_2 is a **proper descendant** of n_1.

4.1.1 Implementation of Trees

One way to implement a tree would be to have in each node, besides its data, a link to each child of the node. However, since the number of children per node can vary so greatly and is not known in advance, it might be infeasible to make the children direct links in the data

```
1  struct TreeNode
2  {
3      Object    element;
4      TreeNode *firstChild;
5      TreeNode *nextSibling;
6  };
```

Figure 4.3 Node declarations for trees

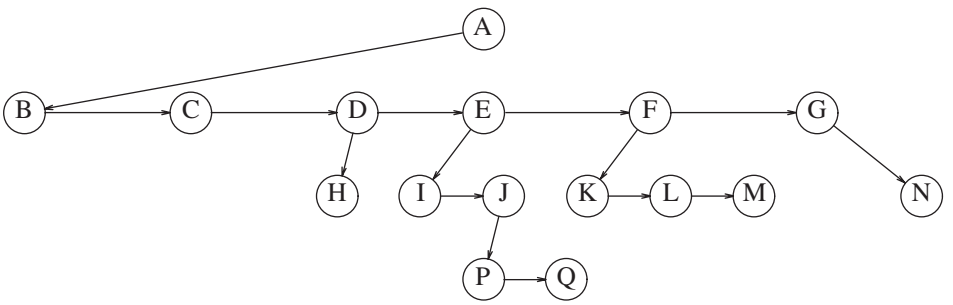


Figure 4.4 First child/next sibling representation of the tree shown in Figure 4.2

structure, because there would be too much wasted space. The solution is simple: Keep the children of each node in a linked list of tree nodes. The declaration in Figure 4.3 is typical.

Figure 4.4 shows how a tree might be represented in this implementation. Horizontal arrows that point downward are `firstChild` links. Arrows that go left to right are `nextSibling` links. Null links are not drawn, because there are too many.

In the tree of Figure 4.4, node *E* has both a link to a sibling (*F*) and a link to a child (*I*), while some nodes have neither.

4.1.2 Tree Traversals with an Application

There are many applications for trees. One of the popular uses is the directory structure in many common operating systems, including UNIX and DOS. Figure 4.5 is a typical directory in the UNIX file system.

The root of this directory is */usr*. (The asterisk next to the name indicates that */usr* is itself a directory.) */usr* has three children, *mark, alex*, and *bill*, which are themselves directories. Thus, */usr* contains three directories and no regular files. The filename */usr/mark/book/ch1.r* is obtained by following the leftmost child three times. Each / after the first indicates an edge; the result is the full **pathname**. This hierarchical file system is very popular because it allows users to organize their data logically. Furthermore, two files in different directories can share the same name, because they must have different paths from the root and thus have different pathnames. A directory in the UNIX file system is just a file with a list of all its children, so the directories are structured almost exactly in accordance

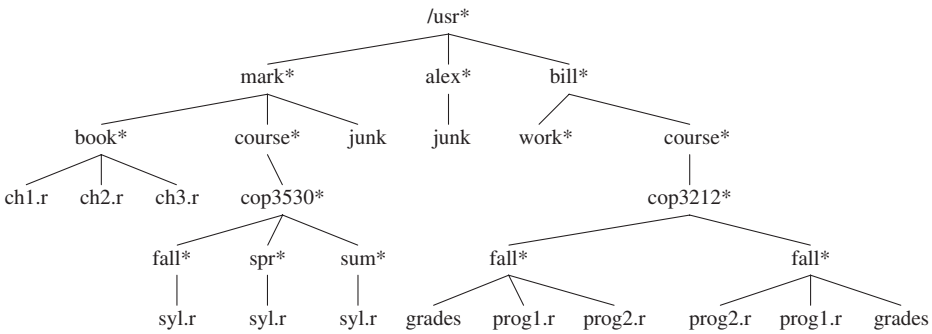


Figure 4.5 UNIX directory

```
    void FileSystem::listAll( int depth = 0 ) const
    {
1       printName( depth );  // Print the name of the object
2       if( isDirectory( ) )
3           for each file c in this directory (for each child)
4               c.listAll( depth + 1 );
    }
```

Figure 4.6 Pseudocode to list a directory in a hierarchical file system

with the type declaration above.[1] Indeed, on some versions of UNIX, if the normal command to print a file is applied to a directory, then the names of the files in the directory can be seen in the output (along with other non-ASCII information).

Suppose we would like to list the names of all of the files in the directory. Our output format will be that files that are depth d_i will have their names indented by d_i tabs. Our algorithm is given in Figure 4.6 as pseudocode.

The recursive function `listAll` needs to be started with a depth of 0 to signify no indenting for the root. This depth is an internal bookkeeping variable, and is hardly a parameter that a calling routine should be expected to know about. Thus, the default value of 0 is provided for `depth`.

The logic of the algorithm is simple to follow. The name of the file object is printed out with the appropriate number of tabs. If the entry is a directory, then we process all children recursively, one by one. These children are one level deeper, and thus need to be indented an extra space. The output is in Figure 4.7.

This traversal strategy is known as a **preorder traversal**. In a preorder traversal, work at a node is performed before (*pre*) its children are processed. When this program is run, it is clear that line 1 is executed exactly once per node, since each name is output once. Since line 1 is executed at most once per node, line 2 must also be executed once per

[1] Each directory in the UNIX file system also has one entry that points to itself and another entry that points to the parent of the directory. Thus, technically, the UNIX file system is not a tree, but is treelike.

```
/usr
    mark
        book
            ch1.r
            ch2.r
            ch3.r
        course
            cop3530
                fall
                    syl.r
                spr
                    syl.r
                sum
                    syl.r
        junk
    alex
        junk
    bill
        work
        course
            cop3212
                fall
                    grades
                    prog1.r
                    prog2.r
                fall
                    prog2.r
                    prog1.r
                    grades
```

Figure 4.7 The (preorder) directory listing

node. Furthermore, line 4 can be executed at most once for each child of each node. But the number of children is exactly one less than the number of nodes. Finally, the `for` loop iterates once per execution of line 4 plus once each time the loop ends. Thus, the total amount of work is constant per node. If there are N file names to be output, then the running time is $O(N)$.

Another common method of traversing a tree is the **postorder traversal**. In a postorder traversal, the work at a node is performed after (*post*) its children are evaluated. As an example, Figure 4.8 represents the same directory structure as before, with the numbers in parentheses representing the number of disk blocks taken up by each file.

Since the directories are themselves files, they have sizes too. Suppose we would like to calculate the total number of blocks used by all the files in the tree. The most natural way to do this would be to find the number of blocks contained in the subdirectories */usr/mark* (30), */usr/alex* (9), and */usr/bill* (32). The total number of blocks is then the total in the

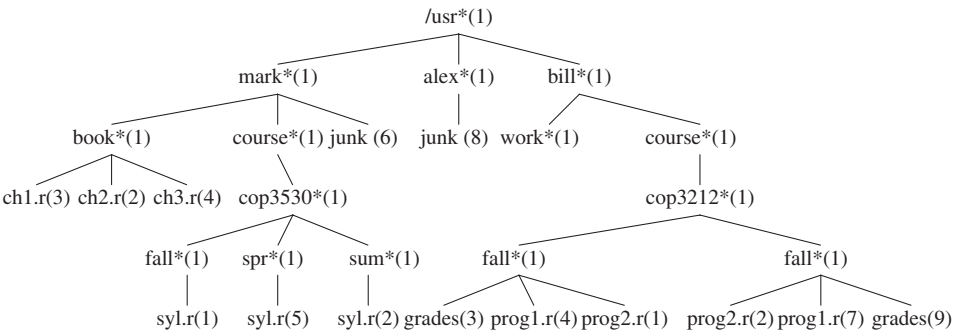


Figure 4.8 UNIX directory with file sizes obtained via postorder traversal

```
int FileSystem::size( ) const
{
    int totalSize = sizeOfThisFile( );

    if( isDirectory( ) )
        for each file c in this directory (for each child)
            totalSize += c.size( );

     return totalSize;
}
```

Figure 4.9 Pseudocode to calculate the size of a directory

subdirectories (71) plus the one block used by */usr*, for a total of 72. The pseudocode method `size` in Figure 4.9 implements this strategy.

If the current object is not a directory, then `size` merely returns the number of blocks it uses in the current object. Otherwise, the number of blocks used by the directory is added to the number of blocks (recursively) found in all the children. To see the difference between the postorder traversal strategy and the preorder traversal strategy, Figure 4.10 shows how the size of each directory or file is produced by the algorithm.

4.2 Binary Trees

A binary tree is a tree in which no node can have more than two children.

Figure 4.11 shows that a binary tree consists of a root and two subtrees, T_L and T_R, both of which could possibly be empty.

A property of a binary tree that is sometimes important is that the depth of an average binary tree is considerably smaller than N. An analysis shows that the average depth is $O(\sqrt{N})$, and that for a special type of binary tree, namely the *binary search tree,* the average value of the depth is $O(\log N)$. Unfortunately, the depth can be as large as $N - 1$, as the example in Figure 4.12 shows.

```
            ch1.r            3
            ch2.r            2
            ch3.r            4
        book                10
                    syl.r    1
                fall         2
                    syl.r    5
                spr          6
                    syl.r    2
                sum          3
            cop3530         12
        course              13
        junk                 6
    mark                    30
        junk                 8
    alex                     9
        work                 1
                    grades   3
                    prog1.r  4
                    prog2.r  1
                fall         9
                    prog2.r  2
                    prog1.r  7
                    grades   9
                fall        19
            cop3212         29
        course              30
    bill                    32
/usr                        72
```

Figure 4.10 Trace of the `size` function

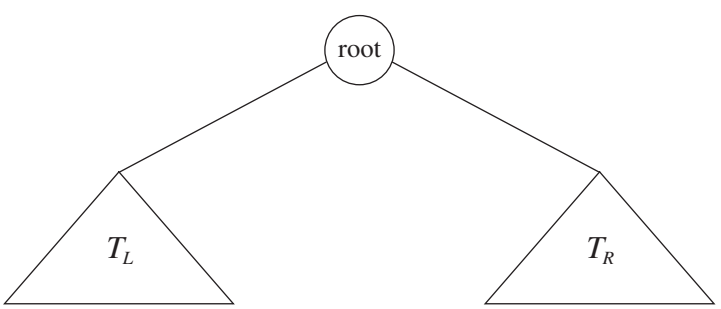


Figure 4.11 Generic binary tree

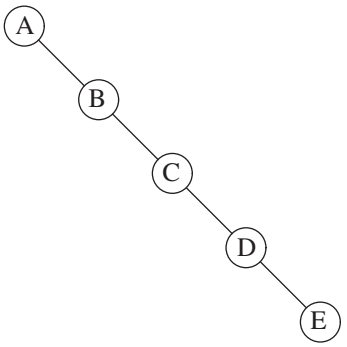


Figure 4.12 Worst-case binary tree

4.2.1 Implementation

Because a binary tree node has at most two children, we can keep direct links to them. The declaration of tree nodes is similar in structure to that for doubly linked lists, in that a node is a structure consisting of the `element` information plus two pointers (`left` and `right`) to other nodes (see Fig. 4.13).

We could draw the binary trees using the rectangular boxes that are customary for linked lists, but trees are generally drawn as circles connected by lines, because they are actually graphs. We also do not explicitly draw `nullptr` links when referring to trees, because every binary tree with N nodes would require $N + 1$ `nullptr` links.

Binary trees have many important uses not associated with searching. One of the principal uses of binary trees is in the area of compiler design, which we will now explore.

4.2.2 An Example: Expression Trees

Figure 4.14 shows an example of an **expression tree**. The leaves of an expression tree are **operands**, such as constants or variable names, and the other nodes contain **operators**. This particular tree happens to be binary, because all the operators are binary, and although this is the simplest case, it is possible for nodes to have more than two children. It is also possible for a node to have only one child, as is the case with the **unary minus** operator. We can evaluate an expression tree, T, by applying the operator at the root to the values

```
struct BinaryNode
{
    Object      element;      // The data in the node
    BinaryNode *left;         // Left child
    BinaryNode *right;        // Right child
};
```

Figure 4.13 Binary tree node class (pseudocode)

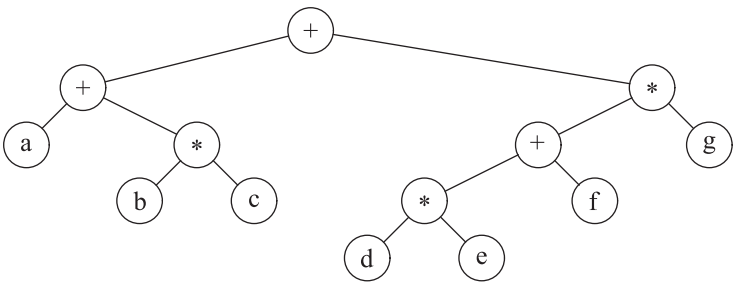


Figure 4.14 Expression tree for `(a + b * c) + ((d * e + f ) * g)`

obtained by recursively evaluating the left and right subtrees. In our example, the left subtree evaluates to `a + (b * c)` and the right subtree evaluates to `((d * e) + f) * g`. The entire tree therefore represents `(a + (b * c)) + (((d * e) + f) * g)`.

We can produce an (overly parenthesized) infix expression by recursively producing a parenthesized left expression, then printing out the operator at the root, and finally recursively producing a parenthesized right expression. This general strategy (left, node, right) is known as an **inorder traversal**; it is easy to remember because of the type of expression it produces.

An alternate traversal strategy is to recursively print out the left subtree, the right subtree, and then the operator. If we apply this strategy to our tree above, the output is `a b c * + d e * f + g * +`, which is easily seen to be the postfix representation of Section 3.6.3. This traversal strategy is generally known as a *postorder* traversal. We have seen this traversal strategy earlier in Section 4.1.

A third traversal strategy is to print out the operator first and then recursively print out the left and right subtrees. The resulting expression, `+ + a * b c * + * d e f g`, is the less useful *prefix* notation, and the traversal strategy is a *preorder* traversal, which we have also seen earlier in Section 4.1. We will return to these traversal strategies later in the chapter.

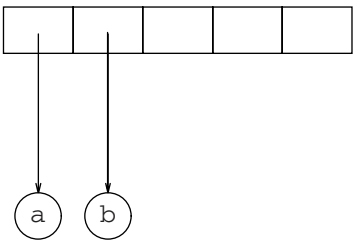
Constructing an Expression Tree

We now give an algorithm to convert a postfix expression into an expression tree. Since we already have an algorithm to convert infix to postfix, we can generate expression trees from the two common types of input. The method we describe strongly resembles the postfix evaluation algorithm of Section 3.6.3. We read our expression one symbol at a time. If the symbol is an operand, we create a one-node tree and push a pointer to it onto a stack. If the symbol is an operator, we pop (pointers) to two trees T_1 and T_2 from the stack (T_1 is popped first) and form a new tree whose root is the operator and whose left and right children point to T_2 and T_1, respectively. A pointer to this new tree is then pushed onto the stack.

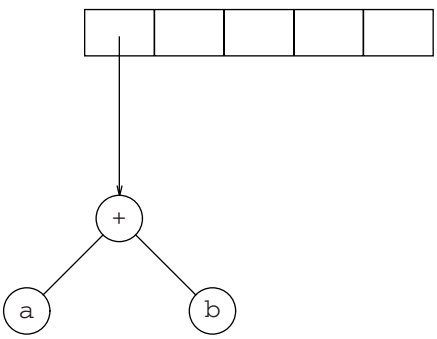
As an example, suppose the input is

```
a b + c d e + * *
```

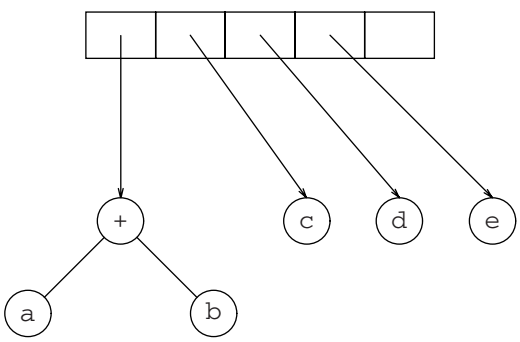
The first two symbols are operands, so we create one-node trees and push pointers to them onto a stack.[2]



Next, a + is read, so two pointers to trees are popped, a new tree is formed, and a pointer to it is pushed onto the stack.

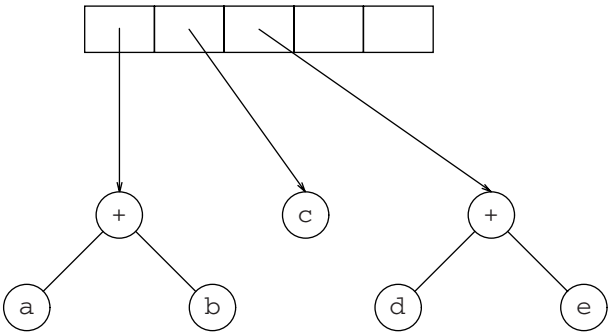


Next, `c`, `d`, and `e` are read, and for each a one-node tree is created and a pointer to the corresponding tree is pushed onto the stack.

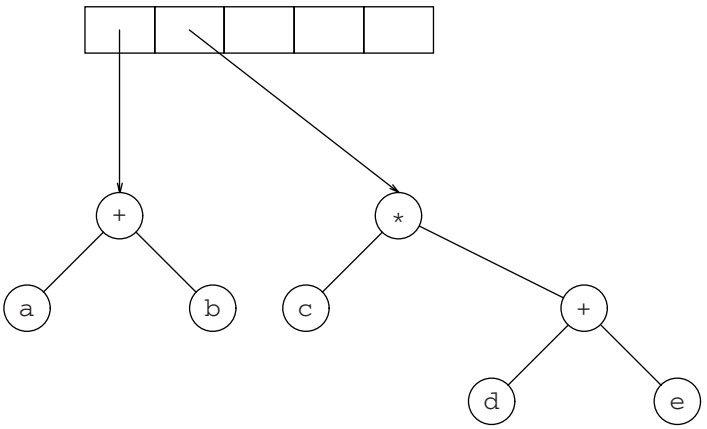


Now a + is read, so two trees are merged.

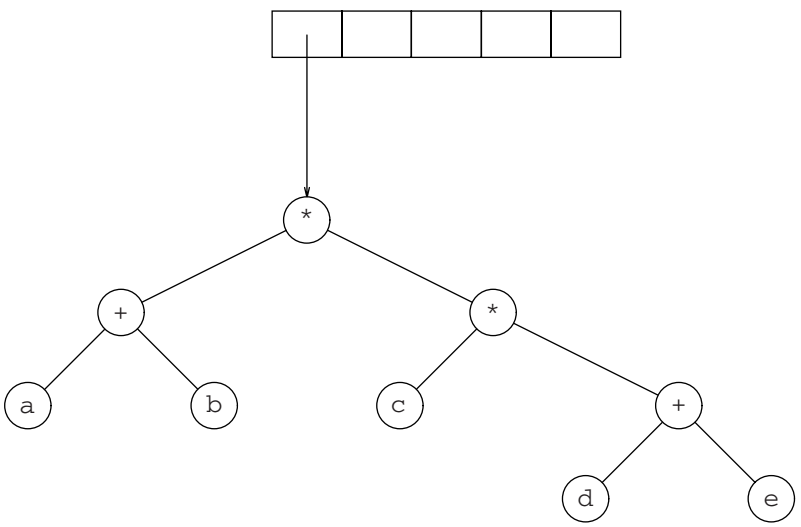
[2] For convenience, we will have the stack grow from left to right in the diagrams.



Continuing, a * is read, so we pop two tree pointers and form a new tree with a * as root.



Finally, the last symbol is read, two trees are merged, and a pointer to the final tree is left on the stack.



4.3 The Search Tree ADT—Binary Search Trees

An important application of binary trees is their use in searching. Let us assume that each node in the tree stores an item. In our examples, we will assume, for simplicity, that these are integers, although arbitrarily complex items are easily handled in C++. We will also assume that all the items are distinct, and we will deal with duplicates later.

The property that makes a binary tree into a binary search tree is that for every node, X, in the tree, the values of all the items in its left subtree are smaller than the item in X, and the values of all the items in its right subtree are larger than the item in X. Notice that this implies that all the elements in the tree can be ordered in some consistent manner. In Figure 4.15, the tree on the left is a binary search tree, but the tree on the right is not. The tree on the right has a node with item 7 in the left subtree of a node with item 6 (which happens to be the root).

We now give brief descriptions of the operations that are usually performed on binary search trees. Note that because of the recursive definition of trees, it is common to write these routines recursively. Because the average depth of a binary search tree turns out to be $O(\log N)$, we generally do not need to worry about running out of stack space.

Figure 4.16 shows the interface for the `BinarySearchTree` class template. There are several things worth noticing. Searching is based on the `<` operator that must be defined for the particular `Comparable` type. Specifically, item `x` matches `y` if both `x<y` and `y<x` are `false`. This allows `Comparable` to be a complex type (such as an employee record), with a comparison function defined on only part of the type (such as the social security number data member or salary). Section 1.6.3 illustrates the general technique of designing a class that can be used as a `Comparable`. An alternative, described in Section 4.3.1, is to allow a function object.

The data member is a pointer to the root node; this pointer is `nullptr` for empty trees. The `public` member functions use the general technique of calling `private` recursive functions. An example of how this is done for `contains`, `insert`, and `remove` is shown in Figure 4.17.

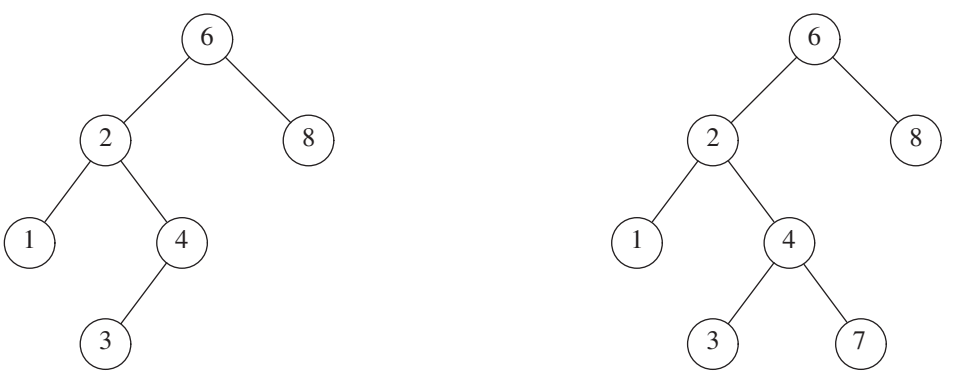


Figure 4.15 Two binary trees (only the left tree is a search tree)

```
template <typename Comparable>
class BinarySearchTree
{
  public:
    BinarySearchTree( );
    BinarySearchTree( const BinarySearchTree & rhs );
    BinarySearchTree( BinarySearchTree && rhs );
    ~BinarySearchTree( );

    const Comparable & findMin( ) const;
    const Comparable & findMax( ) const;
    bool contains( const Comparable & x ) const;
    bool isEmpty( ) const;
    void printTree( ostream & out = cout ) const;

    void makeEmpty( );
    void insert( const Comparable & x );
    void insert( Comparable && x );
    void remove( const Comparable & x );

    BinarySearchTree & operator=( const BinarySearchTree & rhs );
    BinarySearchTree & operator=( BinarySearchTree && rhs );

  private:
    struct BinaryNode
    {
        Comparable element;
        BinaryNode *left;
        BinaryNode *right;

        BinaryNode( const Comparable & theElement, BinaryNode *lt, BinaryNode *rt )
          : element{ theElement }, left{ lt }, right{ rt } { }

        BinaryNode( Comparable && theElement, BinaryNode *lt, BinaryNode *rt )
          : element{ std::move( theElement ) }, left{ lt }, right{ rt } { }
    };

    BinaryNode *root;

    void insert( const Comparable & x, BinaryNode * & t );
    void insert( Comparable && x, BinaryNode * & t );
    void remove( const Comparable & x, BinaryNode * & t );
    BinaryNode * findMin( BinaryNode *t ) const;
    BinaryNode * findMax( BinaryNode *t ) const;
    bool contains( const Comparable & x, BinaryNode *t ) const;
    void makeEmpty( BinaryNode * & t );
    void printTree( BinaryNode *t, ostream & out ) const;
    BinaryNode * clone( BinaryNode *t ) const;
};
```

Figure 4.16 Binary search tree class skeleton

```
1  /**
2   * Returns true if x is found in the tree.
3   */
4  bool contains( const Comparable & x ) const
5  {
6      return contains( x, root );
7  }
8
9  /**
10  * Insert x into the tree; duplicates are ignored.
11  */
12 void insert( const Comparable & x )
13 {
14     insert( x, root );
15 }
16
17 /**
18  * Remove x from the tree. Nothing is done if x is not found.
19  */
20 void remove( const Comparable & x )
21 {
22     remove( x, root );
23 }
```

Figure 4.17 Illustration of public member function calling private recursive member function

Several of the `private` member functions use the technique of passing a pointer variable using call-by-reference. This allows the `public` member functions to pass a pointer to the root to the `private` recursive member functions. The recursive functions can then change the value of the root so that the `root` points to another node. We will describe the technique in more detail when we examine the code for `insert`.

We can now describe some of the `private` methods.

4.3.1 contains

This operation requires returning `true` if there is a node in tree T that has item X, or `false` if there is no such node. The structure of the tree makes this simple. If T is empty, then we can just return `false`. Otherwise, if the item stored at T is X, we can return `true`. Otherwise, we make a recursive call on a subtree of T, either left or right, depending on the relationship of X to the item stored in T. The code in Figure 4.18 is an implementation of this strategy.

```
/**
 * Internal method to test if an item is in a subtree.
 * x is item to search for.
 * t is the node that roots the subtree.
 */
bool contains( const Comparable & x, BinaryNode *t ) const
{
    if( t == nullptr )
        return false;
    else if( x < t->element )
        return contains( x, t->left );
    else if( t->element < x )
        return contains( x, t->right );
    else
        return true;    // Match
}
```

Figure 4.18 `contains` operation for binary search trees

Notice the order of the tests. It is crucial that the test for an empty tree be performed first, since otherwise, we would generate a run time error attempting to access a data member through a `nullptr` pointer. The remaining tests are arranged with the least likely case last. Also note that both recursive calls are actually tail recursions and can be easily removed with a `while` loop. The use of tail recursion is justifiable here because the simplicity of algorithmic expression compensates for the decrease in speed, and the amount of stack space used is expected to be only $O(\log N)$.

Figure 4.19 shows the trivial changes required to use a function object rather than requiring that the items be `Comparable`. This mimics the idioms in Section 1.6.4.

4.3.2 `findMin` and `findMax`

These `private` routines return a pointer to the node containing the smallest and largest elements in the tree, respectively. To perform a `findMin`, start at the root and go left as long as there is a left child. The stopping point is the smallest element. The `findMax` routine is the same, except that branching is to the right child.

Many programmers do not bother using recursion. We will code the routines both ways by doing `findMin` recursively and `findMax` nonrecursively (see Figs. 4.20 and 4.21).

Notice how we carefully handle the degenerate case of an empty tree. Although this is always important to do, it is especially crucial in recursive programs. Also notice that it is safe to change `t` in `findMax`, since we are only working with a copy of a pointer. Always be extremely careful, however, because a statement such as `t->right = t->right->right` will make changes.

```
1  template <typename Object, typename Comparator=less<Object>>
2  class BinarySearchTree
3  {
4    public:
5
6      // Same methods, with Object replacing Comparable
7
8    private:
9
10     BinaryNode *root;
11     Comparator isLessThan;
12
13     // Same methods, with Object replacing Comparable
14
15     /**
16      * Internal method to test if an item is in a subtree.
17      * x is item to search for.
18      * t is the node that roots the subtree.
19      */
20     bool contains( const Object & x, BinaryNode *t ) const
21     {
22         if( t == nullptr )
23             return false;
24         else if( isLessThan( x, t->element ) )
25             return contains( x, t->left );
26         else if( isLessThan( t->element, x ) )
27             return contains( x, t->right );
28         else
29             return true;    // Match
30     }
31 };
```

Figure 4.19 Illustrates use of a function object to implement binary search tree

4.3.3 insert

The insertion routine is conceptually simple. To insert *X* into tree *T*, proceed down the tree as you would with a `contains`. If *X* is found, do nothing. Otherwise, insert *X* at the last spot on the path traversed. Figure 4.22 shows what happens. To insert 5, we traverse the tree as though a `contains` were occurring. At the node with item 4, we need to go right, but there is no subtree, so 5 is not in the tree, and this is the correct spot to place 5.

Duplicates can be handled by keeping an extra field in the node record indicating the frequency of occurrence. This adds some extra space to the entire tree but is better than putting duplicates in the tree (which tends to make the tree very deep). Of course,

```
/**
 * Internal method to find the smallest item in a subtree t.
 * Return node containing the smallest item.
 */
BinaryNode * findMin( BinaryNode *t ) const
{
    if( t == nullptr )
        return nullptr;
    if( t->left == nullptr )
        return t;
    return findMin( t->left );
}
```

Figure 4.20 Recursive implementation of `findMin` for binary search trees

```
/**
 * Internal method to find the largest item in a subtree t.
 * Return node containing the largest item.
 */
BinaryNode * findMax( BinaryNode *t ) const
{
    if( t != nullptr )
        while( t->right != nullptr )
            t = t->right;
    return t;
}
```

Figure 4.21 Nonrecursive implementation of `findMax` for binary search trees

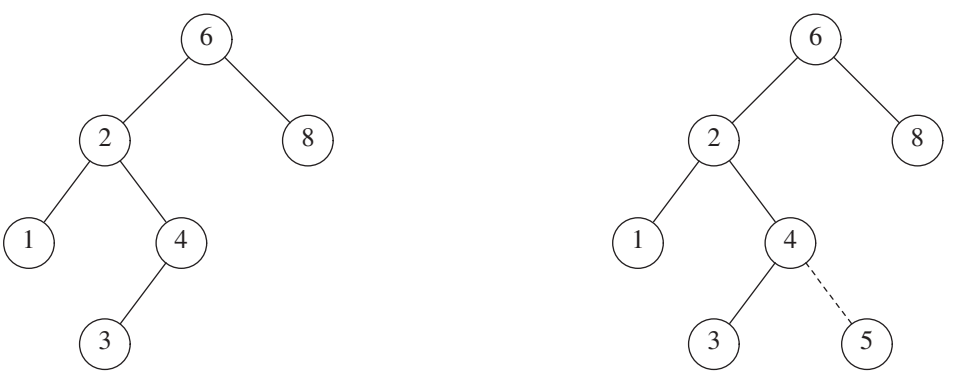


Figure 4.22 Binary search trees before and after inserting 5

this strategy does not work if the key that guides the < operator is only part of a larger structure. If that is the case, then we can keep all of the structures that have the same key in an auxiliary data structure, such as a list or another search tree.

Figure 4.23 shows the code for the insertion routine. Lines 12 and 14 recursively insert and attach x into the appropriate subtree. Notice that in the recursive routine, the only time that t changes is when a new leaf is created. When this happens, it means that the recursive routine has been called from some other node, p, which is to be the leaf's parent. The call

```
1  /**
2   * Internal method to insert into a subtree.
3   * x is the item to insert.
4   * t is the node that roots the subtree.
5   * Set the new root of the subtree.
6   */
7  void insert( const Comparable & x, BinaryNode * & t )
8  {
9      if( t == nullptr )
10         t = new BinaryNode{ x, nullptr, nullptr };
11     else if( x < t->element )
12         insert( x, t->left );
13     else if( t->element < x )
14         insert( x, t->right );
15     else
16         ;  // Duplicate; do nothing
17 }
18
19 /**
20  * Internal method to insert into a subtree.
21  * x is the item to insert by moving.
22  * t is the node that roots the subtree.
23  * Set the new root of the subtree.
24  */
25 void insert( Comparable && x, BinaryNode * & t )
26 {
27     if( t == nullptr )
28         t = new BinaryNode{ std::move( x ), nullptr, nullptr };
29     else if( x < t->element )
30         insert( std::move( x ), t->left );
31     else if( t->element < x )
32         insert( std::move( x ), t->right );
33     else
34         ;  // Duplicate; do nothing
35 }
```

Figure 4.23 Insertion into a binary search tree

will be `insert(x,p->left)` or `insert(x,p->right)`. Either way, `t` is now a reference to either `p->left` or `p->right`, meaning that `p->left` or `p->right` will be changed to point at the new node. All in all, a slick maneuver.

4.3.4 remove

As is common with many data structures, the hardest operation is deletion. Once we have found the node to be deleted, we need to consider several possibilities.

If the node is a leaf, it can be deleted immediately. If the node has one child, the node can be deleted after its parent adjusts a link to bypass the node (we will draw the link directions explicitly for clarity). See Figure 4.24.

The complicated case deals with a node with two children. The general strategy is to replace the data of this node with the smallest data of the right subtree (which is easily found) and recursively delete that node (which is now empty). Because the smallest node in the right subtree cannot have a left child, the second `remove` is an easy one. Figure 4.25 shows an initial tree and the result of a deletion. The node to be deleted is the left child of the root; the key value is 2. It is replaced with the smallest data in its right subtree (3), and then that node is deleted as before.

The code in Figure 4.26 performs deletion. It is inefficient because it makes two passes down the tree to find and delete the smallest node in the right subtree when this is appropriate. It is easy to remove this inefficiency by writing a special `removeMin` method, and we have left it in only for simplicity.

If the number of deletions is expected to be small, then a popular strategy to use is **lazy deletion:** When an element is to be deleted, it is left in the tree and merely *marked* as being deleted. This is especially popular if duplicate items are present, because then the data member that keeps count of the frequency of appearance can be decremented. If the number of real nodes in the tree is the same as the number of "deleted" nodes, then the depth of the tree is only expected to go up by a small constant (why?), so there is a very small time penalty associated with lazy deletion. Also, if a deleted item is reinserted, the overhead of allocating a new cell is avoided.

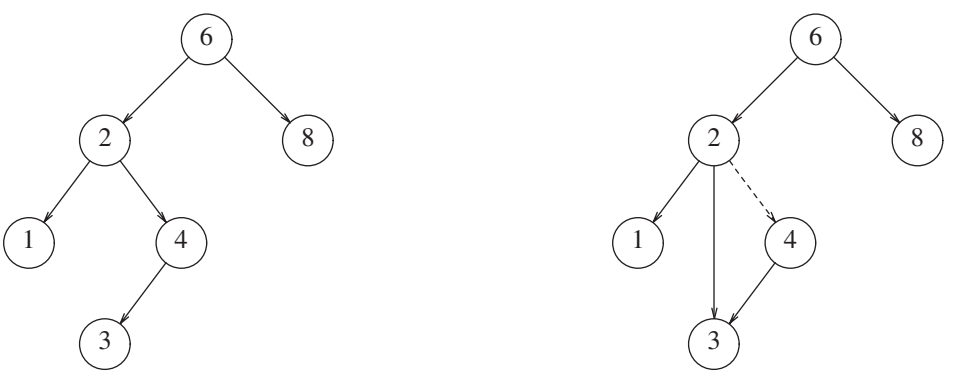


Figure 4.24 Deletion of a node (4) with one child, before and after

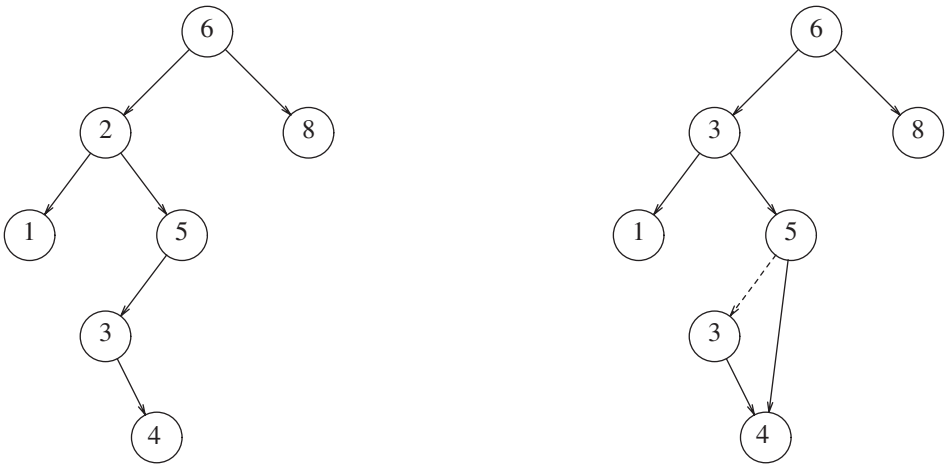


Figure 4.25 Deletion of a node (2) with two children, before and after

```
1  /**
2   * Internal method to remove from a subtree.
3   * x is the item to remove.
4   * t is the node that roots the subtree.
5   * Set the new root of the subtree.
6   */
7  void remove( const Comparable & x, BinaryNode * & t )
8  {
9      if( t == nullptr )
10         return;   // Item not found; do nothing
11     if( x < t->element )
12         remove( x, t->left );
13     else if( t->element < x )
14         remove( x, t->right );
15     else if( t->left != nullptr && t->right != nullptr ) // Two children
16     {
17         t->element = findMin( t->right )->element;
18         remove( t->element, t->right );
19     }
20     else
21     {
22         BinaryNode *oldNode = t;
23         t = ( t->left != nullptr ) ? t->left : t->right;
24         delete oldNode;
25     }
26 }
```

Figure 4.26 Deletion routine for binary search trees

4.3.5 Destructor and Copy Constructor

As usual, the destructor calls `makeEmpty`. The public `makeEmpty` (not shown) simply calls the private recursive version. As shown in Figure 4.27, after recursively processing `t`'s children, a call to `delete` is made for `t`. Thus all nodes are recursively reclaimed. Notice that at the end, `t`, and thus `root`, is changed to point at `nullptr`. The copy constructor, shown in Figure 4.28, follows the usual procedure, first initializing `root` to `nullptr` and then making a copy of `rhs`. We use a very slick recursive function named `clone` to do all the dirty work.

4.3.6 Average-Case Analysis

Intuitively, we expect that all of the operations described in this section, except `makeEmpty` and copying, should take $O(\log N)$ time, because in constant time we descend a level in the tree, thus operating on a tree that is now roughly half as large. Indeed, the running time of all the operations (except `makeEmpty` and copying) is $O(d)$, where d is the depth of the node containing the accessed item (in the case of `remove`, this may be the replacement node in the two-child case).

We prove in this section that the average depth over all nodes in a tree is $O(\log N)$ on the assumption that all insertion sequences are equally likely.

The sum of the depths of all nodes in a tree is known as the **internal path length.** We will now calculate the average internal path length of a binary search tree, where the average is taken over all possible insertion sequences into binary search trees.

```
1  /**
2   * Destructor for the tree
3   */
4  ~BinarySearchTree( )
5  {
6      makeEmpty( );
7  }
8  /**
9   * Internal method to make subtree empty.
10  */
11 void makeEmpty( BinaryNode * & t )
12 {
13     if( t != nullptr )
14     {
15         makeEmpty( t->left );
16         makeEmpty( t->right );
17         delete t;
18     }
19     t = nullptr;
20 }
```

Figure 4.27 Destructor and recursive `makeEmpty` member function

```
1  /**
2   * Copy constructor
3   */
4  BinarySearchTree( const BinarySearchTree & rhs ) : root{ nullptr }
5  {
6      root = clone( rhs.root );
7  }
8
9  /**
10  * Internal method to clone subtree.
11  */
12 BinaryNode * clone( BinaryNode *t ) const
13 {
14     if( t == nullptr )
15         return nullptr;
16     else
17         return new BinaryNode{ t->element, clone( t->left ), clone( t->right ) };
18 }
```

Figure 4.28 Copy constructor and recursive `clone` member function

Let $D(N)$ be the internal path length for some tree T of N nodes. $D(1) = 0$. An N-node tree consists of an i-node left subtree and an $(N - i - 1)$-node right subtree, plus a root at depth zero for $0 \leq i < N$. $D(i)$ is the internal path length of the left subtree with respect to its root. In the main tree, all these nodes are one level deeper. The same holds for the right subtree. Thus, we get the recurrence

$$D(N) = D(i) + D(N - i - 1) + N - 1$$

If all subtree sizes are equally likely, which is true for binary search trees (since the subtree size depends only on the relative rank of the first element inserted into the tree), but not binary trees, then the average value of both $D(i)$ and $D(N - i - 1)$ is $(1/N)\sum_{j=0}^{N-1} D(j)$. This yields

$$D(N) = \frac{2}{N}\left[\sum_{j=0}^{N-1} D(j)\right] + N - 1$$

This recurrence will be encountered and solved in Chapter 7, obtaining an average value of $D(N) = O(N \log N)$. Thus, the expected depth of any node is $O(\log N)$. As an example, the randomly generated 500-node tree shown in Figure 4.29 has nodes at expected depth 9.98.

It is tempting to say immediately that this result implies that the average running time of all the operations discussed in the previous section is $O(\log N)$, but this is not entirely true. The reason for this is that because of deletions, it is not clear that all binary search trees are equally likely. In particular, the deletion algorithm described above favors making the left subtrees deeper than the right, because we are always replacing a deleted node with a node from the right subtree. The exact effect of this strategy is still unknown, but

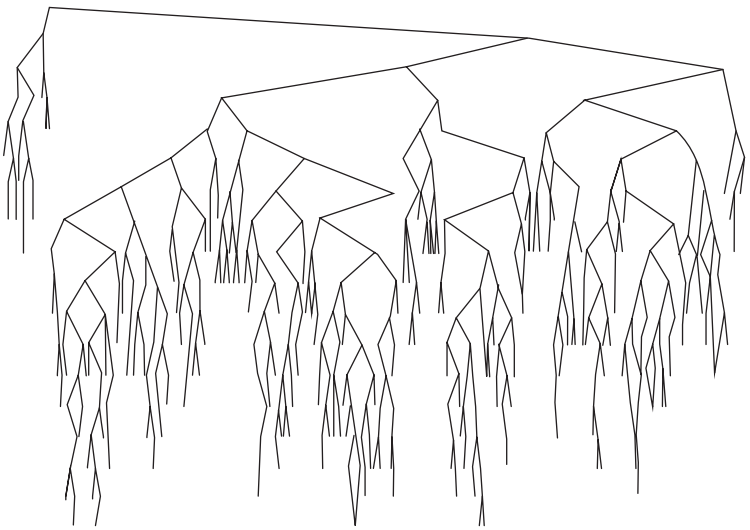


Figure 4.29 A randomly generated binary search tree

it seems only to be a theoretical novelty. It has been shown that if we alternate insertions and deletions $\Theta(N^2)$ times, then the trees will have an expected depth of $\Theta(\sqrt{N})$. After a quarter-million random `insert/remove` pairs, the tree that was somewhat right-heavy in Figure 4.29 looks decidedly unbalanced (average depth = 12.51) in Figure 4.30.

We could try to eliminate the problem by randomly choosing between the smallest element in the right subtree and the largest in the left when replacing the deleted element. This apparently eliminates the bias and should keep the trees balanced, but nobody has

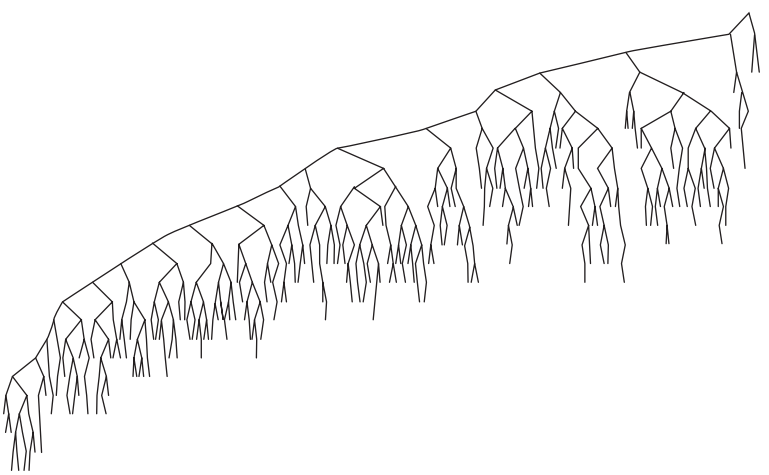


Figure 4.30 Binary search tree after $\Theta(N^2)$ `insert/remove` pairs

actually proved this. In any event, this phenomenon appears to be mostly a theoretical novelty, because the effect does not show up at all for small trees, and, stranger still, if $o(N^2)$ `insert/remove` pairs are used, then the tree seems to gain balance!

The main point of this discussion is that deciding what "average" means is generally extremely difficult and can require assumptions that may or may not be valid. In the absence of deletions, or when lazy deletion is used, we can conclude that the average running times of the operations above are $O(\log N)$. Except for strange cases like the one discussed above, this result is very consistent with observed behavior.

If the input comes into a tree presorted, then a series of `insert`s will take quadratic time and give a very expensive implementation of a linked list, since the tree will consist only of nodes with no left children. One solution to the problem is to insist on an extra structural condition called *balance:* No node is allowed to get too deep.

There are quite a few general algorithms to implement balanced trees. Most are quite a bit more complicated than a standard binary search tree, and all take longer on average for updates. They do, however, provide protection against the embarrassingly simple cases. Below, we will sketch one of the oldest forms of balanced search trees, the AVL tree.

A second method is to forgo the balance condition and allow the tree to be arbitrarily deep, but after every operation, a restructuring rule is applied that tends to make future operations efficient. These types of data structures are generally classified as **self-adjusting.** In the case of a binary search tree, we can no longer guarantee an $O(\log N)$ bound on any single operation but can show that any *sequence* of M operations takes total time $O(M \log N)$ in the worst case. This is generally sufficient protection against a bad worst case. The data structure we will discuss is known as a *splay tree;* its analysis is fairly intricate and is discussed in Chapter 11.

4.4 AVL Trees

An AVL (Adelson-Velskii and Landis) tree is a binary search tree with a **balance condition**. The balance condition must be easy to maintain, and it ensures that the depth of the tree is $O(\log N)$. The simplest idea is to require that the left and right subtrees have the same height. As Figure 4.31 shows, this idea does not force the tree to be shallow.

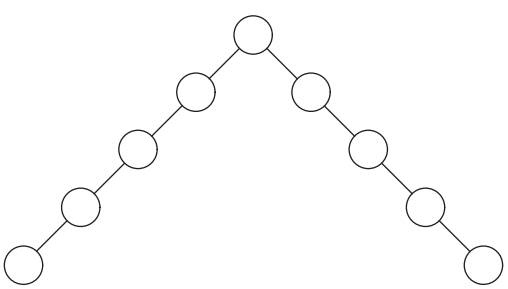


Figure 4.31 A bad binary tree. Requiring balance at the root is not enough.

Another balance condition would insist that every node must have left and right subtrees of the same height. If the height of an empty subtree is defined to be -1 (as is usual), then only perfectly balanced trees of $2^k - 1$ nodes would satisfy this criterion. Thus, although this guarantees trees of small depth, the balance condition is too rigid to be useful and needs to be relaxed.

An **AVL tree** is identical to a binary search tree, except that for every node in the tree, the height of the left and right subtrees can differ by at most 1. (The height of an empty tree is defined to be -1.) In Figure 4.32 the tree on the left is an AVL tree but the tree on the right is not. Height information is kept for each node (in the node structure). It can be shown that the height of an AVL tree is at most roughly $1.44 \log(N + 2) - 1.328$, but in practice it is only slightly more than $\log N$. As an example, the AVL tree of height 9 with the fewest nodes (143) is shown in Figure 4.33. This tree has as a left subtree an AVL tree of height 7 of minimum size. The right subtree is an AVL tree of height 8 of minimum size. This tells us that the minimum number of nodes, $S(h)$, in an AVL tree of height h is given by $S(h) = S(h-1) + S(h-2) + 1$. For $h = 0$, $S(h) = 1$. For $h = 1$, $S(h) = 2$. The function $S(h)$ is closely related to the Fibonacci numbers, from which the bound claimed above on the height of an AVL tree follows.

Thus, all the tree operations can be performed in $O(\log N)$ time, except possibly insertion and deletion. When we do an insertion, we need to update all the balancing information for the nodes on the path back to the root, but the reason that insertion is potentially difficult is that inserting a node could violate the AVL tree property. (For instance, inserting 6 into the AVL tree in Figure 4.32 would destroy the balance condition at the node with key 8.) If this is the case, then the property has to be restored before the insertion step is considered over. It turns out that this can always be done with a simple modification to the tree, known as a **rotation.**

After an insertion, only nodes that are on the path from the insertion point to the root might have their balance altered because only those nodes have their subtrees altered. As we follow the path up to the root and update the balancing information, we may find a node whose new balance violates the AVL condition. We will show how to rebalance the tree at the first (i.e., deepest) such node, and we will prove that this rebalancing guarantees that the entire tree satisfies the AVL property.

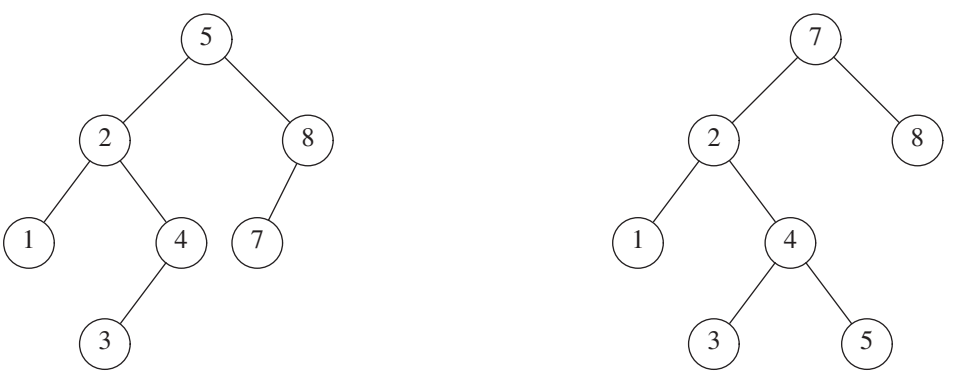


Figure 4.32 Two binary search trees. Only the left tree is AVL.

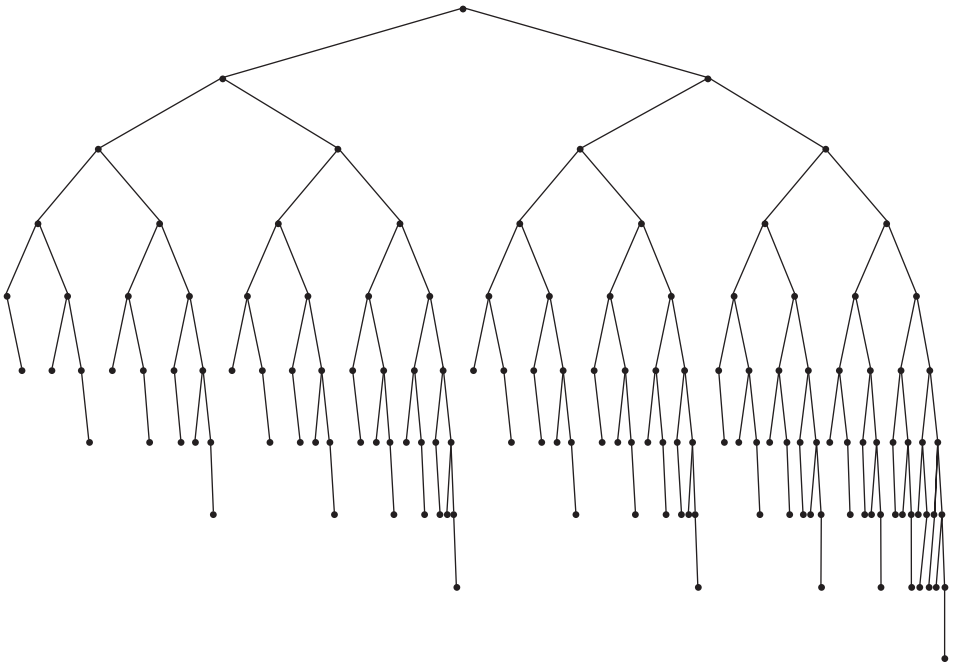


Figure 4.33 Smallest AVL tree of height 9

Let us call the node that must be rebalanced α. Since any node has at most two children, and a height imbalance requires that α's two subtrees' heights differ by two, it is easy to see that a violation might occur in four cases:

1. An insertion into the left subtree of the left child of α
2. An insertion into the right subtree of the left child of α
3. An insertion into the left subtree of the right child of α
4. An insertion into the right subtree of the right child of α

Cases 1 and 4 are mirror image symmetries with respect to α, as are cases 2 and 3. Consequently, as a matter of theory, there are two basic cases. From a programming perspective, of course, there are still four cases.

The first case, in which the insertion occurs on the "outside" (i.e., left–left or right–right), is fixed by a **single rotation** of the tree. The second case, in which the insertion occurs on the "inside" (i.e., left–right or right–left) is handled by the slightly more complex **double rotation.** These are fundamental operations on the tree that we'll see used several times in balanced-tree algorithms. The remainder of this section describes these rotations, proves that they suffice to maintain balance, and gives a casual implementation of the AVL tree. Chapter 12 describes other balanced-tree methods with an eye toward a more careful implementation.

4.4.1 Single Rotation

Figure 4.34 shows the *single rotation* that fixes case 1. The before picture is on the left and the after is on the right. Let us analyze carefully what is going on. Node k_2 violates the AVL balance property because its left subtree is two levels deeper than its right subtree (the dashed lines in the middle of the diagram mark the levels). The situation depicted is the only possible case 1 scenario that allows k_2 to satisfy the AVL property before an insertion but violate it afterwards. Subtree X has grown to an extra level, causing it to be exactly two levels deeper than Z. Y cannot be at the same level as the new X because then k_2 would have been out of balance *before* the insertion, and Y cannot be at the same level as Z because then k_1 would be the first node on the path toward the root that was in violation of the AVL balancing condition.

To ideally rebalance the tree, we would like to move X up a level and Z down a level. Note that this is actually more than the AVL property would require. To do this, we rearrange nodes into an equivalent tree as shown in the second part of Figure 4.34. Here is an abstract scenario: Visualize the tree as being flexible, grab the child node k_1, close your eyes, and shake it, letting gravity take hold. The result is that k_1 will be the new root. The binary search tree property tells us that in the original tree $k_2 > k_1$, so k_2 becomes the right child of k_1 in the new tree. X and Z remain as the left child of k_1 and right child of k_2, respectively. Subtree Y, which holds items that are between k_1 and k_2 in the original tree, can be placed as k_2's left child in the new tree and satisfy all the ordering requirements.

As a result of this work, which requires only a few pointer changes, we have another binary search tree that is an AVL tree. This happens because X moves up one level, Y stays at the same level, and Z moves down one level. k_2 and k_1 not only satisfy the AVL requirements, but they also have subtrees that are exactly the same height. Furthermore, the new height of the entire subtree is *exactly the same* as the height of the original subtree prior to the insertion that caused X to grow. Thus no further updating of heights on the path to the root is needed, and consequently *no further rotations are needed.* Figure 4.35 shows that after the insertion of 6 into the original AVL tree on the left, node 8 becomes unbalanced. Thus, we do a single rotation between 7 and 8, obtaining the tree on the right.

As we mentioned earlier, case 4 represents a symmetric case. Figure 4.36 shows how a single rotation is applied. Let us work through a rather long example. Suppose we start with an initially empty AVL tree and insert the items 3, 2, 1, and then 4 through 7 in sequential order. The first problem occurs when it is time to insert item 1 because the AVL

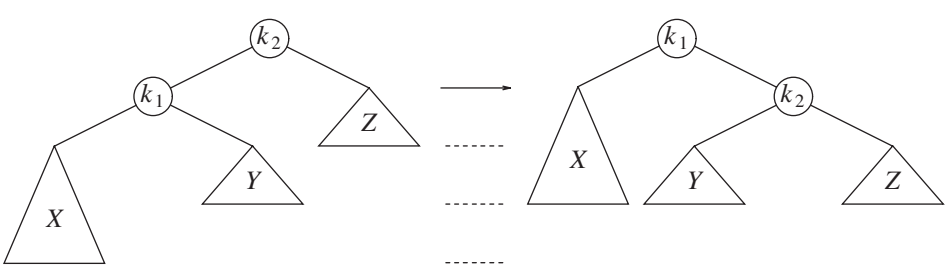


Figure 4.34 Single rotation to fix case 1

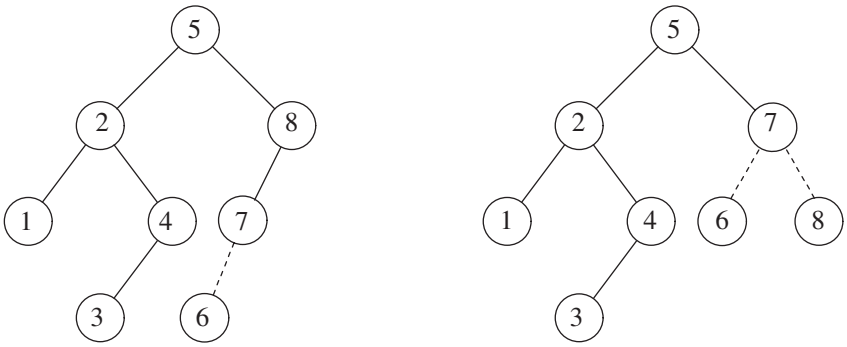


Figure 4.35 AVL property destroyed by insertion of 6, then fixed by a single rotation

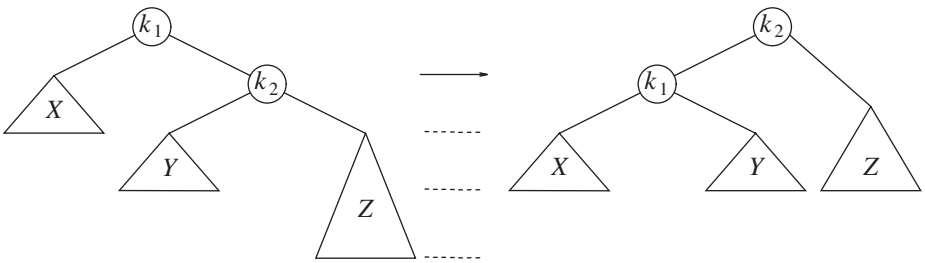
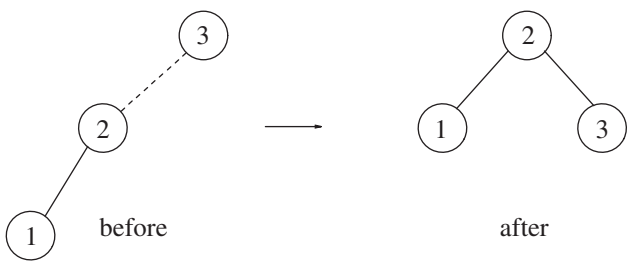
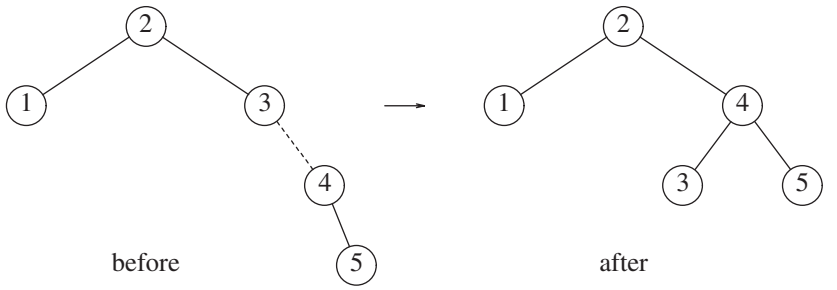


Figure 4.36 Single rotation fixes case 4

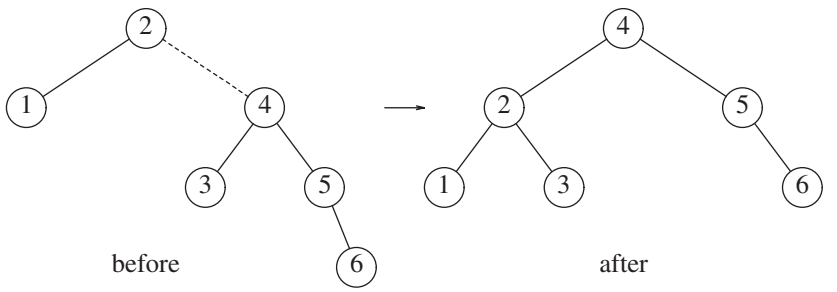
property is violated at the root. We perform a single rotation between the root and its left child to fix the problem. Here are the before and after trees:



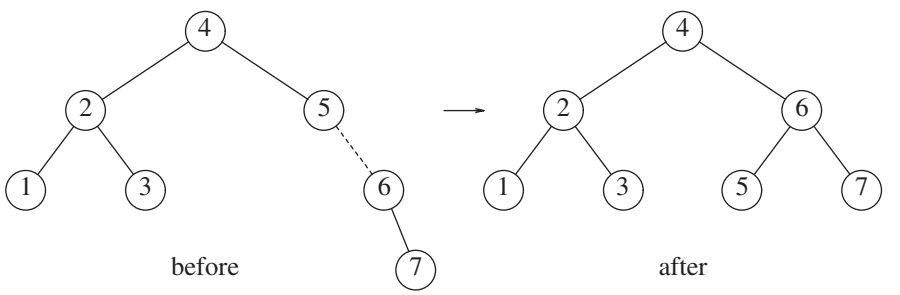
A dashed line joins the two nodes that are the subject of the rotation. Next we insert 4, which causes no problems, but the insertion of 5 creates a violation at node 3 that is fixed by a single rotation. Besides the local change caused by the rotation, the programmer must remember that the rest of the tree has to be informed of this change. Here this means that 2's right child must be reset to link to 4 instead of 3. Forgetting to do so is easy and would destroy the tree (4 would be inaccessible).



Next we insert 6. This causes a balance problem at the root, since its left subtree is of height 0 and its right subtree would be height 2. Therefore, we perform a single rotation at the root between 2 and 4.



The rotation is performed by making 2 a child of 4 and 4's original left subtree the new right subtree of 2. Every item in this subtree must lie between 2 and 4, so this transformation makes sense. The next item we insert is 7, which causes another rotation:



4.4.2 Double Rotation

The algorithm described above has one problem: As Figure 4.37 shows, it does not work for cases 2 or 3. The problem is that subtree *Y* is too deep, and a single rotation does not make it any less deep. The *double rotation* that solves the problem is shown in Figure 4.38.

The fact that subtree *Y* in Figure 4.37 has had an item inserted into it guarantees that it is nonempty. Thus, we may assume that it has a root and two subtrees. Consequently, the

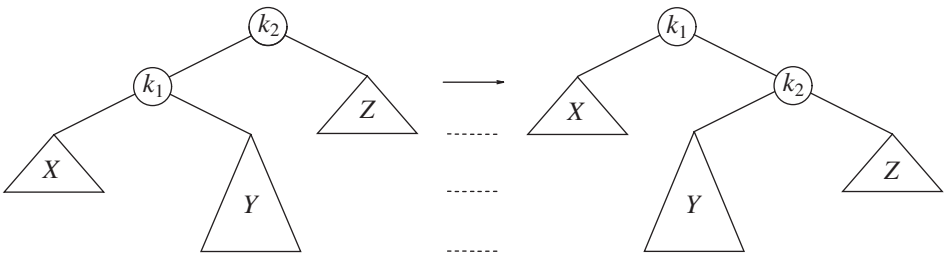


Figure 4.37 Single rotation fails to fix case 2

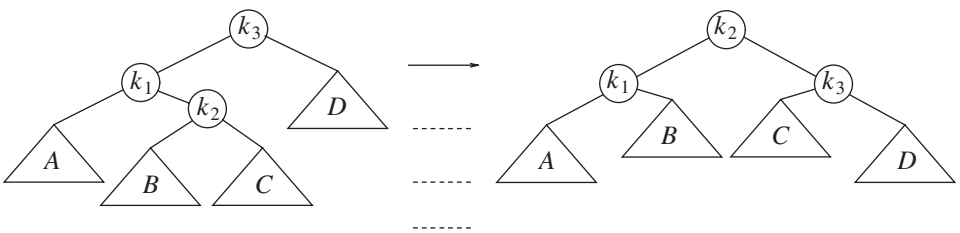


Figure 4.38 Left–right double rotation to fix case 2

tree may be viewed as four subtrees connected by three nodes. As the diagram suggests, exactly one of tree B or C is two levels deeper than D (unless all are empty), but we cannot be sure which one. It turns out not to matter; in Figure 4.38, both B and C are drawn at $1\frac{1}{2}$ levels below D.

To rebalance, we see that we cannot leave k_3 as the root, and a rotation between k_3 and k_1 was shown in Figure 4.37 to not work, so the only alternative is to place k_2 as the new root. This forces k_1 to be k_2's left child and k_3 to be its right child, and it also completely determines the resulting locations of the four subtrees. It is easy to see that the resulting tree satisfies the AVL tree property, and as was the case with the single rotation, it restores the height to what it was before the insertion, thus guaranteeing that all rebalancing and height updating is complete. Figure 4.39 shows that the symmetric case 3 can also be fixed

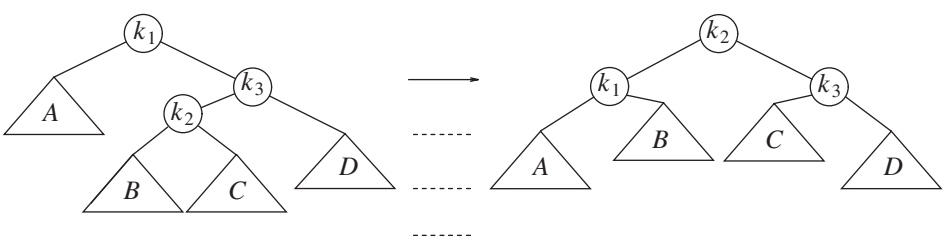
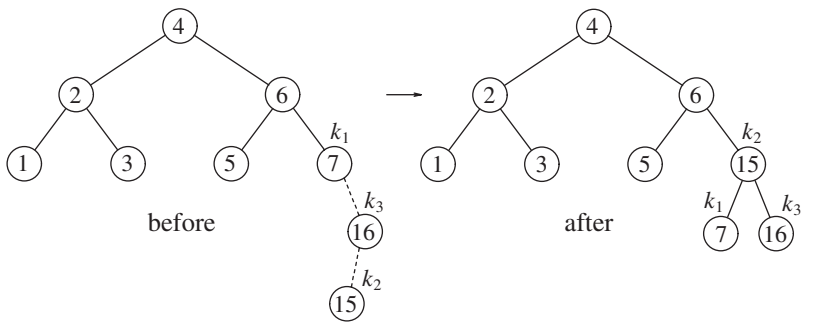


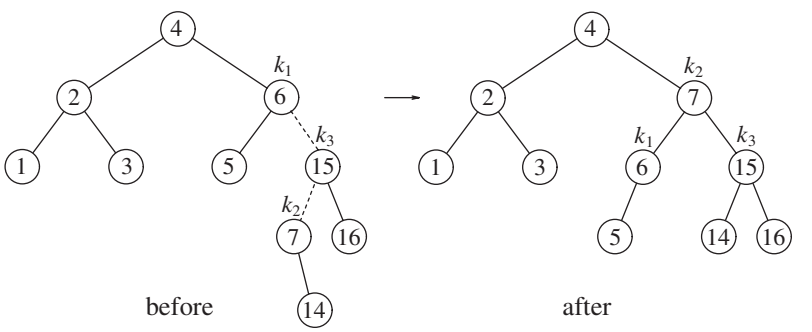
Figure 4.39 Right–left double rotation to fix case 3

by a double rotation. In both cases the effect is the same as rotating between α's child and grandchild, and then between α and its new child.

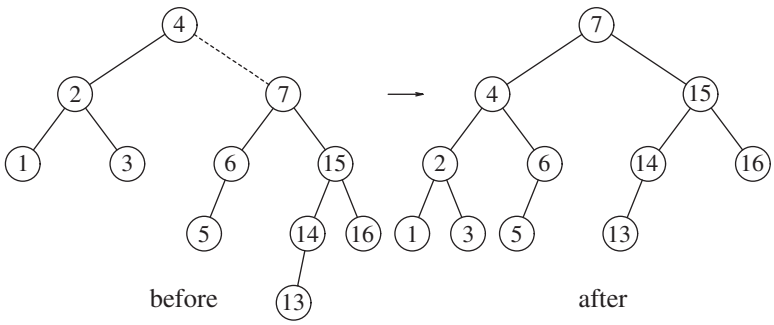
We will continue our previous example by inserting 10 through 16 in reverse order, followed by 8 and then 9. Inserting 16 is easy, since it does not destroy the balance property, but inserting 15 causes a height imbalance at node 7. This is case 3, which is solved by a right–left double rotation. In our example, the right–left double rotation will involve 7, 16, and 15. In this case, k_1 is the node with item 7, k_3 is the node with item 16, and k_2 is the node with item 15. Subtrees *A, B, C,* and *D* are empty.



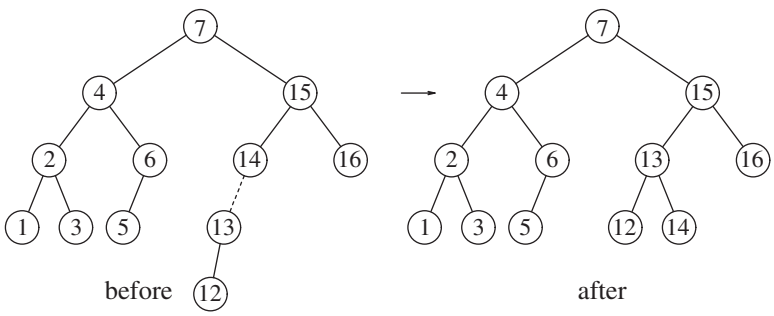
Next we insert 14, which also requires a double rotation. Here the double rotation that will restore the tree is again a right–left double rotation that will involve 6, 15, and 7. In this case, k_1 is the node with item 6, k_2 is the node with item 7, and k_3 is the node with item 15. Subtree *A* is the tree rooted at the node with item 5; subtree *B* is the empty subtree that was originally the left child of the node with item 7, subtree *C* is the tree rooted at the node with item 14, and finally, subtree *D* is the tree rooted at the node with item 16.



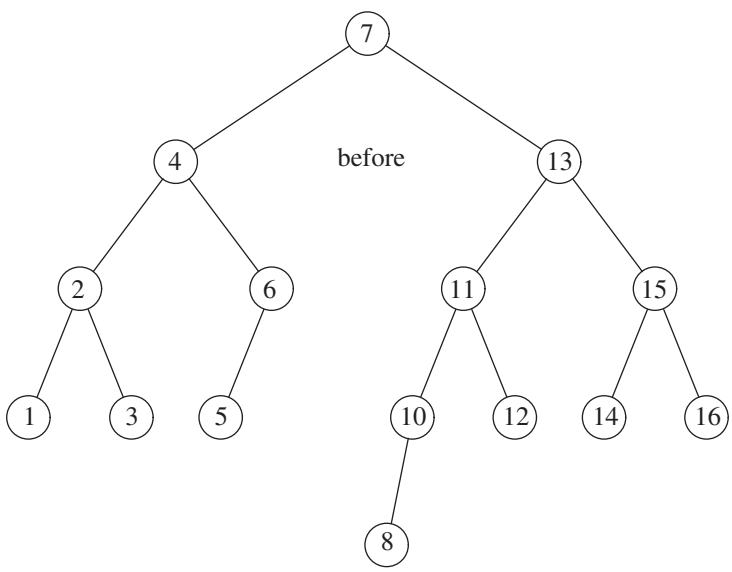
If 13 is now inserted, there is an imbalance at the root. Since 13 is not between 4 and 7, we know that the single rotation will work.



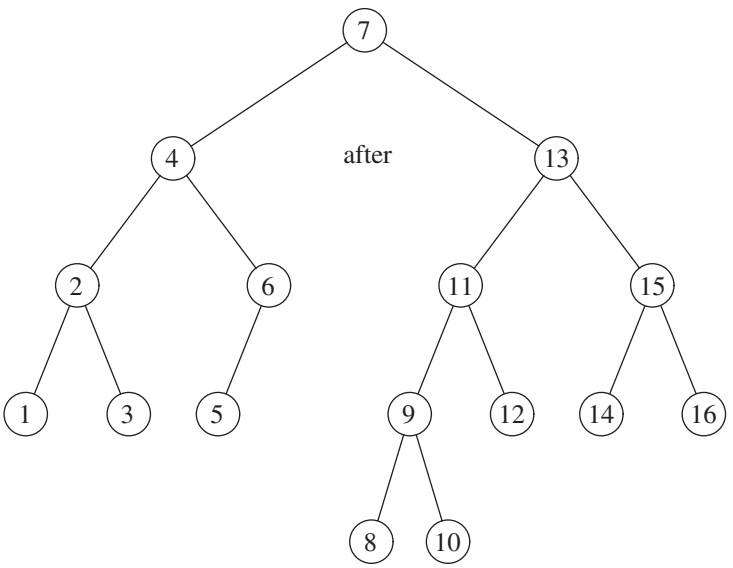
Insertion of 12 will also require a single rotation:



To insert 11, a single rotation needs to be performed, and the same is true for the subsequent insertion of 10. We insert 8 without a rotation, creating an almost perfectly balanced tree:



Finally, we will insert 9 to show the symmetric case of the double rotation. Notice that 9 causes the node containing 10 to become unbalanced. Since 9 is between 10 and 8 (which is 10's child on the path to 9), a double rotation needs to be performed, yielding the following tree:



Let us summarize what happens. The programming details are fairly straightforward except that there are several cases. To insert a new node with item X into an AVL tree T, we recursively insert X into the appropriate subtree of T (let us call this T_{LR}). If the height of T_{LR} does not change, then we are done. Otherwise, if a height imbalance appears in T, we do the appropriate single or double rotation depending on X and the items in T and T_{LR}, update the heights (making the connection from the rest of the tree above), and we are done. Since one rotation always suffices, a carefully coded nonrecursive version generally turns out to be faster than the recursive version, but on modern compilers the difference is not as significant as in the past. However, nonrecursive versions are quite difficult to code correctly, whereas a casual recursive implementation is easily readable.

Another efficiency issue concerns storage of the height information. Since all that is really required is the difference in height, which is guaranteed to be small, we could get by with two bits (to represent $+1$, 0, -1) if we really try. Doing so will avoid repetitive calculation of balance factors but results in some loss of clarity. The resulting code is somewhat more complicated than if the height were stored at each node. If a recursive routine is written, then speed is probably not the main consideration. In this case, the slight speed advantage obtained by storing balance factors hardly seems worth the loss of clarity and relative simplicity. Furthermore, since most machines will align this to at least an 8-bit boundary anyway, there is not likely to be any difference in the amount of space used. An 8-bit (signed) `char` will allow us to store absolute heights of up to 127. Since the tree is balanced, it is inconceivable that this would be insufficient (see the exercises).

```
1  struct AvlNode
2  {
3      Comparable element;
4      AvlNode   *left;
5      AvlNode   *right;
6      int       height;
7
8      AvlNode( const Comparable & ele, AvlNode *lt, AvlNode *rt, int h = 0 )
9        : element{ ele }, left{ lt }, right{ rt }, height{ h } { }
10
11     AvlNode( Comparable && ele, AvlNode *lt, AvlNode *rt, int h = 0 )
12       : element{ std::move( ele ) }, left{ lt }, right{ rt }, height{ h } { }
13 };
```

Figure 4.40 Node declaration for AVL trees

```
1  /**
2   * Return the height of node t or -1 if nullptr.
3   */
4  int height( AvlNode *t ) const
5  {
6      return t == nullptr ? -1 : t->height;
7  }
```

Figure 4.41 Function to compute height of an AVL node

With all this, we are ready to write the AVL routines. We show some of the code here; the rest is online. First, we need the `AvlNode` class. This is given in Figure 4.40. We also need a quick function to return the height of a node. This function is necessary to handle the annoying case of a `nullptr` pointer. This is shown in Figure 4.41. The basic insertion routine (see Figure 4.42) adds only a single line at the end that invokes a balancing method. The balancing method applies a single or double rotation if needed, updates the height, and returns the resulting tree.

For the trees in Figure 4.43, `rotateWithLeftChild` converts the tree on the left to the tree on the right, returning a pointer to the new root. `rotateWithRightChild` is symmetric. The code is shown in Figure 4.44.

Similarly, the double rotation pictured in Figure 4.45 can be implemented by the code shown in Figure 4.46.

Since deletion in a binary search tree is somewhat more complicated than insertion, one can assume that deletion in an AVL tree is also more complicated. In a perfect world, one would hope that the deletion routine in Figure 4.26 could easily be modified by changing the last line to return after calling the `balance` method, as was done for insertion. This would yield the code in Figure 4.47. This change works! A deletion could cause one side

```
1  /**
2   * Internal method to insert into a subtree.
3   * x is the item to insert.
4   * t is the node that roots the subtree.
5   * Set the new root of the subtree.
6   */
7  void insert( const Comparable & x, AvlNode * & t )
8  {
9      if( t == nullptr )
10         t = new AvlNode{ x, nullptr, nullptr };
11     else if( x < t->element )
12         insert( x, t->left );
13     else if( t->element < x )
14         insert( x, t->right );
15
16     balance( t );
17 }
18
19 static const int ALLOWED_IMBALANCE = 1;
20
21 // Assume t is balanced or within one of being balanced
22 void balance( AvlNode * & t )
23 {
24     if( t == nullptr )
25         return;
26
27     if( height( t->left ) - height( t->right ) > ALLOWED_IMBALANCE )
28         if( height( t->left->left ) >= height( t->left->right ) )
29             rotateWithLeftChild( t );
30         else
31             doubleWithLeftChild( t );
32     else
33     if( height( t->right ) - height( t->left ) > ALLOWED_IMBALANCE )
34         if( height( t->right->right ) >= height( t->right->left ) )
35             rotateWithRightChild( t );
36         else
37             doubleWithRightChild( t );
38
39     t->height = max( height( t->left ), height( t->right ) ) + 1;
40 }
```

Figure 4.42 Insertion into an AVL tree

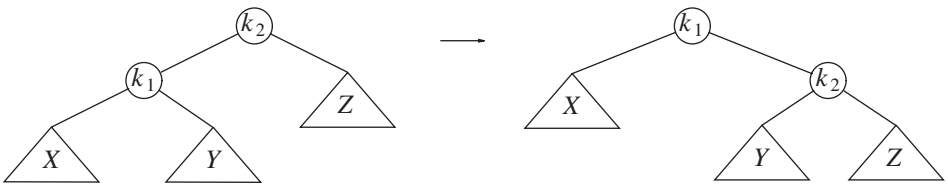


Figure 4.43 Single rotation

```
1   /**
2    * Rotate binary tree node with left child.
3    * For AVL trees, this is a single rotation for case 1.
4    * Update heights, then set new root.
5    */
6   void rotateWithLeftChild( AvlNode * & k2 )
7   {
8       AvlNode *k1 = k2->left;
9       k2->left = k1->right;
10      k1->right = k2;
11      k2->height = max( height( k2->left ), height( k2->right ) ) + 1;
12      k1->height = max( height( k1->left ), k2->height ) + 1;
13      k2 = k1;
14  }
```

Figure 4.44 Routine to perform single rotation

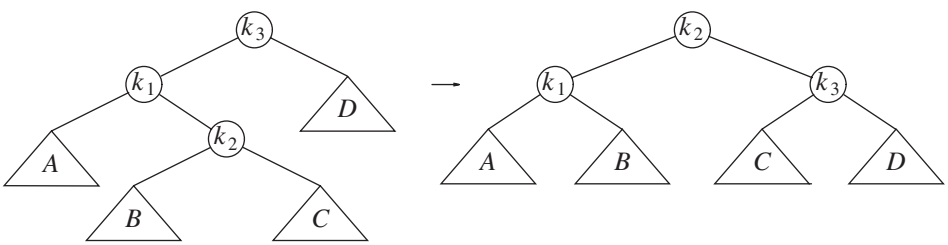


Figure 4.45 Double rotation

of the tree to become two levels shallower than the other side. The case-by-case analysis is similar to the imbalances that are caused by insertion, but not exactly the same. For instance, case 1 in Figure 4.34, which would now reflect a deletion from tree Z (rather than an insertion into X), must be augmented with the possibility that tree Y could be as deep as tree X. Even so, it is easy to see that the rotation rebalances this case and the symmetric case 4 in Figure 4.36. Thus the code for balance in Figure 4.42 lines 28 and 34 uses >=

```
/**
 * Double rotate binary tree node: first left child
 * with its right child; then node k3 with new left child.
 * For AVL trees, this is a double rotation for case 2.
 * Update heights, then set new root.
 */
void doubleWithLeftChild( AvlNode * & k3 )
{
    rotateWithRightChild( k3->left );
    rotateWithLeftChild( k3 );
}
```

Figure 4.46 Routine to perform double rotation

```
/**
 * Internal method to remove from a subtree.
 * x is the item to remove.
 * t is the node that roots the subtree.
 * Set the new root of the subtree.
 */
void remove( const Comparable & x, AvlNode * & t )
{
   if( t == nullptr )
       return;   // Item not found; do nothing

   if( x < t->element )
       remove( x, t->left );
   else if( t->element < x )
       remove( x, t->right );
   else if( t->left != nullptr && t->right != nullptr )  // Two children
   {
       t->element = findMin( t->right  )->element;
       remove( t->element, t->right );
   }
   else
   {
       AvlNode *oldNode = t;
       t = ( t->left != nullptr ) ? t->left : t->right;
       delete oldNode;
   }

    balance( t );
}
```

Figure 4.47 Deletion in an AVL tree

instead of > specifically to ensure that single rotations are done in these cases rather than double rotations. We leave verification of the remaining cases as an exercise.

4.5 Splay Trees

We now describe a relatively simple data structure known as a **splay tree** that guarantees that any M consecutive tree operations starting from an empty tree take at most $O(M \log N)$ time. Although this guarantee does not preclude the possibility that any *single* operation might take $\Theta(N)$ time, and thus the bound is not as strong as an $O(\log N)$ worst-case bound per operation, the net effect is the same: There are no bad input sequences. Generally, when a sequence of M operations has total worst-case running time of $O(Mf(N))$, we say that the **amortized** running time is $O(f(N))$. Thus, a splay tree has an $O(\log N)$ amortized cost per operation. Over a long sequence of operations, some may take more, some less.

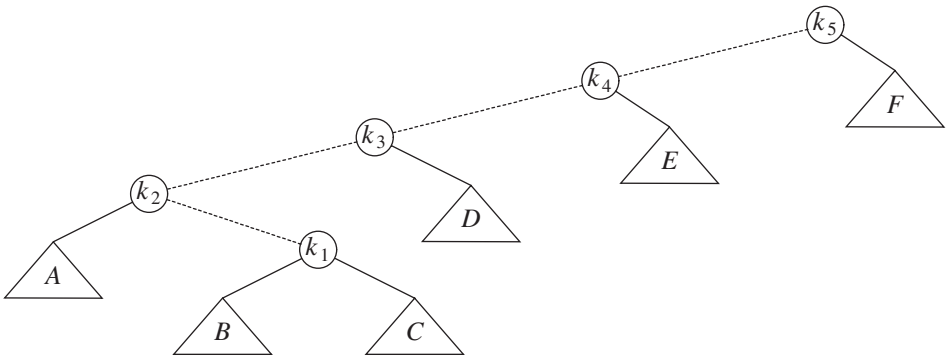
Splay trees are based on the fact that the $O(N)$ worst-case time per operation for binary search trees is not bad, as long as it occurs relatively infrequently. Any one access, even if it takes $\Theta(N)$, is still likely to be extremely fast. The problem with binary search trees is that it is possible, and not uncommon, for a whole sequence of bad accesses to take place. The cumulative running time then becomes noticeable. A search tree data structure with $O(N)$ worst-case time, but a *guarantee* of at most $O(M \log N)$ for any M consecutive operations, is certainly satisfactory, because there are no bad sequences.

If any particular operation is allowed to have an $O(N)$ worst-case time bound, and we still want an $O(\log N)$ amortized time bound, then it is clear that whenever a node is accessed, it must be moved. Otherwise, once we find a deep node, we could keep performing accesses on it. If the node does not change location, and each access costs $\Theta(N)$, then a sequence of M accesses will cost $\Theta(M \cdot N)$.

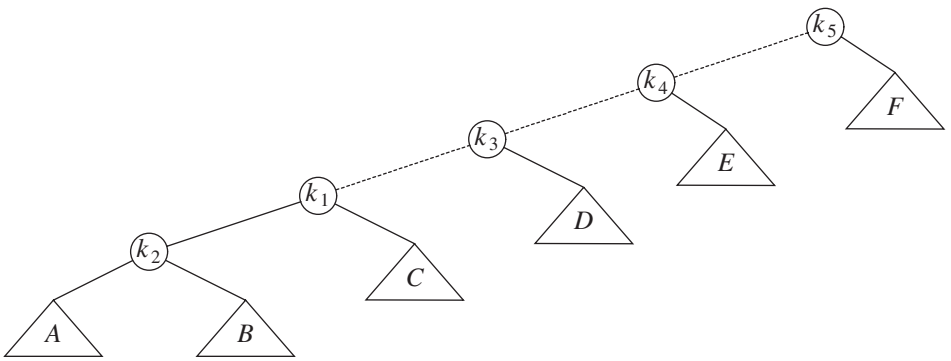
The basic idea of the splay tree is that after a node is accessed, it is pushed to the root by a series of AVL tree rotations. Notice that if a node is deep, there are many nodes on the path that are also relatively deep, and by restructuring we can make future accesses cheaper on all these nodes. Thus, if the node is unduly deep, then we want this restructuring to have the side effect of balancing the tree (to some extent). Besides giving a good time bound in theory, this method is likely to have practical utility, because in many applications, when a node is accessed, it is likely to be accessed again in the near future. Studies have shown that this happens much more often than one would expect. Splay trees also do not require the maintenance of height or balance information, thus saving space and simplifying the code to some extent (especially when careful implementations are written).

4.5.1 A Simple Idea (That Does Not Work)

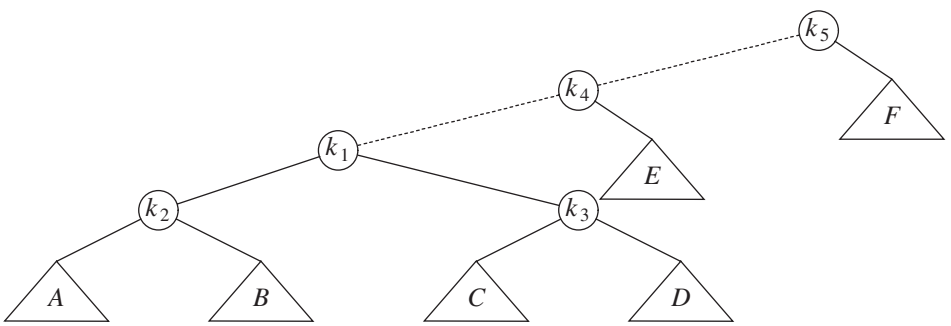
One way of performing the restructuring described above is to perform single rotations, bottom up. This means that we rotate every node on the access path with its parent. As an example, consider what happens after an access (a `find`) on k_1 in the following tree:



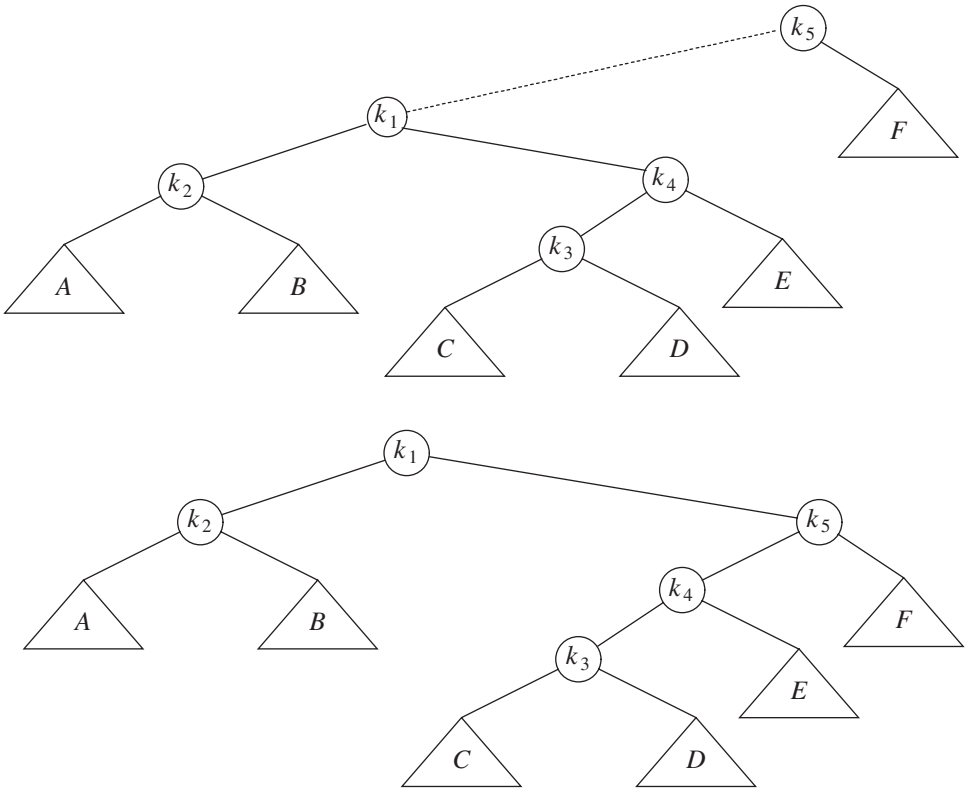
The access path is dashed. First, we would perform a single rotation between k_1 and its parent, obtaining the following tree:



Then, we rotate between k_1 and k_3, obtaining the next tree:



Then two more rotations are performed until we reach the root:



These rotations have the effect of pushing k_1 all the way to the root, so that future accesses on k_1 are easy (for a while). Unfortunately, it has pushed another node (k_3) almost as deep as k_1 used to be. An access on that node will then push another node deep, and so on. Although this strategy makes future accesses of k_1 cheaper, it has not significantly improved the situation for the other nodes on the (original) access path. It turns out that it is possible to prove that using this strategy, there is a sequence of M operations requiring $\Omega(M \cdot N)$ time, so this idea is not quite good enough. The simplest way to show this is to consider the tree formed by inserting keys $1, 2, 3, \ldots, N$ into an initially empty tree (work this example out). This gives a tree consisting of only left children. This is not necessarily bad, though, since the time to build this tree is $O(N)$ total. The bad part is that accessing the node with key 1 takes N units of time, where each node on the access path counts as one unit. After the rotations are complete, an access of the node with key 2 takes N units of time, key 3 takes $N - 1$ units, and so on. The total for accessing all the keys in order is $N + \sum_{i=2}^{N} i = \Omega(N^2)$. After they are accessed, the tree reverts to its original state, and we can repeat the sequence.

4.5.2 Splaying

The splaying strategy is similar to the rotation idea above, except that we are a little more selective about how rotations are performed. We will still rotate bottom up along the access

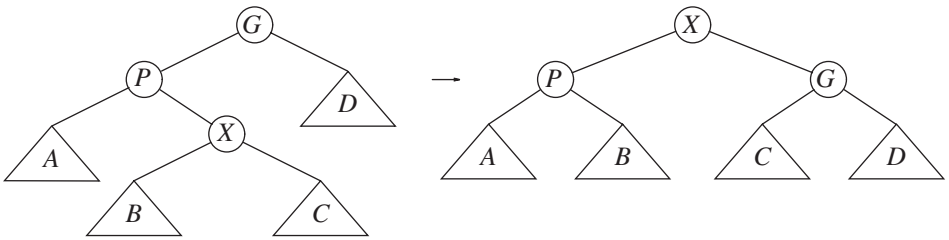


Figure 4.48 Zig-zag

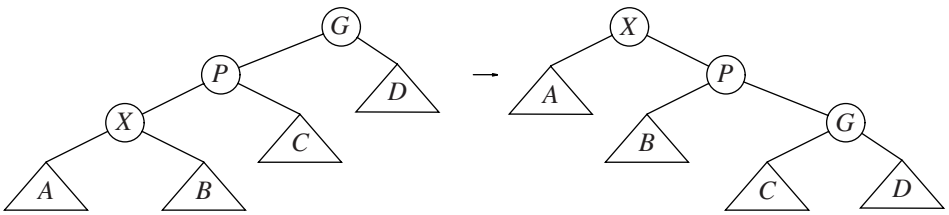
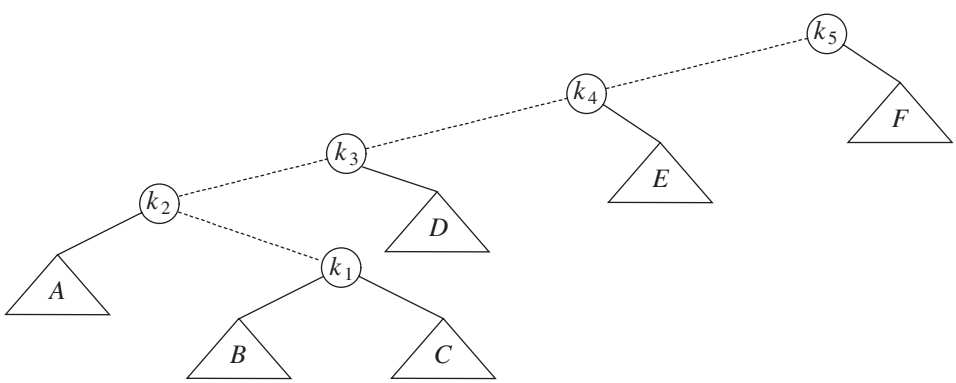


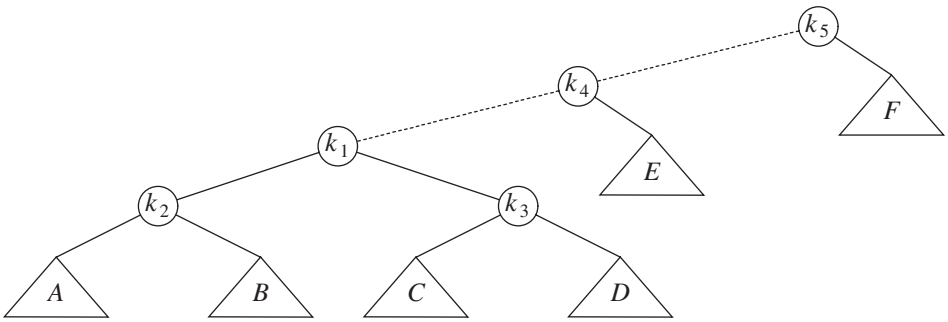
Figure 4.49 Zig-zig

path. Let X be a (non-root) node on the access path at which we are rotating. If the parent of X is the root of the tree, we merely rotate X and the root. This is the last rotation along the access path. Otherwise, X has both a parent (P) and a grandparent (G), and there are two cases, plus symmetries, to consider. The first case is the **zig-zag** case (see Fig. 4.48). Here X is a right child and P is a left child (or vice versa). If this is the case, we perform a double rotation, exactly like an AVL double rotation. Otherwise, we have a **zig-zig** case: X and P are both left children (or, in the symmetric case, both right children). In that case, we transform the tree on the left of Figure 4.49 to the tree on the right.

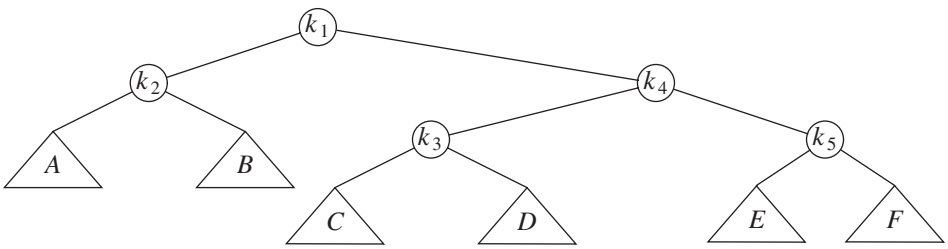
As an example, consider the tree from the last example, with a `contains` on k_1:



The first splay step is at k_1 and is clearly a *zig-zag*, so we perform a standard AVL double rotation using k_1, k_2, and k_3. The resulting tree follows:



The next splay step at k_1 is a *zig-zig*, so we do the *zig-zig* rotation with k_1, k_4, and k_5, obtaining the final tree:



Although it is hard to see from small examples, splaying not only moves the accessed node to the root but also has the effect of roughly halving the depth of most nodes on the access path (some shallow nodes are pushed down at most two levels).

To see the difference that splaying makes over simple rotation, consider again the effect of inserting items $1, 2, 3, \ldots, N$ into an initially empty tree. This takes a total of $O(N)$, as before, and yields the same tree as simple rotations. Figure 4.50 shows the result of splaying at the node with item 1. The difference is that after an access of the node with item 1, which

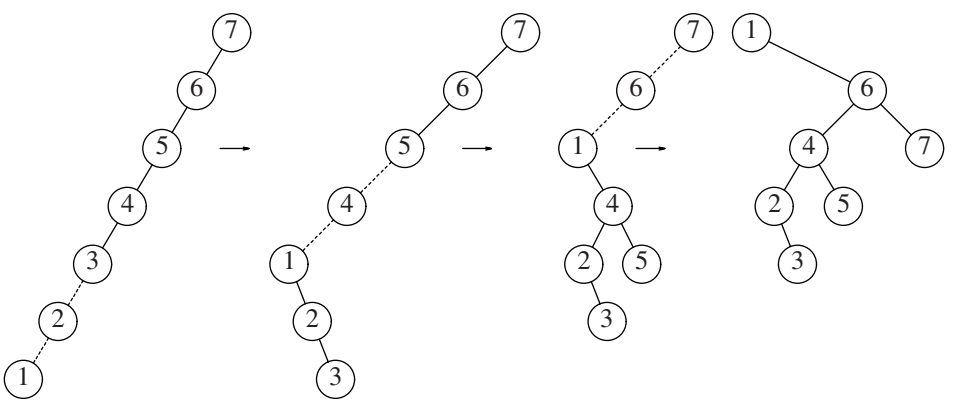


Figure 4.50 Result of splaying at node 1

takes N units, the access on the node with item 2 will only take about $N/2$ units instead of N units; there are no nodes quite as deep as before.

An access on the node with item 2 will bring nodes to within $N/4$ of the root, and this is repeated until the depth becomes roughly $\log N$ (an example with $N = 7$ is too small to see the effect well). Figures 4.51 to 4.59 show the result of accessing items 1 through 9 in a 32-node tree that originally contains only left children. Thus we do not get the same bad behavior from splay trees that is prevalent in the simple rotation strategy. (Actually, this turns out to be a very good case. A rather complicated proof shows that for this example, the N accesses take a total of $O(N)$ time.)

These figures highlight the fundamental and crucial property of splay trees. When access paths are long, thus leading to a longer-than-normal search time, the rotations tend to be good for future operations. When accesses are cheap, the rotations are not as good and can be bad. The extreme case is the initial tree formed by the insertions. All the insertions were constant-time operations leading to a bad initial tree. At that point in time, we had a very bad tree, but we were running ahead of schedule and had the compensation of less total running time. Then a couple of really horrible accesses left a nearly balanced tree, but the cost was that we had to give back some of the time that had been saved. The main theorem, which we will prove in Chapter 11, is that we never fall behind a pace of $O(\log N)$ per operation: We are always on schedule, even though there are occasionally bad operations.

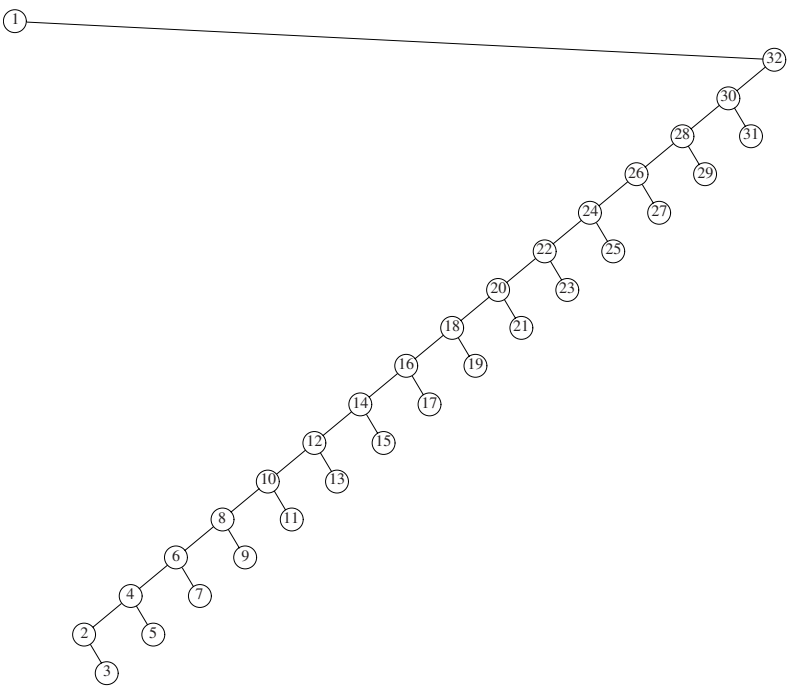


Figure 4.51 Result of splaying at node 1 a tree of all left children

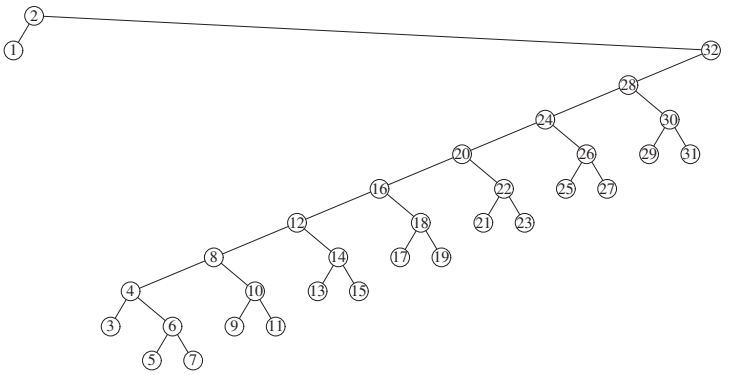


Figure 4.52 Result of splaying the previous tree at node 2

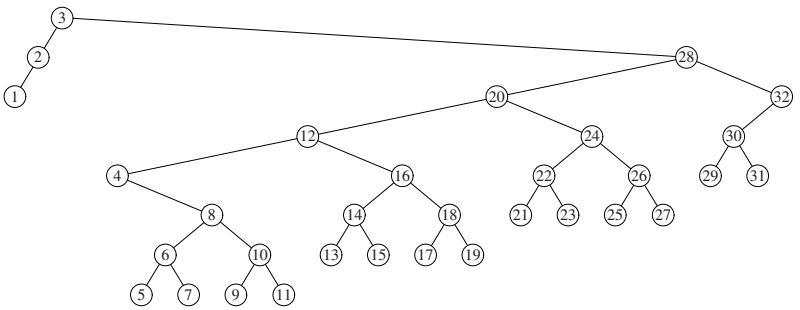


Figure 4.53 Result of splaying the previous tree at node 3

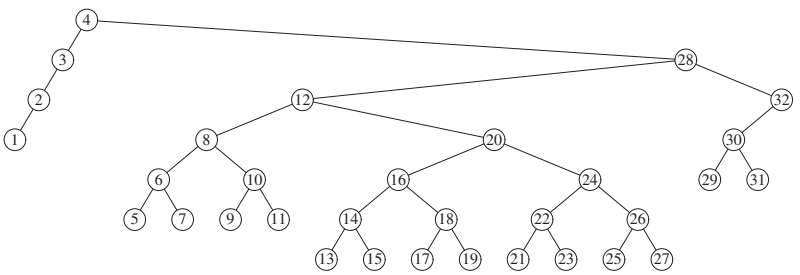


Figure 4.54 Result of splaying the previous tree at node 4

We can perform deletion by accessing the node to be deleted. This puts the node at the root. If it is deleted, we get two subtrees T_L and T_R (left and right). If we find the largest element in T_L (which is easy), then this element is rotated to the root of T_L, and T_L will now have a root with no right child. We can finish the deletion by making T_R the right child.

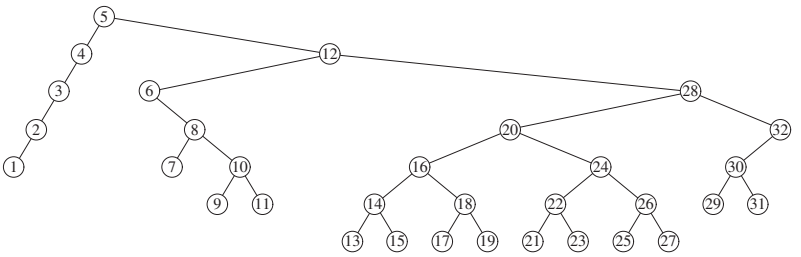


Figure 4.55 Result of splaying the previous tree at node 5

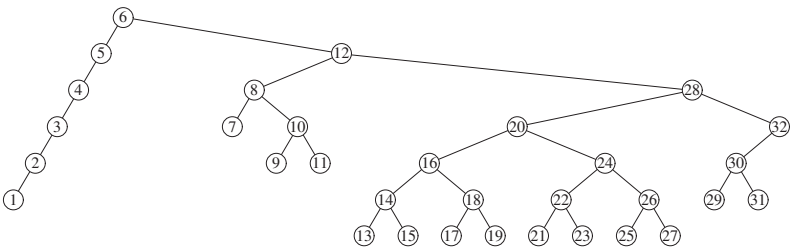


Figure 4.56 Result of splaying the previous tree at node 6

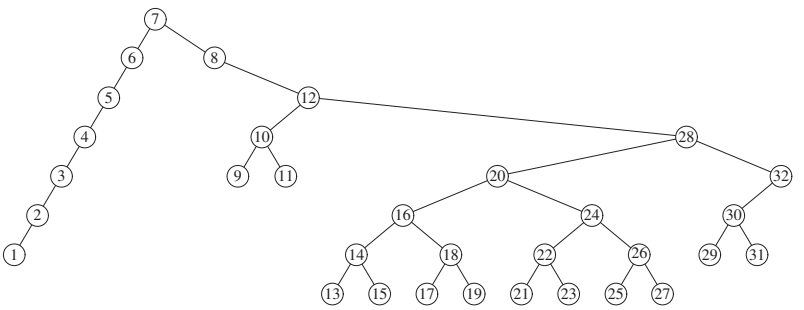


Figure 4.57 Result of splaying the previous tree at node 7

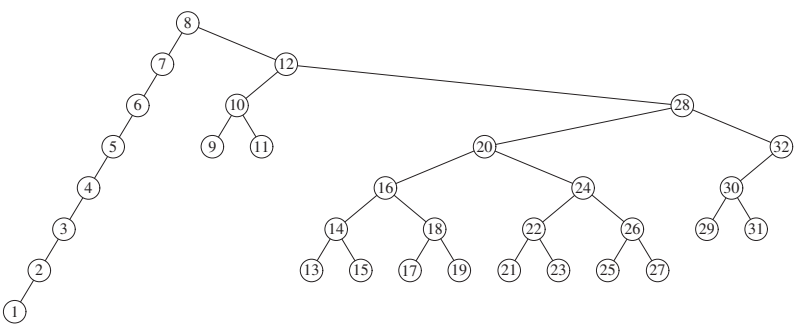


Figure 4.58 Result of splaying the previous tree at node 8

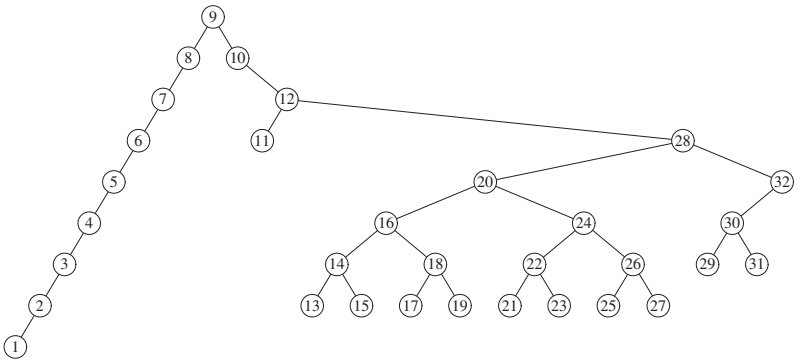


Figure 4.59 Result of splaying the previous tree at node 9

The analysis of splay trees is difficult, because it must take into account the ever-changing structure of the tree. On the other hand, splay trees are much simpler to program than most balanced search trees, since there are fewer cases to consider and no balance information to maintain. Some empirical evidence suggests that this translates into faster code in practice, although the case for this is far from complete. Finally, we point out that there are several variations of splay trees that can perform even better in practice. One variation is completely coded in Chapter 12.

4.6 Tree Traversals (Revisited)

Because of the ordering information in a binary search tree, it is simple to list all the items in sorted order. The recursive function in Figure 4.60 does the real work.

Convince yourself that this function works. As we have seen before, this kind of routine when applied to trees is known as an **inorder traversal** (which makes sense, since it lists the items in order). The general strategy of an inorder traversal is to process the left subtree first, then perform processing at the current node, and finally process the right subtree. The interesting part about this algorithm, aside from its simplicity, is that the total running time is $O(N)$. This is because there is constant work being performed at every node in the tree. Each node is visited once, and the work performed at each node is testing against `nullptr`, setting up two function calls, and doing an output statement. Since there is constant work per node and N nodes, the running time is $O(N)$.

Sometimes we need to process both subtrees first before we can process a node. For instance, to compute the height of a node, we need to know the height of the subtrees first. The code in Figure 4.61 computes this. Since it is always a good idea to check the special cases—and crucial when recursion is involved—notice that the routine will declare the height of a leaf to be zero, which is correct. This general order of traversal, which we have also seen before, is known as a **postorder traversal**. Again, the total running time is $O(N)$, because constant work is performed at each node.

```
/**
 * Print the tree contents in sorted order.
 */
void printTree( ostream & out = cout ) const
{
    if( isEmpty( ) )
        out << "Empty tree" << endl;
    else
        printTree( root, out );
}

/**
 * Internal method to print a subtree rooted at t in sorted order.
 */
void printTree( BinaryNode *t, ostream & out ) const
{
    if( t != nullptr )
    {
        printTree( t->left, out );
        out << t->element << endl;
        printTree( t->right, out );
    }
}
```

Figure 4.60 Routine to print a binary search tree in order

```
/**
 * Internal method to compute the height of a subtree rooted at t.
 */
int height( BinaryNode *t )
{
    if( t == nullptr )
        return -1;
    else
        return 1 + max( height( t->left ), height( t->right ) );
}
```

Figure 4.61 Routine to compute the height of a tree using a postorder traversal

The third popular traversal scheme that we have seen is **preorder traversal**. Here, the node is processed before the children. This could be useful, for example, if you wanted to label each node with its depth.

The common idea in all of these routines is that you handle the `nullptr` case first and then the rest. Notice the lack of extraneous variables. These routines pass only the pointer

to the node that roots the subtree, and do not declare or pass any extra variables. The more compact the code, the less likely that a silly bug will turn up. A fourth, less often used, traversal (which we have not seen yet) is **level-order traversal**. In a level-order traversal, all nodes at depth d are processed before any node at depth $d + 1$. Level-order traversal differs from the other traversals in that it is not done recursively; a queue is used, instead of the implied stack of recursion.

4.7 B-Trees

So far, we have assumed that we can store an entire data structure in the main memory of a computer. Suppose, however, that we have more data than can fit in main memory, and, as a result, must have the data structure reside on disk. When this happens, the rules of the game change, because the Big-Oh model is no longer meaningful.

The problem is that a Big-Oh analysis assumes that all operations are equal. However, this is not true, especially when disk I/O is involved. Modern computers execute billions of instructions per second. That is pretty fast, mainly because the speed depends largely on electrical properties. On the other hand, a disk is mechanical. Its speed depends largely on the time it takes to spin the disk and to move a disk head. Many disks spin at 7,200 RPM. Thus, in 1 min it makes 7,200 revolutions; hence, one revolution occurs in 1/120 of a second, or 8.3 ms. On average, we might expect that we have to spin a disk halfway to find what we are looking for, but this is compensated by the time to move the disk head, so we get an access time of 8.3 ms. (This is a very charitable estimate; 9–11 ms access times are more common.) Consequently, we can do approximately 120 disk accesses per second. This sounds pretty good, until we compare it with the processor speed. What we have is billions instructions equal to 120 disk accesses. Of course, everything here is a rough calculation, but the relative speeds are pretty clear: Disk accesses are incredibly expensive. Furthermore, processor speeds are increasing at a much faster rate than disk speeds (it is disk *sizes* that are increasing quite quickly). So we are willing to do lots of calculations just to save a disk access. In almost all cases, it is the number of disk accesses that will dominate the running time. Thus, if we halve the number of disk accesses, the running time will halve.

Here is how the typical search tree performs on disk: Suppose we want to access the driving records for citizens in the state of Florida. We assume that we have 10,000,000 items, that each key is 32 bytes (representing a name), and that a record is 256 bytes. We assume this does not fit in main memory and that we are 1 of 20 users on a system (so we have 1/20 of the resources). Thus, in 1 sec we can execute many millions of instructions or perform six disk accesses.

The unbalanced binary search tree is a disaster. In the worst case, it has linear depth and thus could require 10,000,000 disk accesses. On average, a successful search would require $1.38 \log N$ disk accesses, and since $\log 10000000 \approx 24$, an average search would require 32 disk accesses, or 5 sec. In a typical randomly constructed tree, we would expect that a few nodes are three times deeper; these would require about 100 disk accesses, or 16 sec. An AVL tree is somewhat better. The worst case of $1.44 \log N$ is unlikely to occur, and the typical case is very close to $\log N$. Thus an AVL tree would use about 25 disk accesses on average, requiring 4 sec.

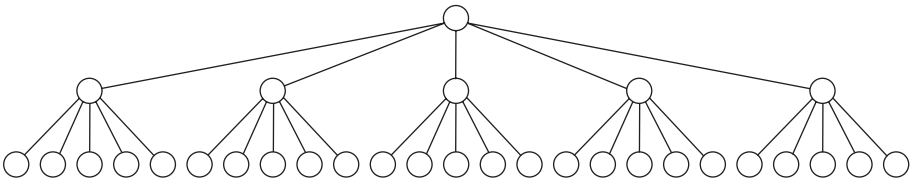


Figure 4.62 5-ary tree of 31 nodes has only three levels

We want to reduce the number of disk accesses to a very small constant, such as three or four. We are willing to write complicated code to do this, because machine instructions are essentially free, as long as we are not ridiculously unreasonable. It should probably be clear that a binary search tree will not work, since the typical AVL tree is close to optimal height. We cannot go below $\log N$ using a binary search tree. The solution is intuitively simple: If we have more branching, we have less height. Thus, while a perfect binary tree of 31 nodes has five levels, a 5-ary tree of 31 nodes has only three levels, as shown in Figure 4.62. An **M-ary search tree** allows M-way branching. As branching increases, the depth decreases. Whereas a complete binary tree has height that is roughly $\log_2 N$, a complete M-ary tree has height that is roughly $\log_M N$.

We can create an M-ary search tree in much the same way as a binary search tree. In a binary search tree, we need one key to decide which of two branches to take. In an M-ary search tree, we need $M - 1$ keys to decide which branch to take. To make this scheme efficient in the worst case, we need to ensure that the M-ary search tree is balanced in some way. Otherwise, like a binary search tree, it could degenerate into a linked list. Actually, we want an even more restrictive balancing condition. That is, we do not want an M-ary search tree to degenerate to even a binary search tree, because then we would be stuck with $\log N$ accesses.

One way to implement this is to use a **B-tree.** The basic B-tree[3] is described here. Many variations and improvements are known, and an implementation is somewhat complex because there are quite a few cases. However, it is easy to see that, in principle, a B-tree guarantees only a few disk accesses.

A B-tree of order M is an M-ary tree with the following properties:[4]

1. The data items are stored at leaves.
2. The nonleaf nodes store up to $M - 1$ keys to guide the searching; key i represents the smallest key in subtree $i + 1$.
3. The root is either a leaf or has between two and M children.
4. All nonleaf nodes (except the root) have between $\lceil M/2 \rceil$ and M children.
5. All leaves are at the same depth and have between $\lceil L/2 \rceil$ and L data items, for some L (the determination of L is described shortly).

[3] What is described is popularly known as a B^+ tree.

[4] Rules 3 and 5 must be relaxed for the first L insertions.

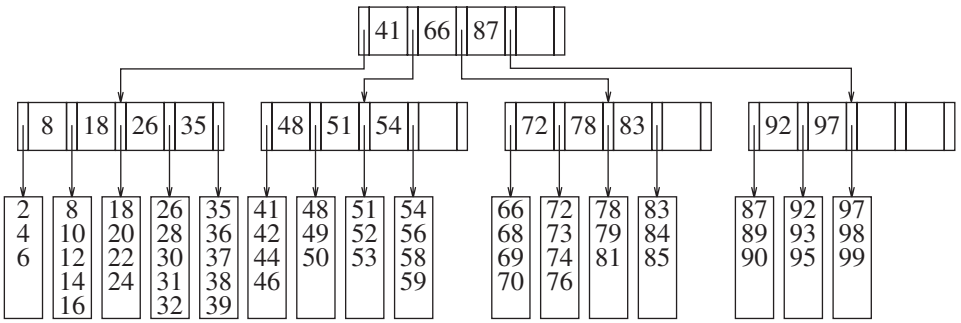


Figure 4.63 B-tree of order 5

An example of a B-tree of order 5 is shown in Figure 4.63. Notice that all nonleaf nodes have between three and five children (and thus between two and four keys); the root could possibly have only two children. Here, we have $L = 5$. It happens that L and M are the same in this example, but this is not necessary. Since L is 5, each leaf has between three and five data items. Requiring nodes to be half full guarantees that the B-tree does not degenerate into a simple binary tree. Although there are various definitions of B-trees that change this structure, mostly in minor ways, this definition is one of the popular forms.

Each node represents a disk block, so we choose M and L on the basis of the size of the items that are being stored. As an example, suppose one block holds 8,192 bytes. In our Florida example, each key uses 32 bytes. In a B-tree of order M, we would have $M-1$ keys, for a total of $32M - 32$ bytes, plus M branches. Since each branch is essentially a number of another disk block, we can assume that a branch is 4 bytes. Thus the branches use $4M$ bytes. The total memory requirement for a nonleaf node is thus $36M-32$. The largest value of M for which this is no more than 8,192 is 228. Thus we would choose $M = 228$. Since each data record is 256 bytes, we would be able to fit 32 records in a block. Thus we would choose $L = 32$. We are guaranteed that each leaf has between 16 and 32 data records and that each internal node (except the root) branches in at least 114 ways. Since there are 10,000,000 records, there are, at most, 625,000 leaves. Consequently, in the worst case, leaves would be on level 4. In more concrete terms, the worst-case number of accesses is given by approximately $\log_{M/2} N$, give or take 1. (For example, the root and the next level could be cached in main memory, so that over the long run, disk accesses would be needed only for level 3 and deeper.)

The remaining issue is how to add and remove items from the B-tree. The ideas involved are sketched next. Note that many of the themes seen before recur.

We begin by examining insertion. Suppose we want to insert 57 into the B-tree in Figure 4.63. A search down the tree reveals that it is not already in the tree. We can add it to the leaf as a fifth item. Note that we may have to reorganize all the data in the leaf to do this. However, the cost of doing this is negligible when compared to that of the disk access, which in this case also includes a disk write.

Of course, that was relatively painless, because the leaf was not already full. Suppose we now want to insert 55. Figure 4.64 shows a problem: The leaf where 55 wants to go is already full. The solution is simple: Since we now have $L+1$ items, we split them into two

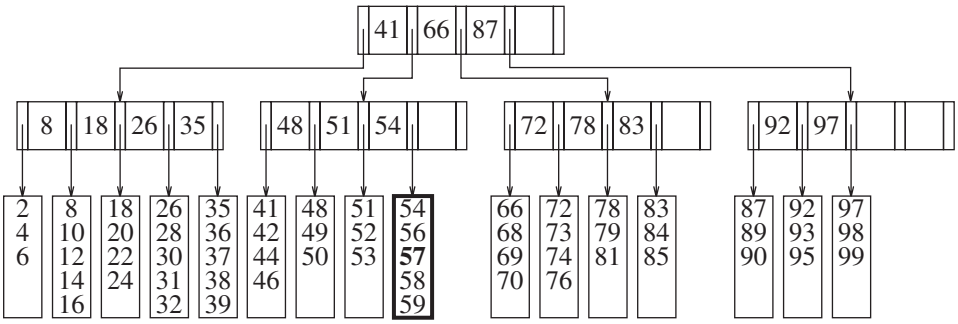


Figure 4.64 B-tree after insertion of 57 into the tree in Figure 4.63

leaves, both guaranteed to have the minimum number of data records needed. We form two leaves with three items each. Two disk accesses are required to write these leaves, and a third disk access is required to update the parent. Note that in the parent, both keys and branches change, but they do so in a controlled way that is easily calculated. The resulting B-tree is shown in Figure 4.65. Although splitting nodes is time-consuming because it requires at least two additional disk writes, it is a relatively rare occurrence. If L is 32, for example, then when a node is split, two leaves with 16 and 17 items, respectively, are created. For the leaf with 17 items, we can perform 15 more insertions without another split. Put another way, for every split, there are roughly $L/2$ nonsplits.

The node splitting in the previous example worked because the parent did not have its full complement of children. But what would happen if it did? Suppose, for example, that we insert 40 into the B-tree in Figure 4.65. We must split the leaf containing the keys 35 through 39, and now 40, into two leaves. But doing this would give the parent six children, and it is allowed only five. The solution is to split the parent. The result of this is shown in Figure 4.66. When the parent is split, we must update the values of the keys and also the parent's parent, thus incurring an additional two disk writes (so this insertion costs five disk writes). However, once again, the keys change in a very controlled manner, although the code is certainly not simple because of a host of cases.

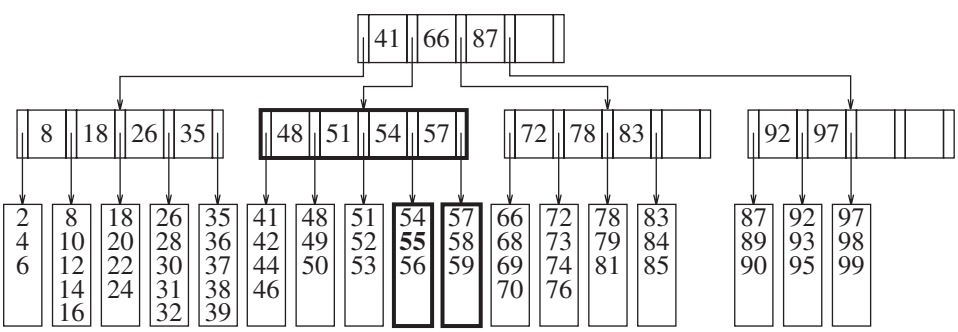


Figure 4.65 Insertion of 55 into the B-tree in Figure 4.64 causes a split into two leaves

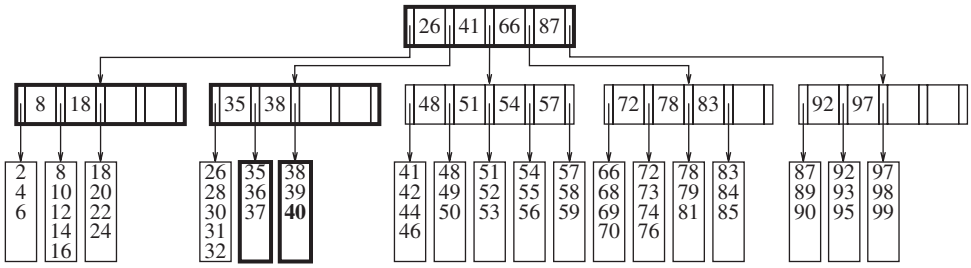


Figure 4.66 Insertion of 40 into the B-tree in Figure 4.65 causes a split into two leaves and then a split of the parent node

When a nonleaf node is split, as is the case here, its parent gains a child. What if the parent already has reached its limit of children? Then we continue splitting nodes up the tree until either we find a parent that does not need to be split or we reach the root. If we split the root, then we have two roots. Obviously, this is unacceptable, but we can create a new root that has the split roots as its two children. This is why the root is granted the special two-child minimum exemption. It also is the only way that a B-tree gains height. Needless to say, splitting all the way up to the root is an exceptionally rare event. This is because a tree with four levels indicates that the root has been split three times throughout the entire sequence of insertions (assuming no deletions have occurred). In fact, the splitting of any nonleaf node is also quite rare.

There are other ways to handle the overflowing of children. One technique is to put a child up for adoption should a neighbor have room. To insert 29 into the B-tree in Figure 4.66, for example, we could make room by moving 32 to the next leaf. This technique requires a modification of the parent, because the keys are affected. However, it tends to keep nodes fuller and saves space in the long run.

We can perform deletion by finding the item that needs to be removed and then removing it. The problem is that if the leaf it was in had the minimum number of data items, then it is now below the minimum. We can rectify this situation by adopting a neighboring item, if the neighbor is not itself at its minimum. If it is, then we can combine with the neighbor to form a full leaf. Unfortunately, this means that the parent has lost a child. If this causes the parent to fall below its minimum, then it follows the same strategy. This process could

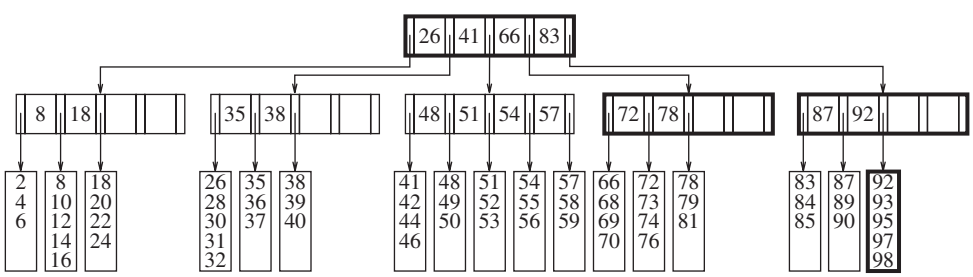


Figure 4.67 B-tree after the deletion of 99 from the B-tree in Figure 4.66

percolate all the way up to the root. The root cannot have just one child (and even if this were allowed, it would be silly). If a root is left with one child as a result of the adoption process, then we remove the root and make its child the new root of the tree. This is the only way for a B-tree to lose height. For example, suppose we want to remove 99 from the B-tree in Figure 4.66. Since the leaf has only two items and its neighbor is already at its minimum of three, we combine the items into a new leaf of five items. As a result, the parent has only two children. However, it can adopt from a neighbor, because the neighbor has four children. As a result, both have three children. The result is shown in Figure 4.67.

4.8 Sets and Maps in the Standard Library

The STL containers discussed in Chapter 3—namely, `vector` and `list`—are inefficient for searching. Consequently, the STL provides two additional containers, `set` and `map`, that guarantee logarithmic cost for basic operations such as insertion, deletion, and searching.

4.8.1 Sets

The `set` is an ordered container that does not allow duplicates. Many of the idioms used to access items in `vector` and `list` also work for a `set`. Specifically, nested in the `set` are `iterator` and `const_iterator` types that allow traversal of the `set`, and several methods from `vector` and `list` are identically named in `set`, including `begin`, `end`, `size`, and `empty`. The `print` function template described in Figure 3.6 will work if passed a `set`.

The unique operations required by the `set` are the abilities to insert, remove, and perform a basic search (efficiently).

The insert routine is aptly named `insert`. However, because a `set` does not allow duplicates, it is possible for the `insert` to fail. As a result, we want the return type to be able to indicate this with a Boolean variable. However, `insert` has a more complicated return type than a `bool`. This is because `insert` also returns an `iterator` that represents where `x` is when `insert` returns. This `iterator` represents either the newly inserted item or the existing item that caused the `insert` to fail, and it is useful, because knowing the position of the item can make removing it more efficient by avoiding the search and getting directly to the node containing the item.

The STL defines a class template called `pair` that is little more than a `struct` with members `first` and `second` to access the two items in the `pair`. There are two different `insert` routines:

```
pair<iterator,bool> insert( const Object & x );
pair<iterator,bool> insert( iterator hint, const Object & x );
```

The one-parameter `insert` behaves as described above. The two-parameter `insert` allows the specification of a hint, which represents the position where `x` should go. If the hint is accurate, the insertion is fast, often $O(1)$. If not, the insertion is done using the normal insertion algorithm and performs comparably with the one-parameter `insert`. For instance, the following code might be faster using the two-parameter `insert` rather than the one-parameter `insert`:

```
set<int> s;
for( int i = 0; i < 1000000; ++i )
    s.insert( s.end( ), i );
```

There are several versions of `erase`:

```
int erase( const Object & x );
iterator erase( iterator itr );
iterator erase( iterator start, iterator end );
```

The first one-parameter `erase` removes `x` (if found) and returns the number of items actually removed, which is obviously either 0 or 1. The second one-parameter `erase` behaves the same as in `vector` and `list`. It removes the object at the position given by the `iterator`, returns an `iterator` representing the element that followed `itr` immediately prior to the call to `erase`, and invalidates `itr`, which becomes stale. The two-parameter `erase` behaves the same as in a `vector` or `list`, removing all the items starting at `start`, up to but not including the item at `end`.

For searching, rather than a `contains` routine that returns a Boolean variable, the `set` provides a `find` routine that returns an `iterator` representing the location of the item (or the endmarker if the search fails). This provides considerably more information, at no cost in running time. The signature of `find` is

```
iterator find( const Object & x ) const;
```

By default, ordering uses the `less<Object>` function object, which itself is implemented by invoking `operator<` for the `Object`. An alternative ordering can be specified by instantiating the `set` template with a function object type. For instance, we can create a `set` that stores `string` objects, ignoring case distinctions by using the `CaseInsensitiveCompare` function object coded in Figure 1.25. In the following code, the `set s` has size 1:

```
set<string,CaseInsensitiveCompare> s;
s.insert( "Hello" ); s.insert( "HeLLo" );
cout << "The size is: " << s.size( ) << endl;
```

4.8.2 Maps

A `map` is used to store a collection of ordered entries that consists of keys and their values. Keys must be unique, but several keys can map to the same values. Thus values need not be unique. The keys in the `map` are maintained in logically sorted order.

The `map` behaves like a `set` instantiated with a `pair`, whose comparison function refers only to the key.[5] Thus it supports `begin`, `end`, `size`, and `empty`, but the underlying iterator is a key-value pair. In other words, for an `iterator itr`, `*itr` is of type `pair<KeyType,ValueType>`. The `map` also supports `insert`, `find`, and `erase`. For `insert`, one must provide a `pair<KeyType,ValueType>` object. Although `find` requires only a key, the

[5] Like a `set`, an optional template parameter can be used to specify a comparison function that differs from `less<KeyType>`.

iterator it returns references a pair. Using only these operations is often not worthwhile because the syntactic baggage can be expensive.

Fortunately, the map has an important extra operation that yields simple syntax. The array-indexing operator is overloaded for maps as follows:

```
ValueType & operator[] ( const KeyType & key );
```

The semantics of operator[] are as follows. If key is present in the map, a reference to the corresponding value is returned. If key is not present in the map, it is inserted with a default value into the map and then a reference to the inserted default value is returned. The default value is obtained by applying a zero-parameter constructor or is zero for the primitive types. These semantics do not allow an accessor version of operator[], so operator[] cannot be used on a map that is constant. For instance, if a map is passed by constant reference, inside the routine, operator[] is unusable.

The code snippet in Figure 4.68 illustrates two techniques to access items in a map. First, observe that at line 3, the left-hand side invokes operator[], thus inserting "Pat" and a double of value 0 into the map, and returning a reference to that double. Then the assignment changes that double inside the map to 75000. Line 4 outputs 75000. Unfortunately, line 5 inserts "Jan" and a salary of 0.0 into the map and then prints it. This may or may not be the proper thing to do, depending on the application. If it is important to distinguish between items that are in the map and those not in the map, or if it is important not to insert into the map (because it is immutable), then an alternate approach shown at lines 7 to 12 can be used. There we see a call to find. If the key is not found, the iterator is the endmarker and can be tested. If the key is found, we can access the second item in the pair referenced by the iterator, which is the value associated with the key. We could also assign to itr->second if, instead of a const_iterator, itr is an iterator.

4.8.3 Implementation of set and map

C++ requires that set and map support the basic insert, erase, and find operations in logarithmic worst-case time. Consequently, the underlying implementation is a balanced

```
1       map<string,double> salaries;
2
3       salaries[ "Pat" ] = 75000.00;
4       cout << salaries[ "Pat" ] << endl;
5       cout << salaries[ "Jan" ] << endl;
6
7       map<string,double>::const_iterator itr;
8       itr = salaries.find( "Chris" );
9       if( itr == salaries.end( ) )
10          cout << "Not an employee of this company!" << endl;
11      else
12          cout << itr->second << endl;
```

Figure 4.68 Accessing values in a map

binary search tree. Typically, an AVL tree is not used; instead, top-down red-black trees, which are discussed in Section 12.2, are often used.

An important issue in implementing `set` and `map` is providing support for the iterator classes. Of course, internally, the iterator maintains a pointer to the "current" node in the iteration. The hard part is efficiently advancing to the next node. There are several possible solutions, some of which are listed here:

1. When the iterator is constructed, have each iterator store as its data an array containing the `set` items. This doesn't work: It makes it impossible to efficiently implement any of the routines that return an iterator after modifying the `set`, such as some of the versions of `erase` and `insert`.
2. Have the iterator maintain a stack storing nodes on the path to the current node. With this information, one can deduce the next node in the iteration, which is either the node in the current node's right subtree that contains the minimum item or the nearest ancestor that contains the current node in its left subtree. This makes the iterator somewhat large and makes the iterator code clumsy.
3. Have each node in the search tree store its parent in addition to the children. The iterator is not as large, but there is now extra memory required in each node, and the code to iterate is still clumsy.
4. Have each node maintain extra links: one to the next smaller, and one to the next larger node. This takes space, but the iteration is very simple to do, and it is easy to maintain these links.
5. Maintain the extra links only for nodes that have `nullptr` left or right links by using extra Boolean variables to allow the routines to tell if a left link is being used as a standard binary search tree left link or a link to the next smaller node, and similarly for the right link (Exercise 4.49). This idea is called a **threaded tree** and is used in many of the STL implementations.

4.8.4 An Example That Uses Several Maps

Many words are similar to other words. For instance, by changing the first letter, the word `wine` can become `dine`, `fine`, `line`, `mine`, `nine`, `pine`, or `vine`. By changing the third letter, `wine` can become `wide`, `wife`, `wipe`, or `wire`, among others. By changing the fourth letter, `wine` can become `wind`, `wing`, `wink`, or `wins`, among others. This gives 15 different words that can be obtained by changing only one letter in `wine`. In fact, there are over 20 different words, some more obscure. We would like to write a program to find all words that can be changed into at least 15 other words by a single one-character substitution. We assume that we have a dictionary consisting of approximately 89,000 different words of varying lengths. Most words are between 6 and 11 characters. The distribution includes 8,205 six-letter words, 11,989 seven-letter words, 13,672 eight-letter words, 13,014 nine-letter words, 11,297 ten-letter words, and 8,617 eleven-letter words. (In reality, the most changeable words are three-, four-, and five-letter words, but the longer words are the time-consuming ones to check.)

The most straightforward strategy is to use a `map` in which the keys are words and the values are vectors containing the words that can be changed from the key with a

```
1   void printHighChangeables( const map<string,vector<string>> & adjacentWords,
2                              int minWords = 15 )
3   {
4       for( auto & entry : adjacentWords )
5       {
6           const vector<string> & words = entry.second;
7
8           if( words.size( ) >= minWords )
9           {
10              cout << entry.first << " (" << words.size( ) << "):";
11              for( auto & str : words )
12                  cout << " " << str;
13              cout << endl;
14          }
15      }
16  }
```

Figure 4.69 Given a `map` containing words as keys and a `vector` of words that differ in only one character as values, output words that have `minWords` or more words obtainable by a one-character substitution

one-character substitution. The routine in Figure 4.69 shows how the `map` that is eventually produced (we have yet to write code for that part) can be used to print the required answers. The code uses a range `for` loop to step through the `map` and views entries that are pairs consisting of a word and a vector of words. The constant references at lines 4 and 6 are used to replace complicated expressions and avoid making unneeded copies.

The main issue is how to construct the `map` from an array that contains the 89,000 words. The routine in Figure 4.70 is a straightforward function to test if two words are identical except for a one-character substitution. We can use the routine to provide the simplest algorithm for the `map` construction, which is a brute-force test of all pairs of words. This algorithm is shown in Figure 4.71.

If we find a pair of words that differ in only one character, we can update the `map` at lines 12 and 13. The idiom we are using at line 12 is that `adjWords[str]` represents the `vector` of words that are identical to `str`, except for one character. If we have previously seen `str`, then it is in the `map`, and we need only add the new word to the `vector` in the `map`, and we do this by calling `push_back`. If we have never seen `str` before, then the act of using `operator[]` places it in the `map`, with a `vector` of size 0, and returns this `vector`, so the `push_back` updates the `vector` to be size 1. All in all, a super-slick idiom for maintaining a `map` in which the value is a collection.

The problem with this algorithm is that it is slow and takes 97 seconds on our computer. An obvious improvement is to avoid comparing words of different lengths. We can do this by grouping words by their length, and then running the previous algorithm on each of the separate groups.

To do this, we can use a second `map`! Here the key is an integer representing a word length, and the value is a collection of all the words of that length. We can use a `vector` to